Attention mechanisms Transformer

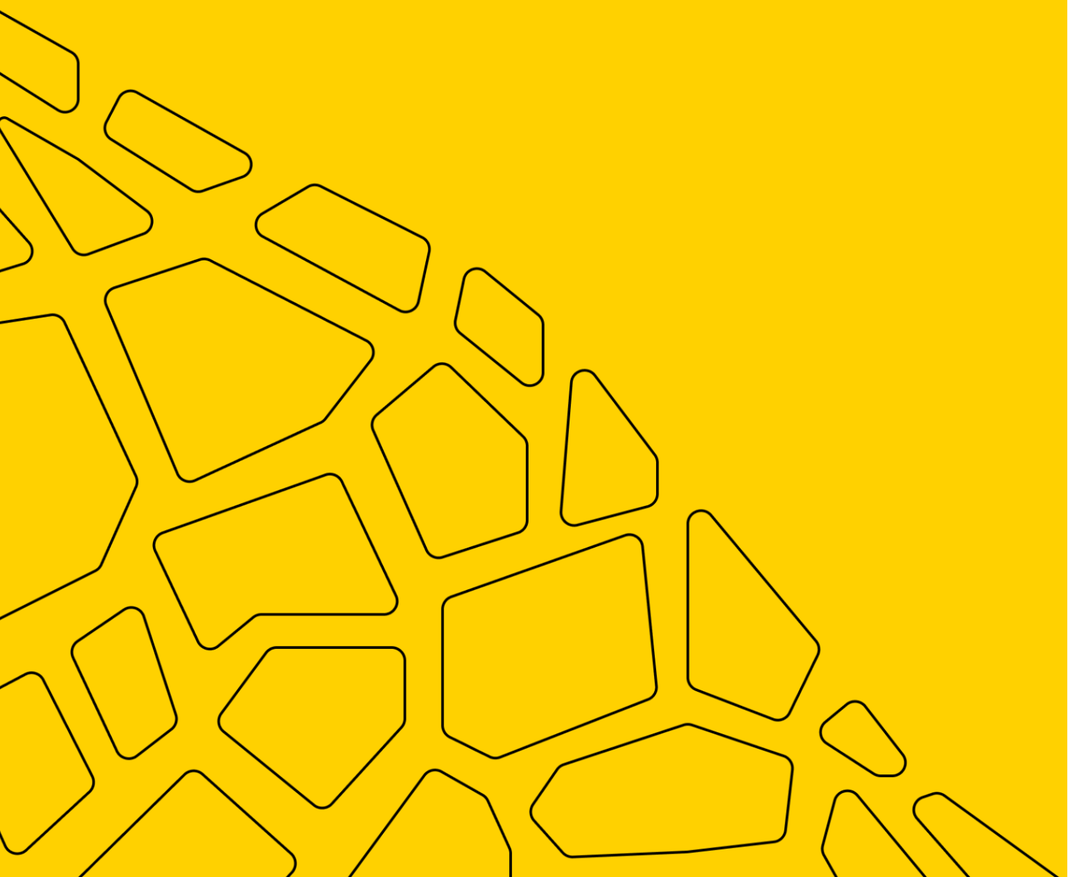
Vladislav Goncharenko
Senior researcher

girafe ai

MIPT, fall 2025

Outline

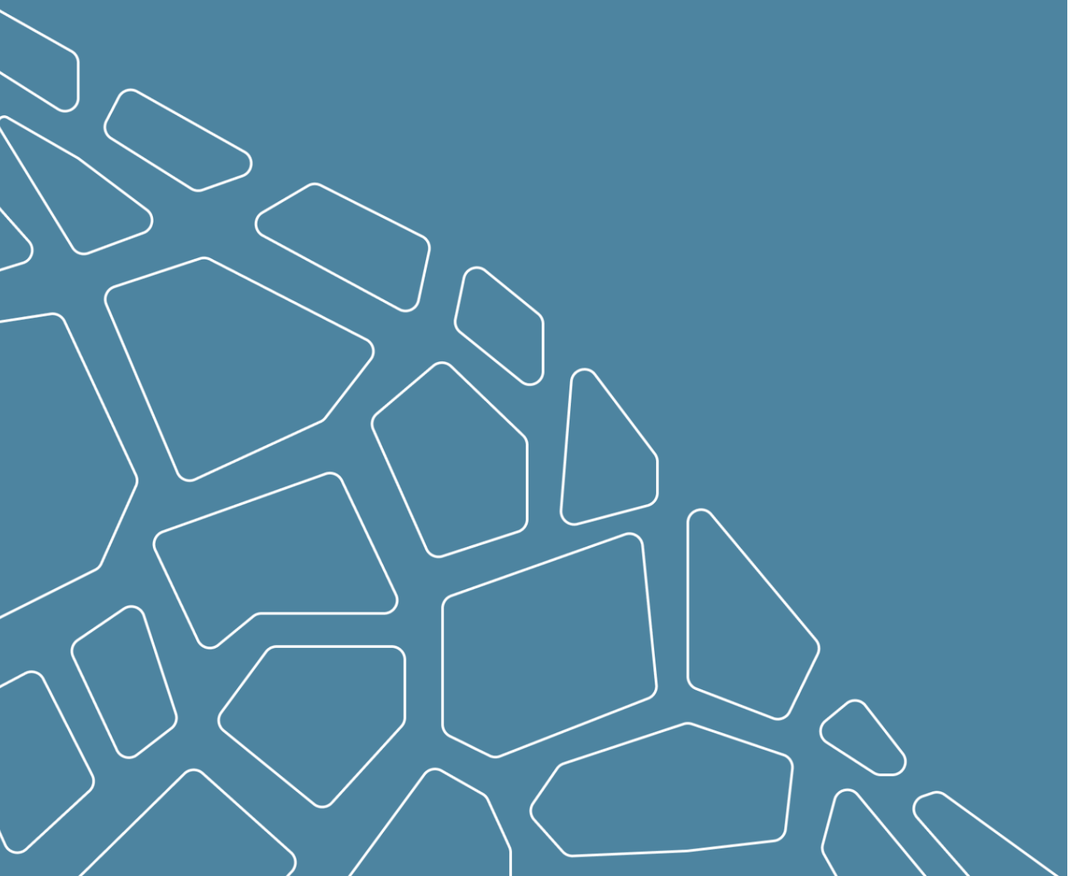
Lecture 11:
Convolutional NNs



1. Convolutional layer
    a. hyperparameters
    b. receptive field
    c. 1x1 conv
    d. memory consumption
2. CNN filter analysis
3. Pooling layers
    a. Adaptive poolings
4. Classification architectures overview
    a. LeNet
    b. AlexNet
    c. VGG
    d. Inception
    e. ResNet
    f. SENet
    g. EfficientNet

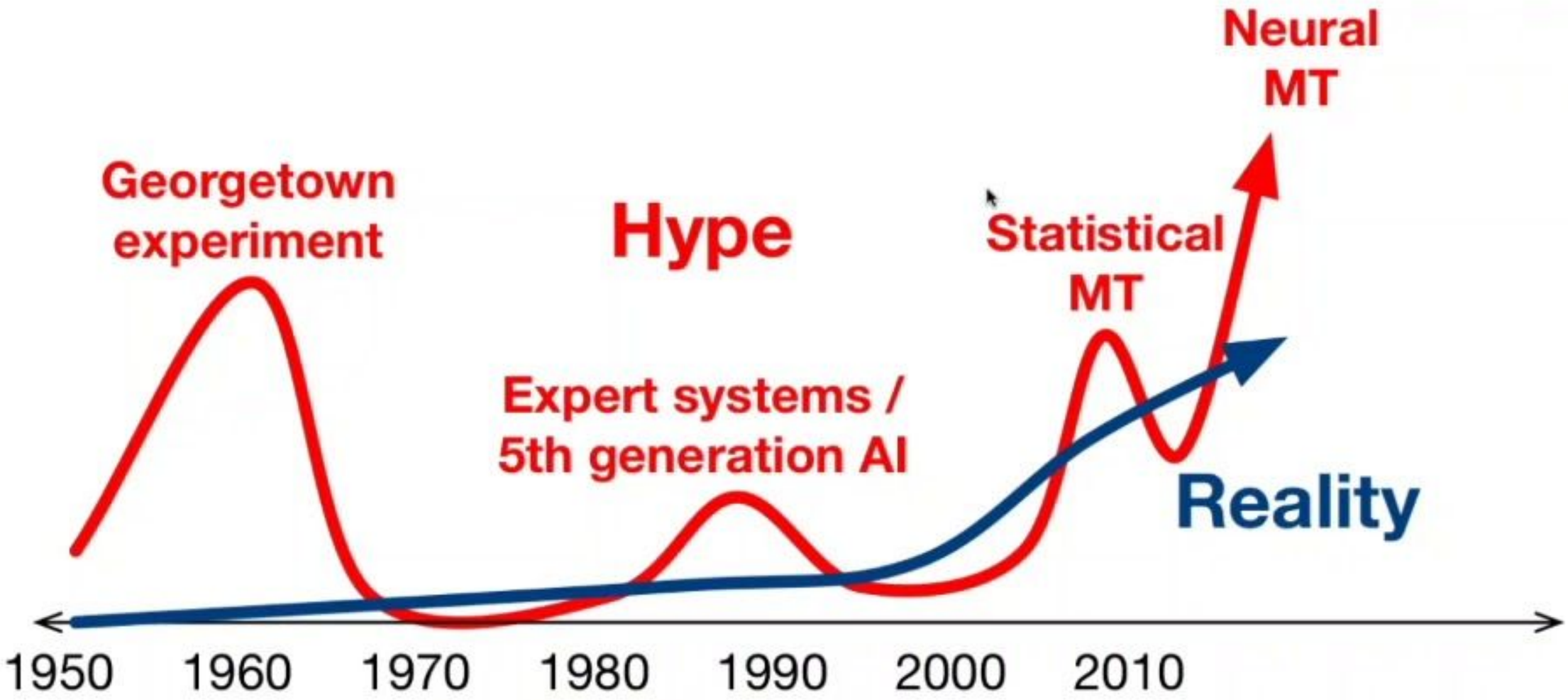
Outline

Lecture 12:
Attention mechanisms
Transformer architecture



1. Machine Translation problem
2. Neural Machine Translation (NMT)
    a. Seq2Seq RNN
3. RNN Attention mechanism
4. Self-Attention
5. Transformer architecture
6. Positional encoding
7. Layer normalization

Machine translation historical overview



Machine Translation problem

girafe ai

01

1950s: first Machine Translation

- Georgetown experiment (7 Jan 1954)
  - Automatic Russian-English translation of 60 sentences
  - 250 vocabulary articles
  - 6 grammar rules
  - Calculated on Mainframe IBM 701
- The same experiment in the USSR (1954 too)
  - Rule-based translation
  - Calculated on BESM
  - read more (ru)

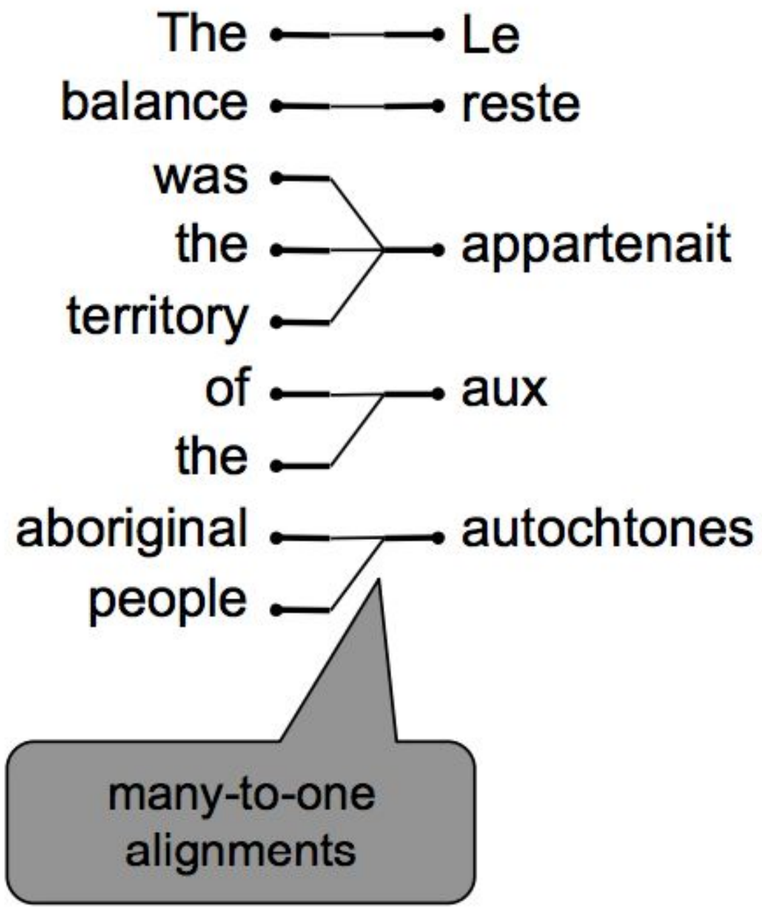
Dataset format for Machine Translation

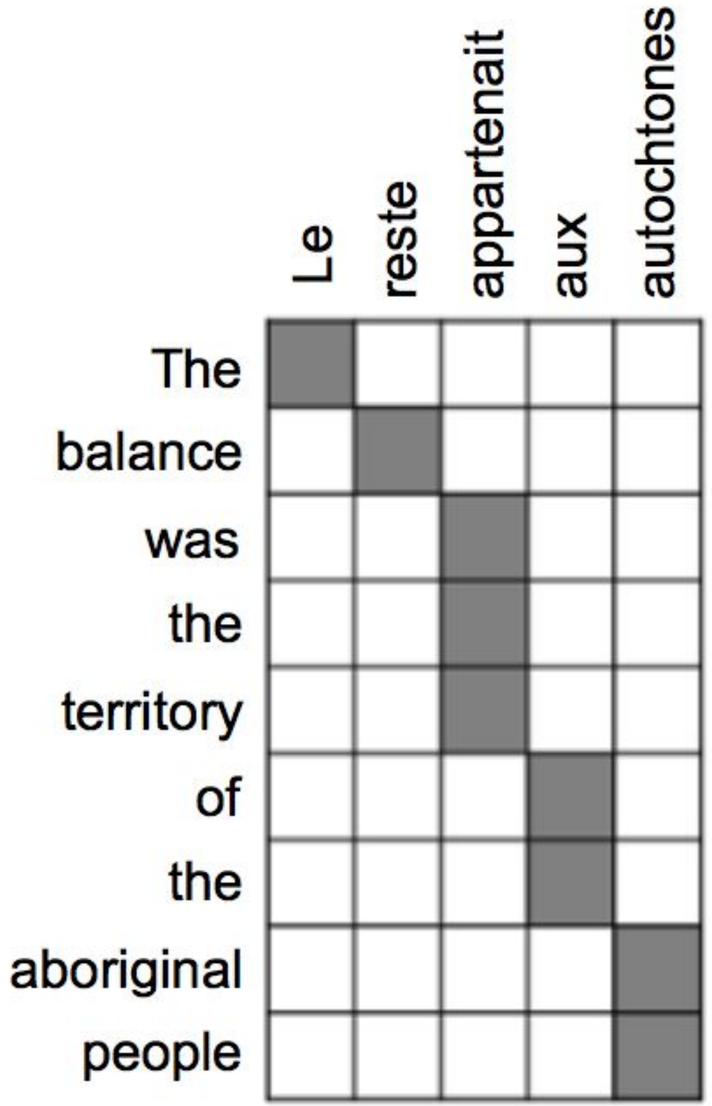
- Parallel corpora
  - Pairs <source, translation>
  - Typically sentence-level (although sometimes can be paragraph- or doc-level)
  - Can be manually curated by translators, crawled from web or synthetic

Source	角柱を過ぎる粘性流体の乱流をラージエディシミュレーションし， フイルタ幅と数値粘性の影響を調べた。
Reference	The large eddy simulation of a turbulent flow of a viscous fluid passing through a square column was conducted, and the effects of the filter width and numerical viscosity were examined.

1990-2010: Statistical Machine Translation

Alignment can be: many-to-one

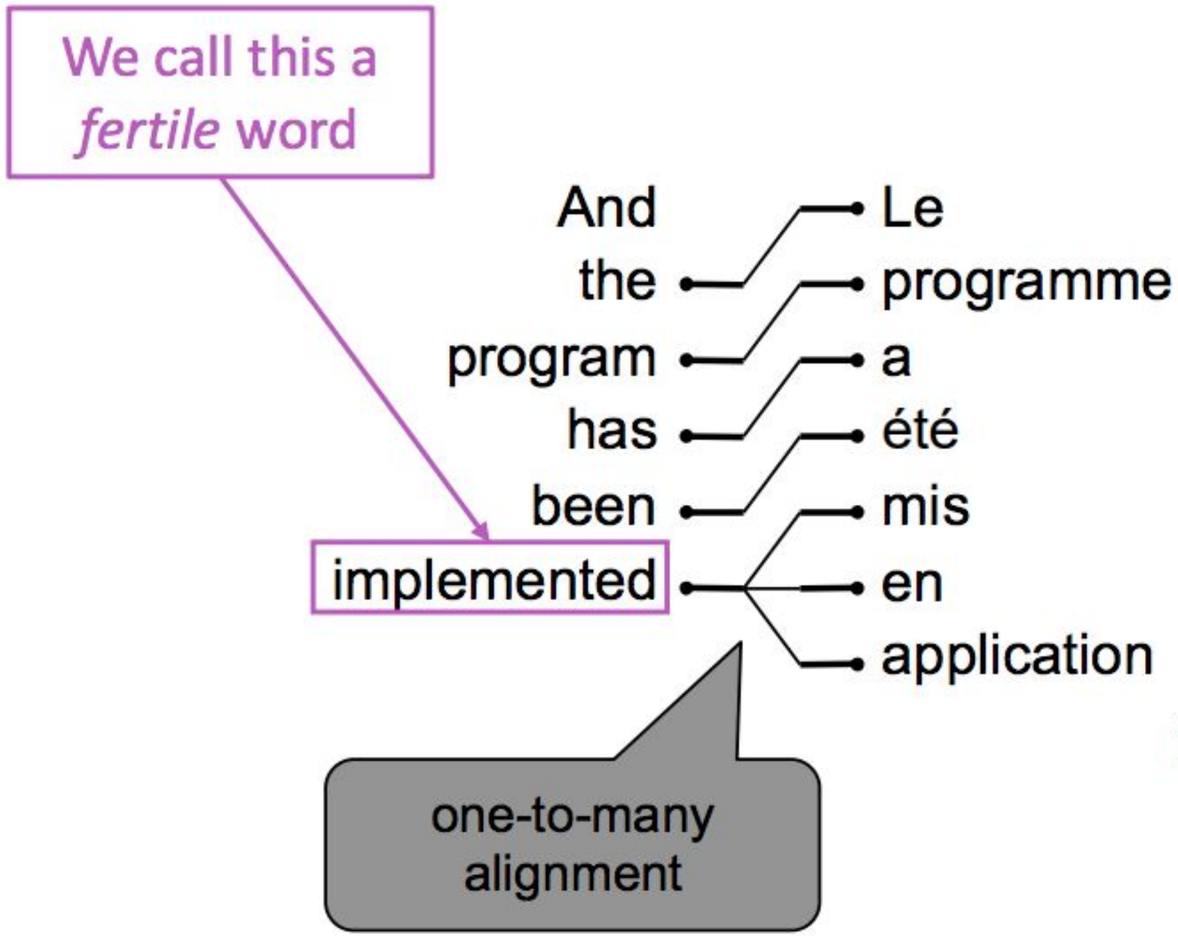


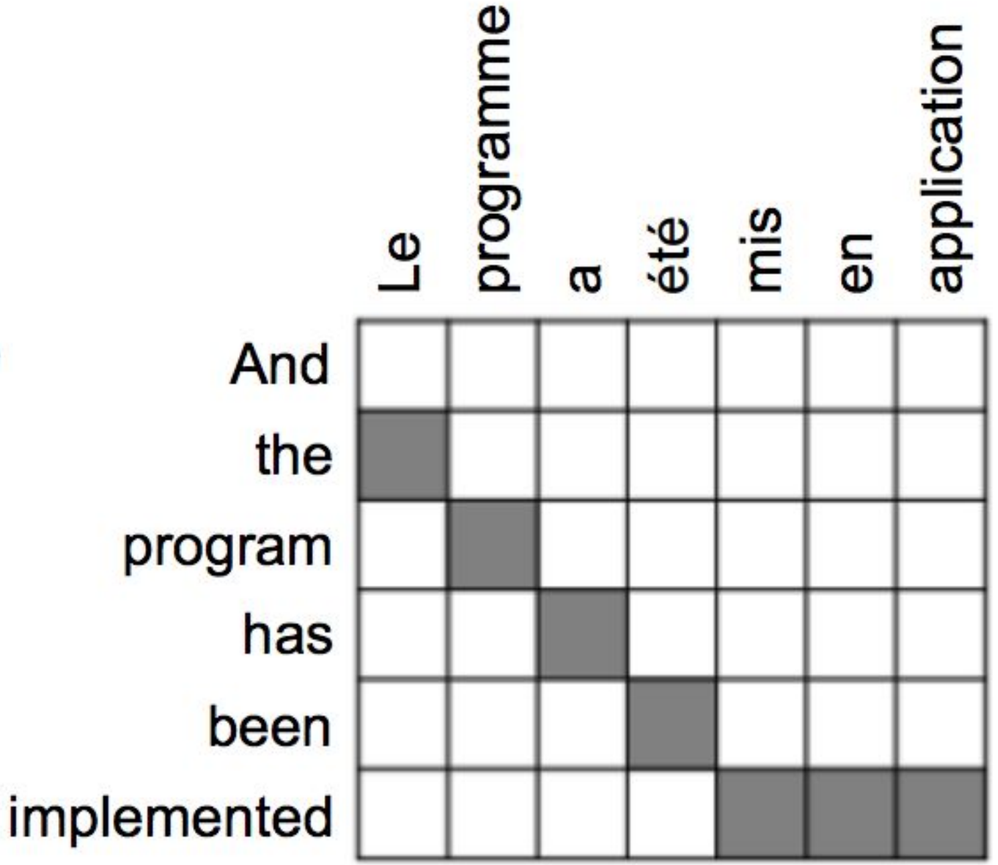


Source: http://web.stanford.edu/class/cs224n/slides/cs224n-2019-lecture08-nmt.pdf

1990-2010: Statistical Machine Translation

Alignment can be: one-to-many

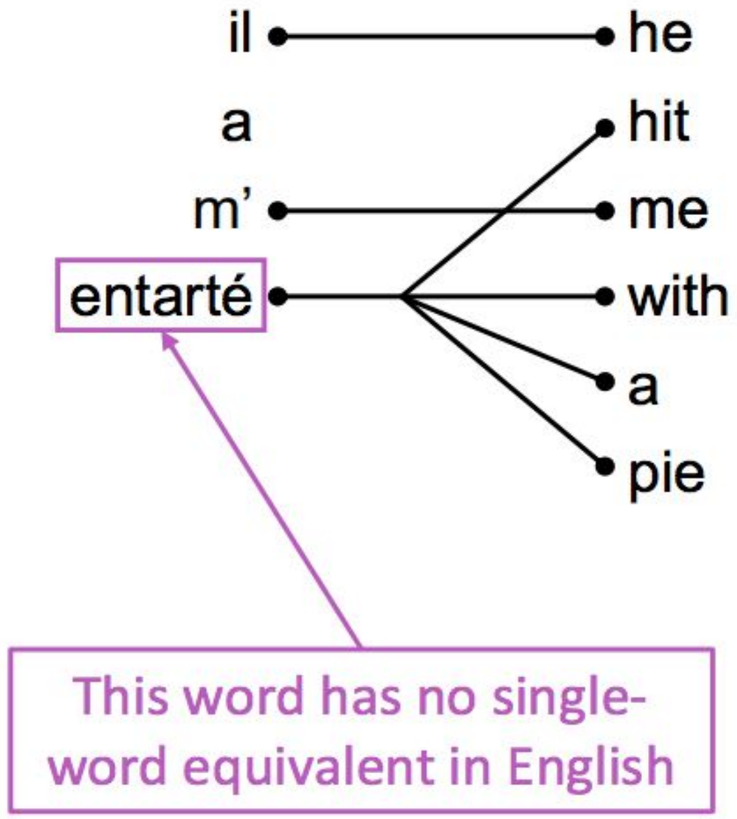


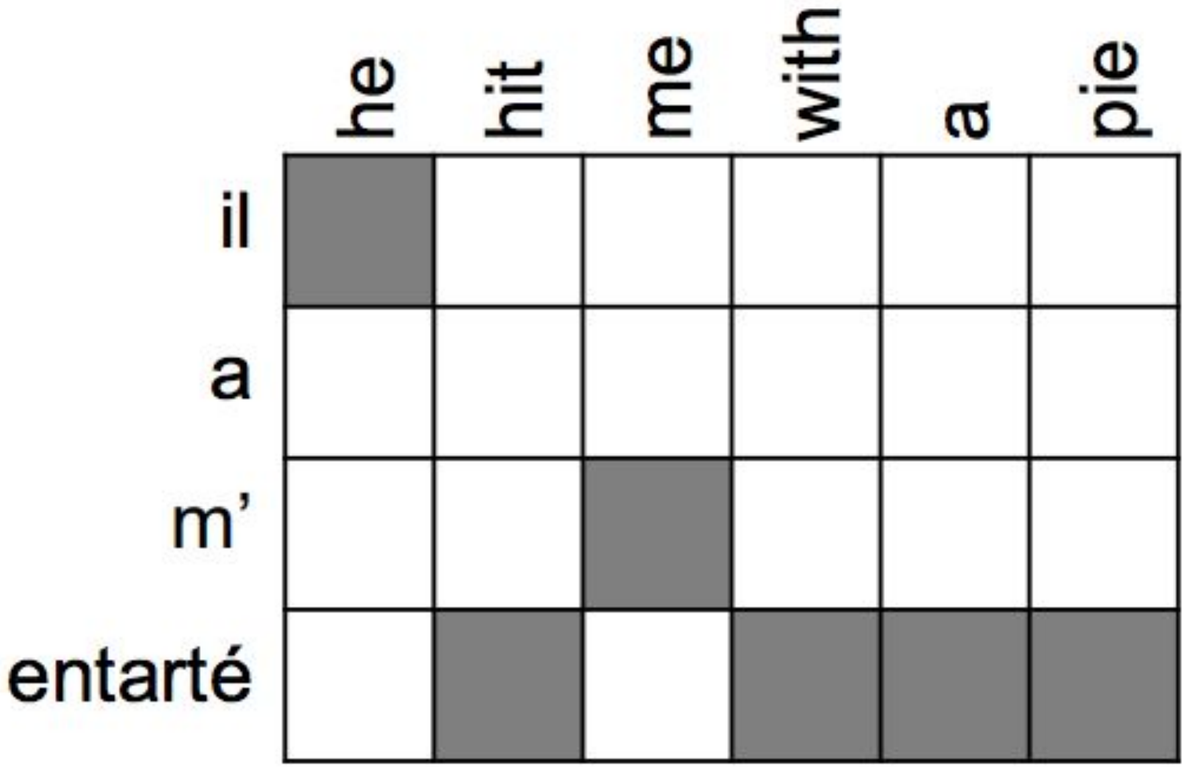


Source: http://web.stanford.edu/class/cs224n/slides/cs224n-2019-lecture08-nmt.pdf

1990-2010: Statistical Machine Translation

Some words are very fertile!

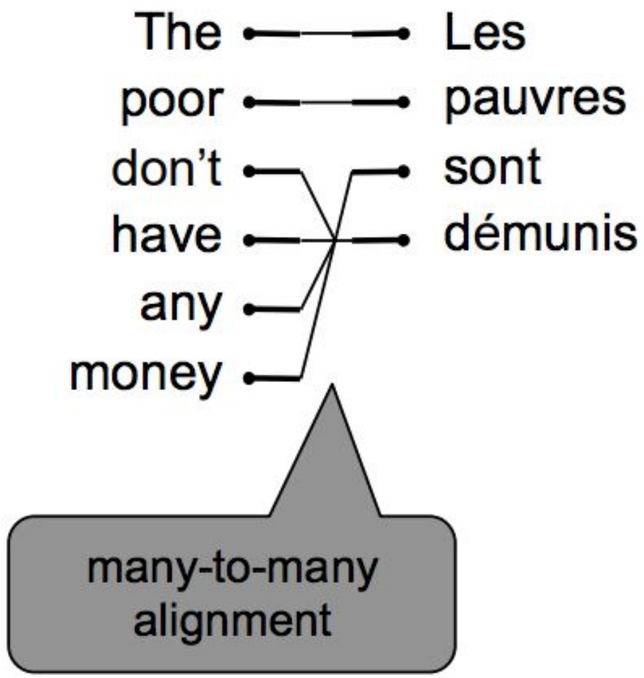


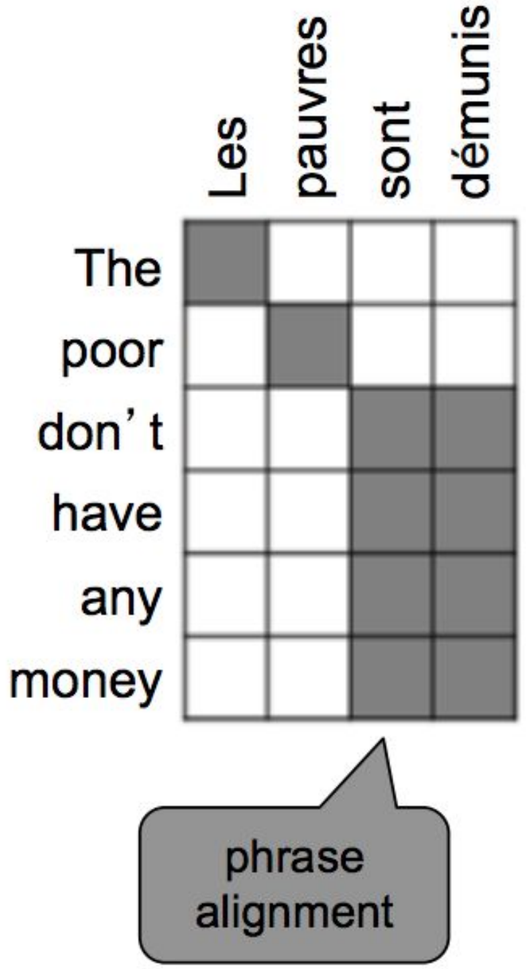


Source:

1990-2010: Statistical Machine Translation

Alignment can be: many-to-many





Source: http://web.stanford.edu/class/cs224n/slides/cs224n-2019-lecture08-nmt.pdf

Neural Machine Translation with RNN

girafe ai

02

Statistics versus Neural Networks



NO, you cannot understand the meaning of a text without explicitly evaluating its linguistic constituents and defining grammar rules!



haha gpus go brrrrrrrrr

Advanced task: name two gentlemens on the slide. More info (ru)

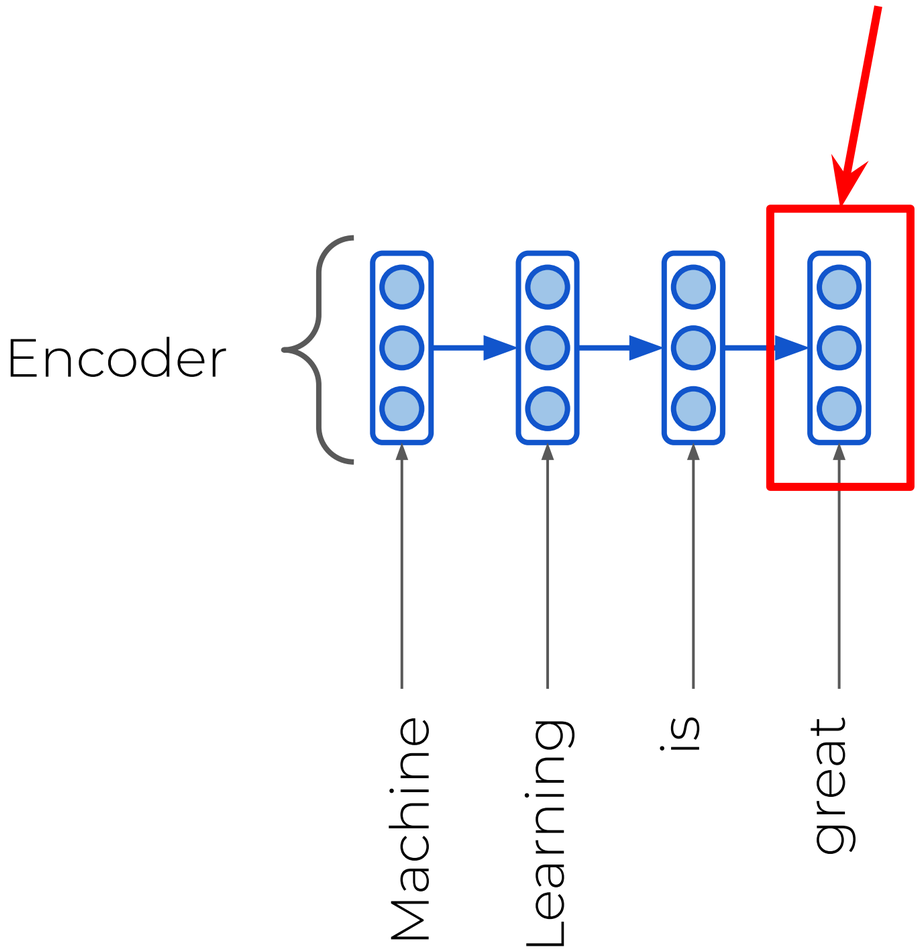
What is Neural Machine Translation?

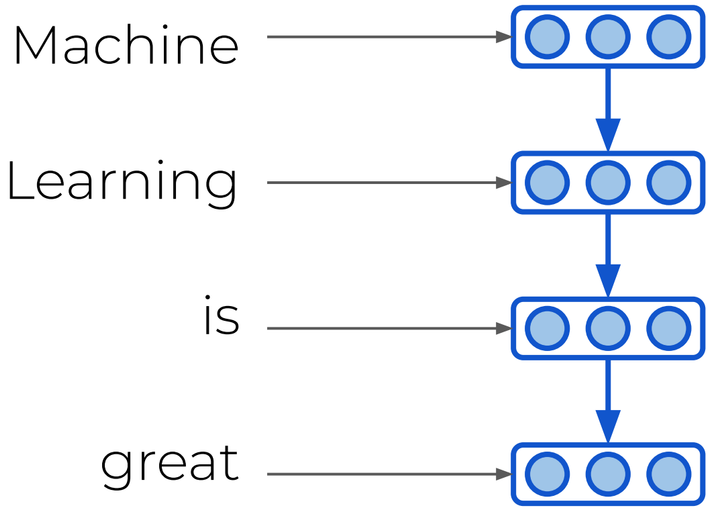
- Neural Machine Translation (NMT) is a way to do Machine Translation with a single neural network

- The neural network architecture is called sequence-to-sequence (aka seq2seq), it involves two RNNs

Seq2seq NMT

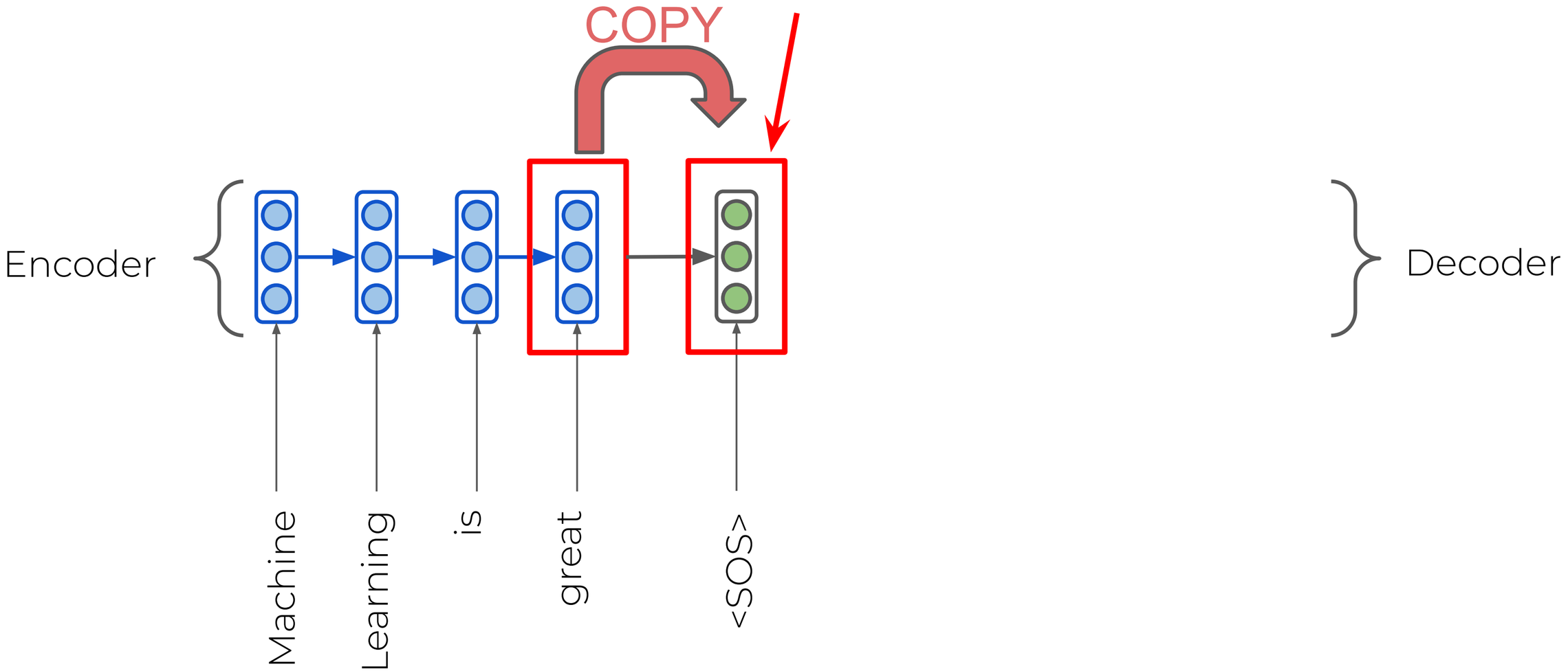
This state encodes the whole sentence



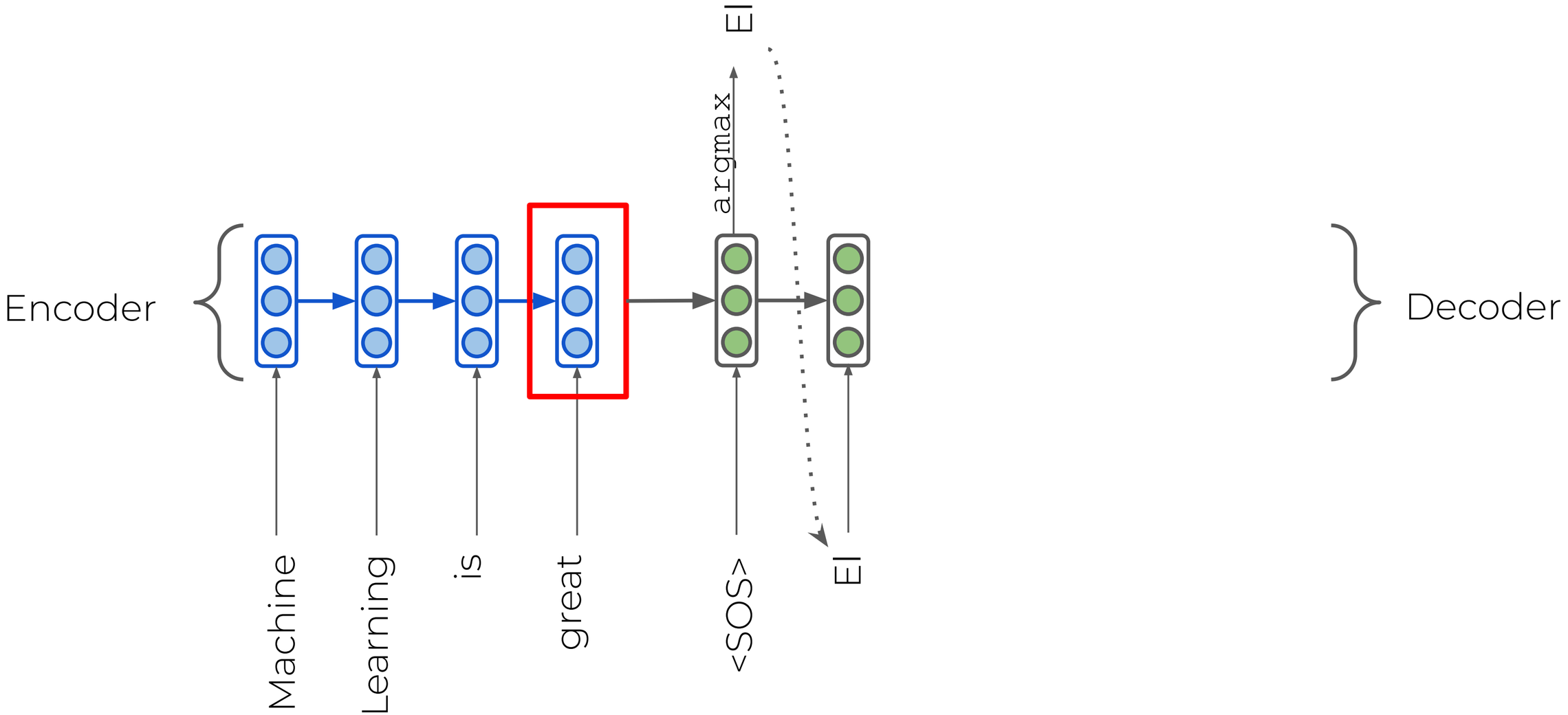

Machine
Learning
is
great

Seq2seq NMT

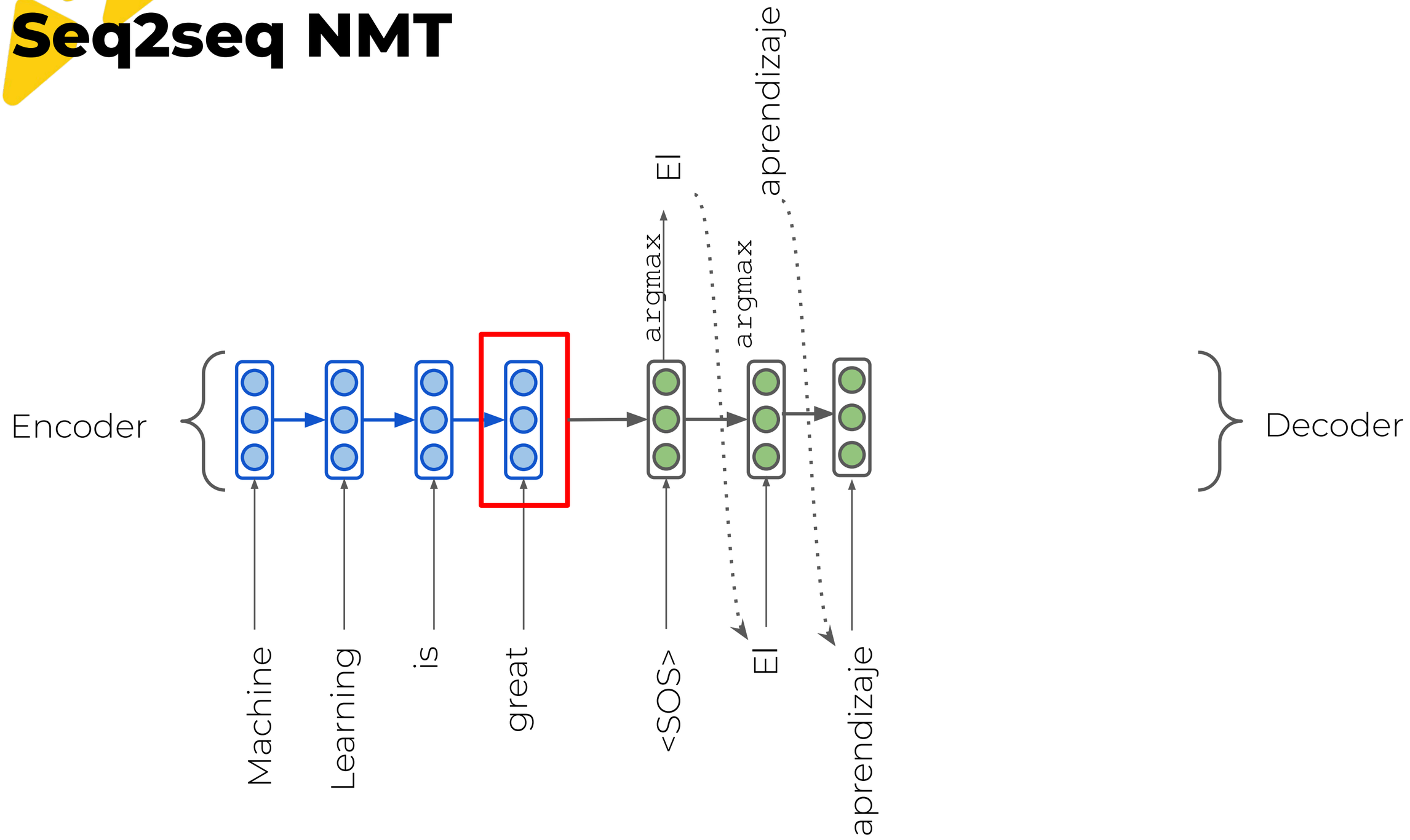
Forwarded as initial hidden state to decoder



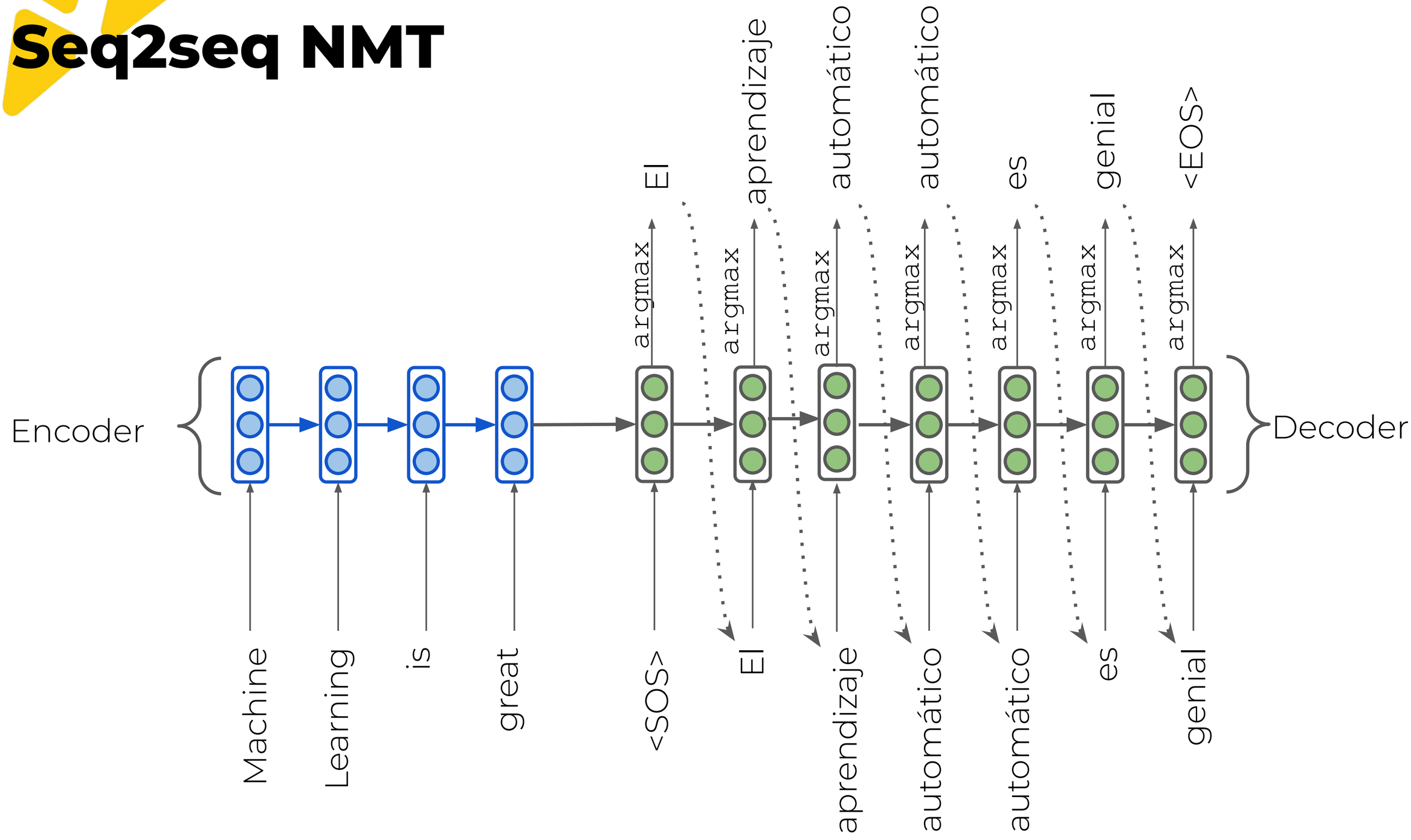
Seq2seq NMT



Seq2seq NMT


Encoder
Machine
Learning
is
great
<SOS>
argmax
El
El
argmax
aprendizaje
aprendizaje
Decoder

Seq2seq NMT


Encoder
Machine
Learning
is
great
<SOS>
argmax
El
El
argmax
aprendizaje
aprendizaje
argmax
automático
automático
argmax
automático
automático
argmax
es
es
argmax
genial
genial
argmax
<EOS>
Decoder

NMT: how does it work?

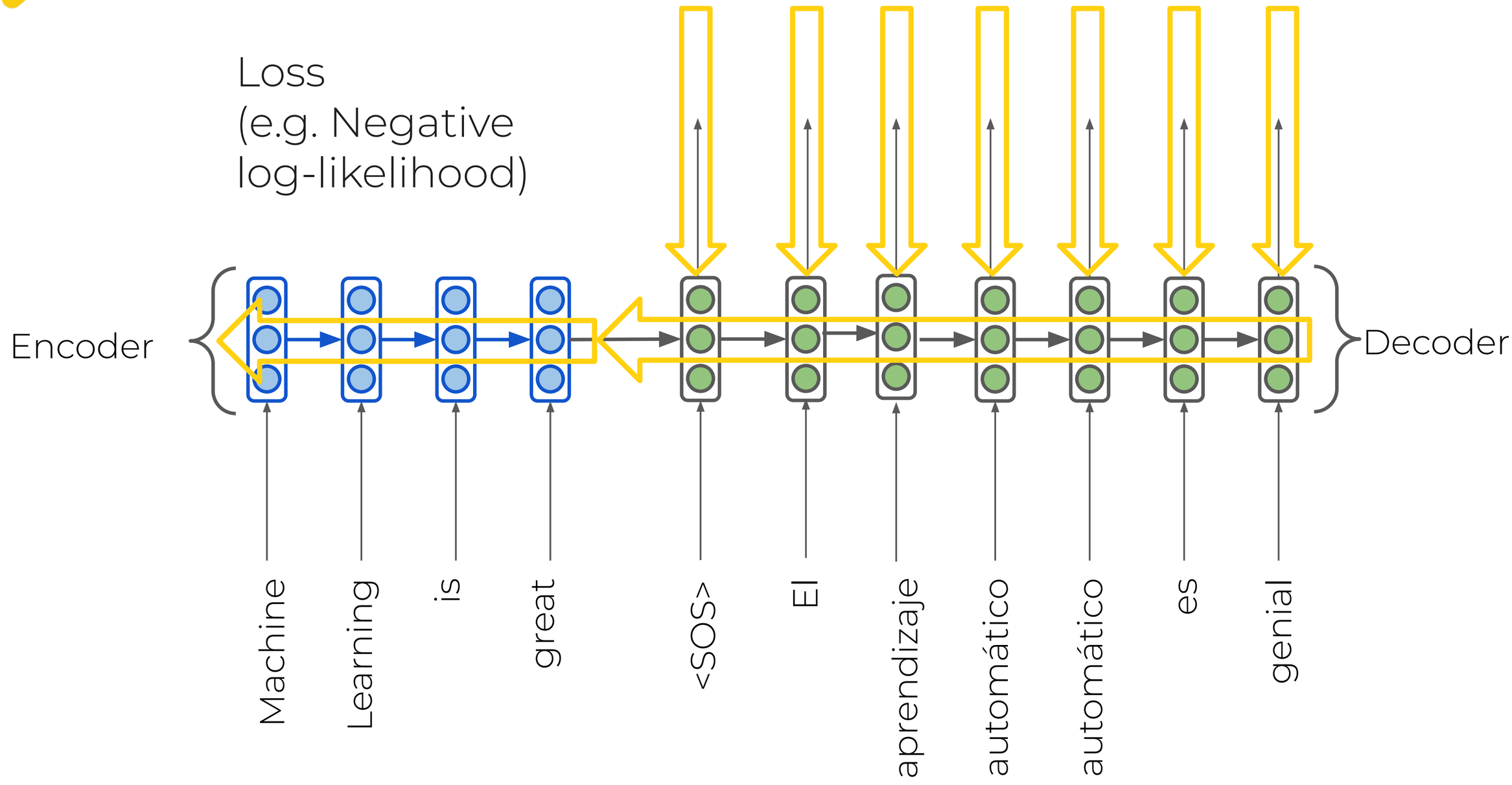
- NMT directly calculates P(y|x)
  - y – target sentence, x – source sentence

$$P(y|x) = P(y_2|y_1, x)P(y_3|y_1, y_2, x)\ldots\underbrace{P(y_T|y_1, y_2, \ldots, x)}$$

Probability of next word in target language

- To train it we need a huge parallel corpus.

Seq2seq is trained end-to-end

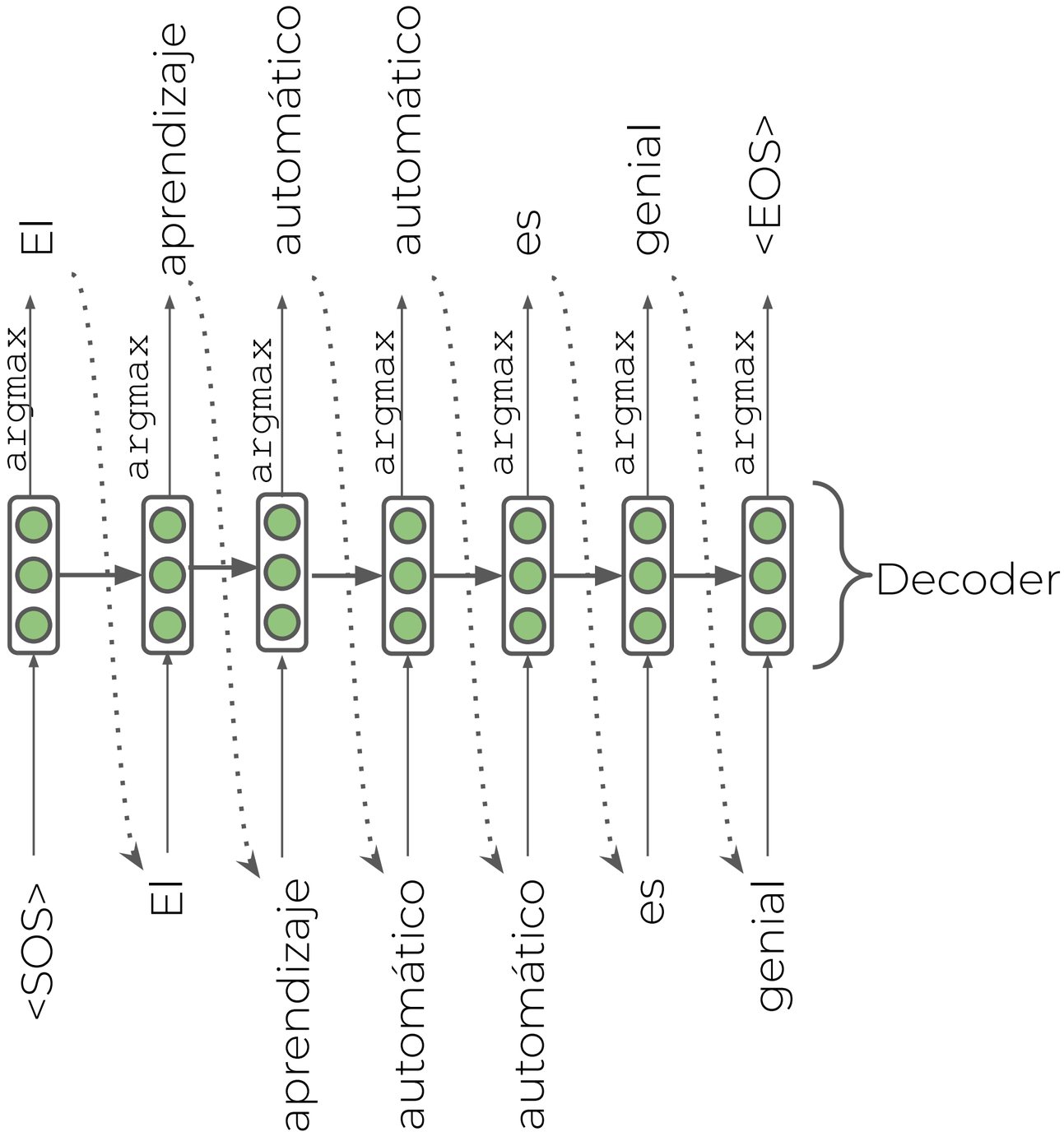


Greedy decoding

- Decoder predicts the most probable token (argmax) on each step
- The approach is greedy

Any problems with it?

Any mistake is treated as input on the next step!



Attention

girafe
ai

03

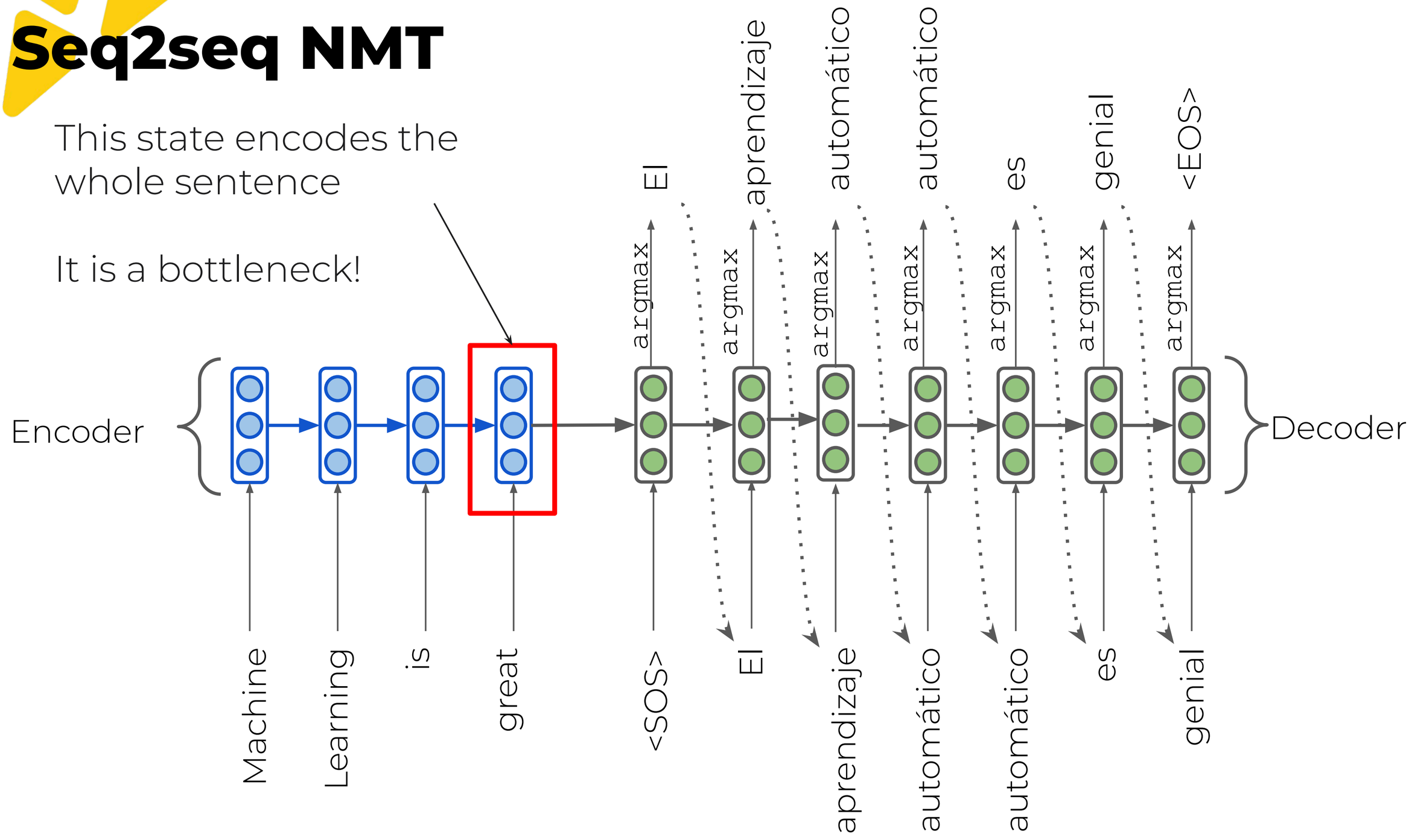
Attention timeline

- Papers on attention mechanism started being published since 2014
  - e.g. https://arxiv.org/abs/1409.0473
- but most hype awarded paper released in 2017
  - “Attention is all you need” https://arxiv.org/abs/1706.03762

Seq2seq NMT

This state encodes the whole sentence

It is a bottleneck!



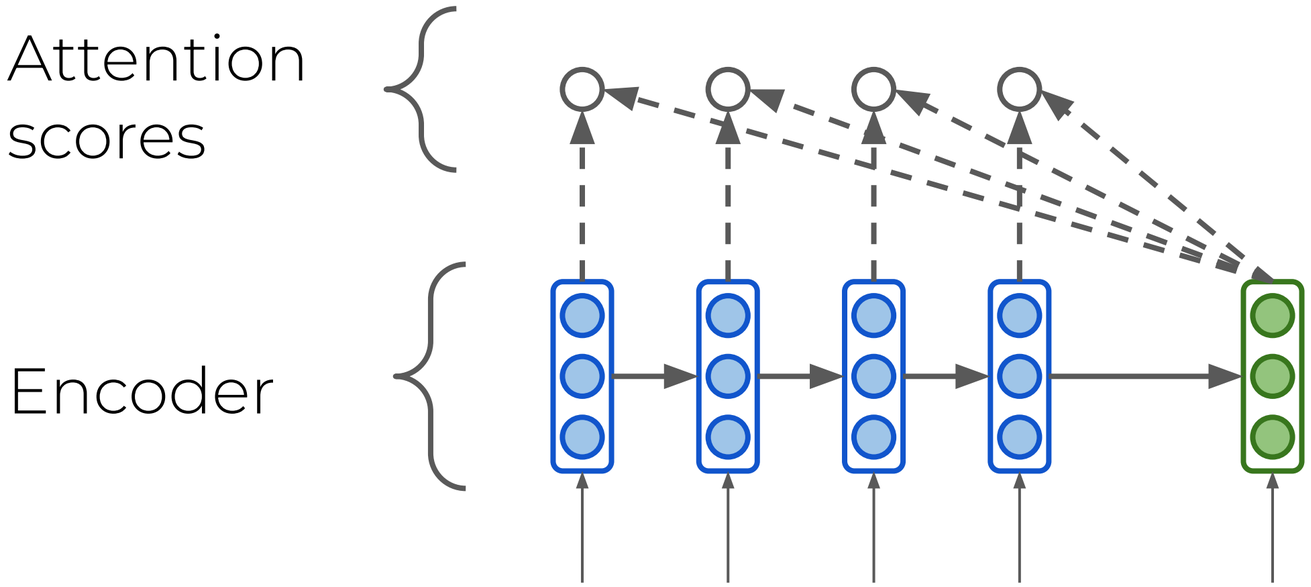
Seq2seq RNN attention

Main idea:

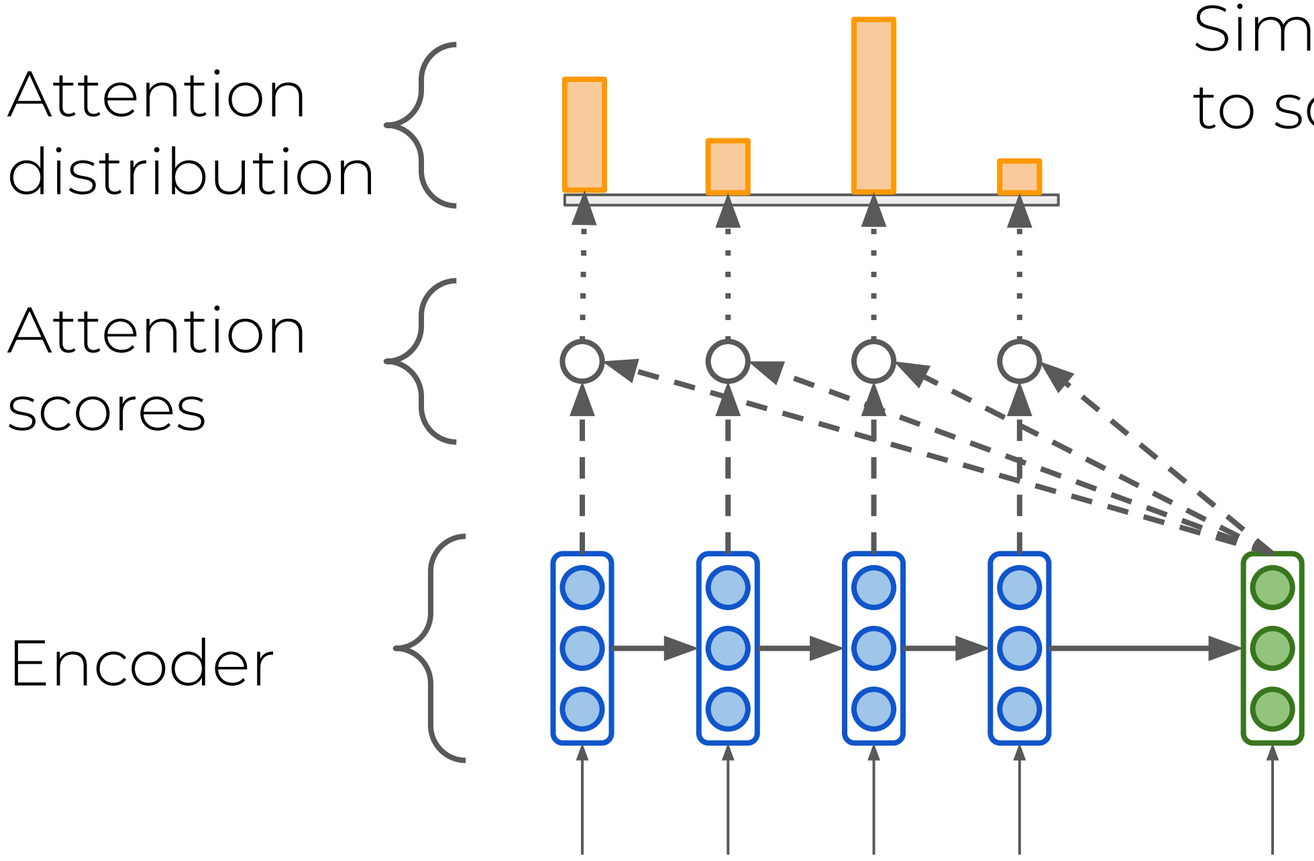
on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence



Seq2seq with attention



Seq2seq with attention



Simply apply softmax to scores

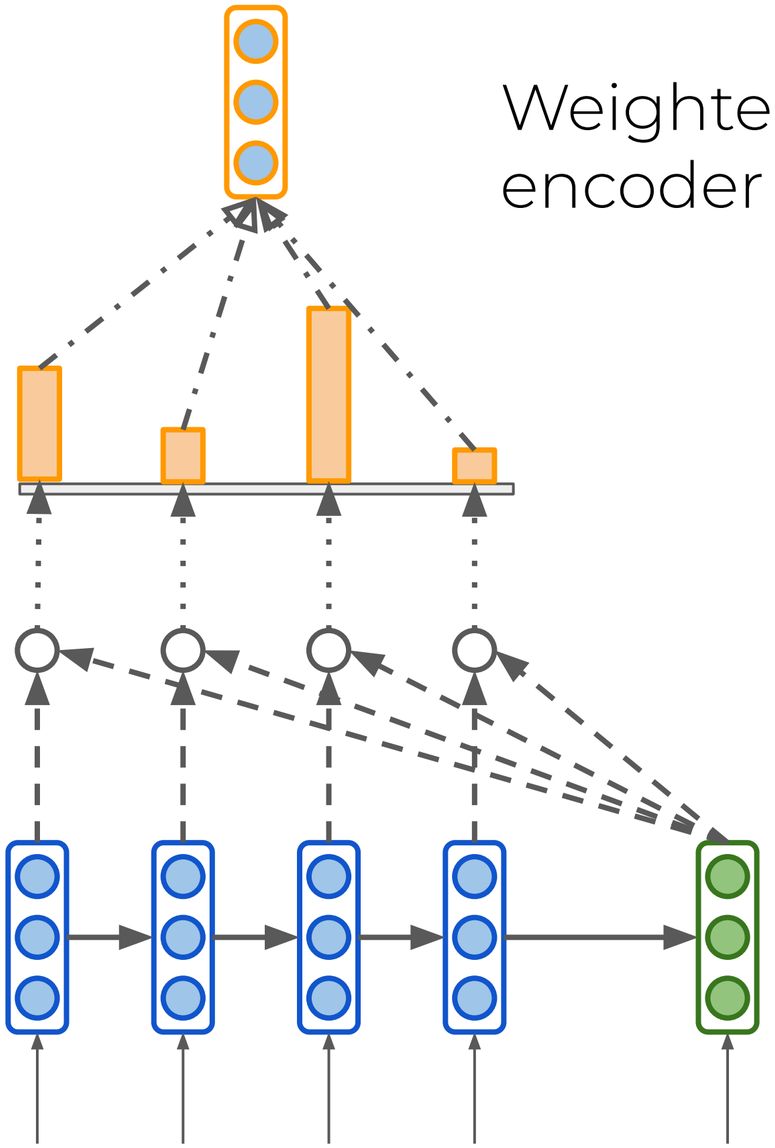
Seq2seq with attention

Attention output

Attention distribution

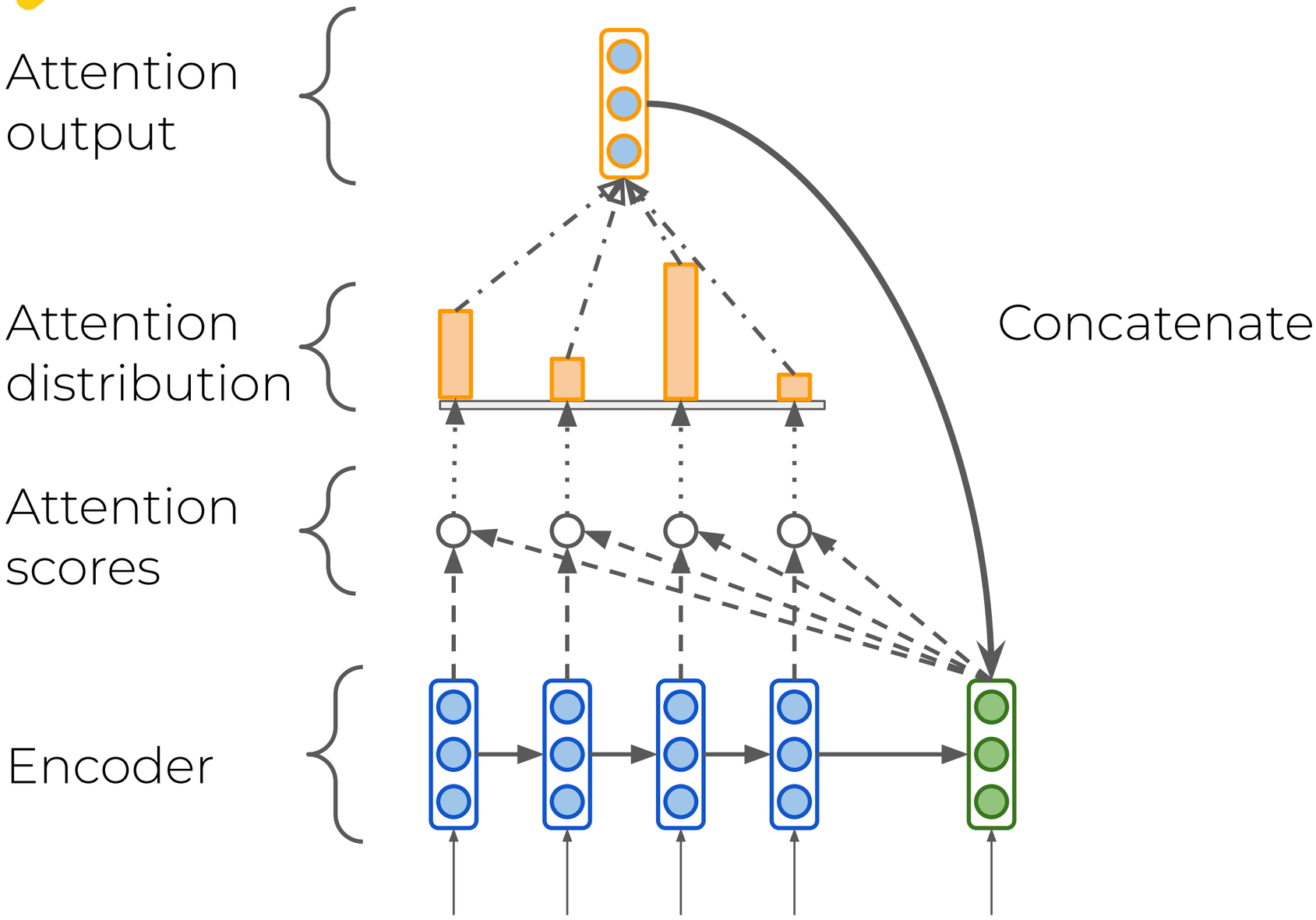
Attention scores

Encoder

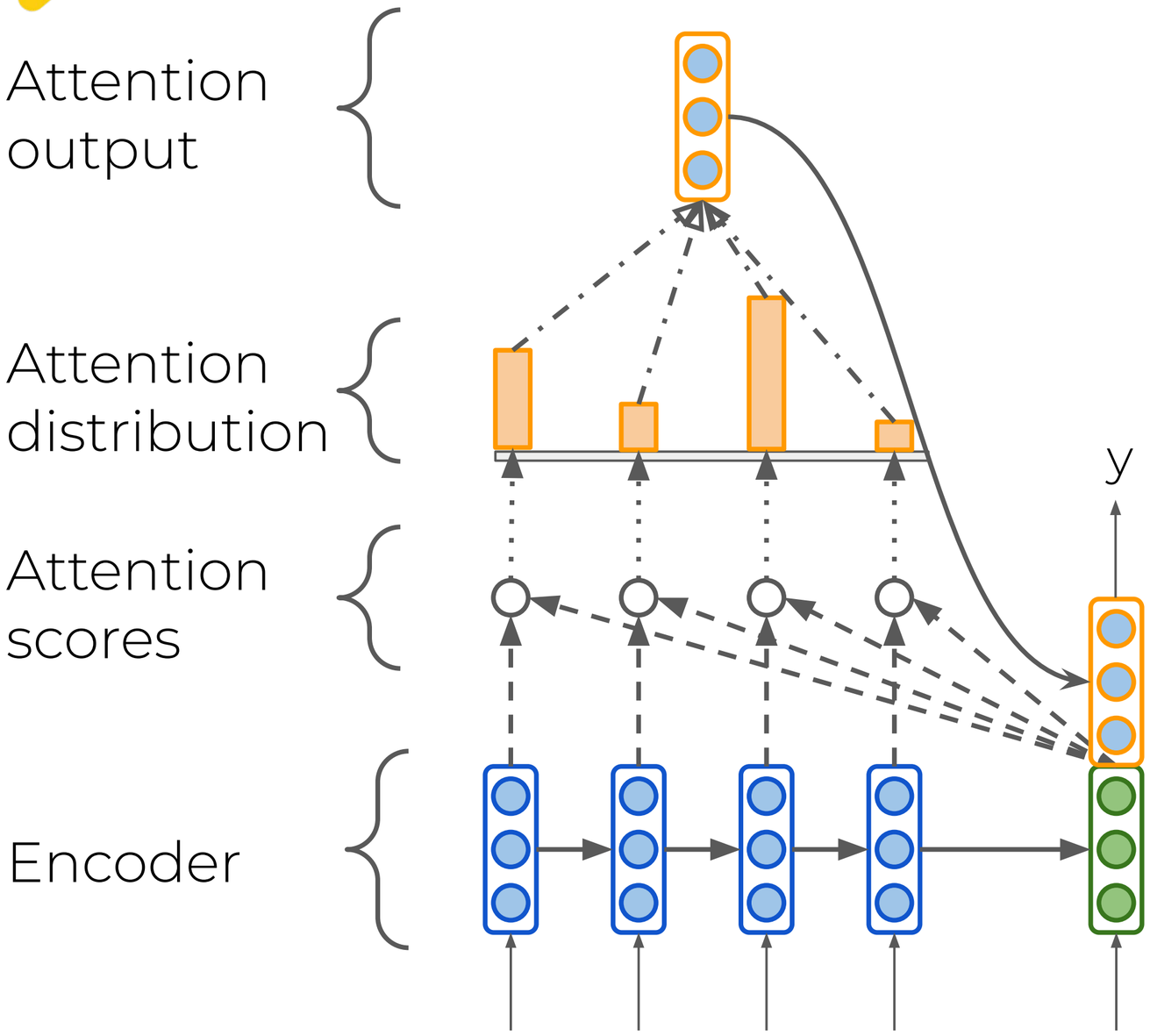


Weighted sum of all encoder states

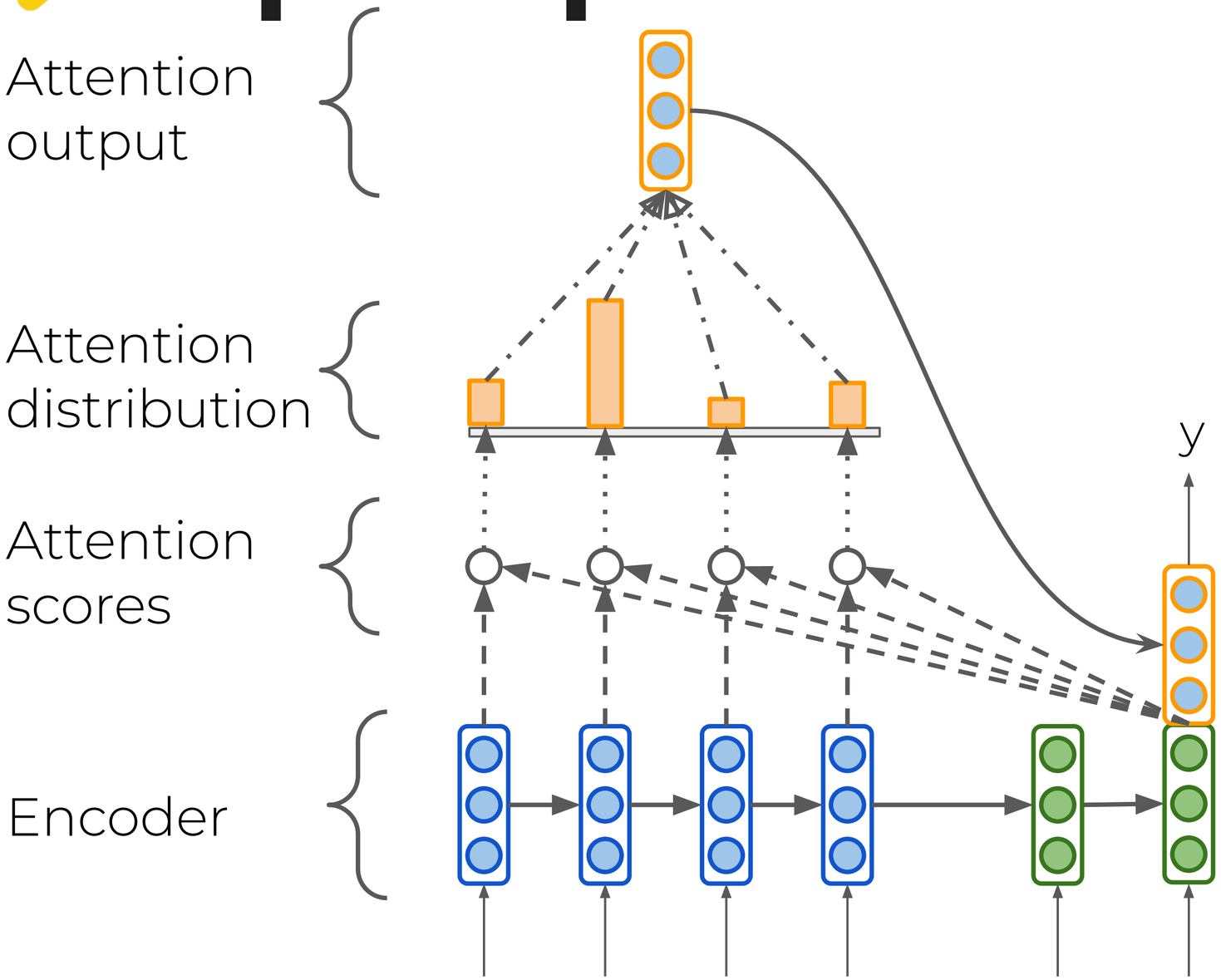
Seq2seq with attention



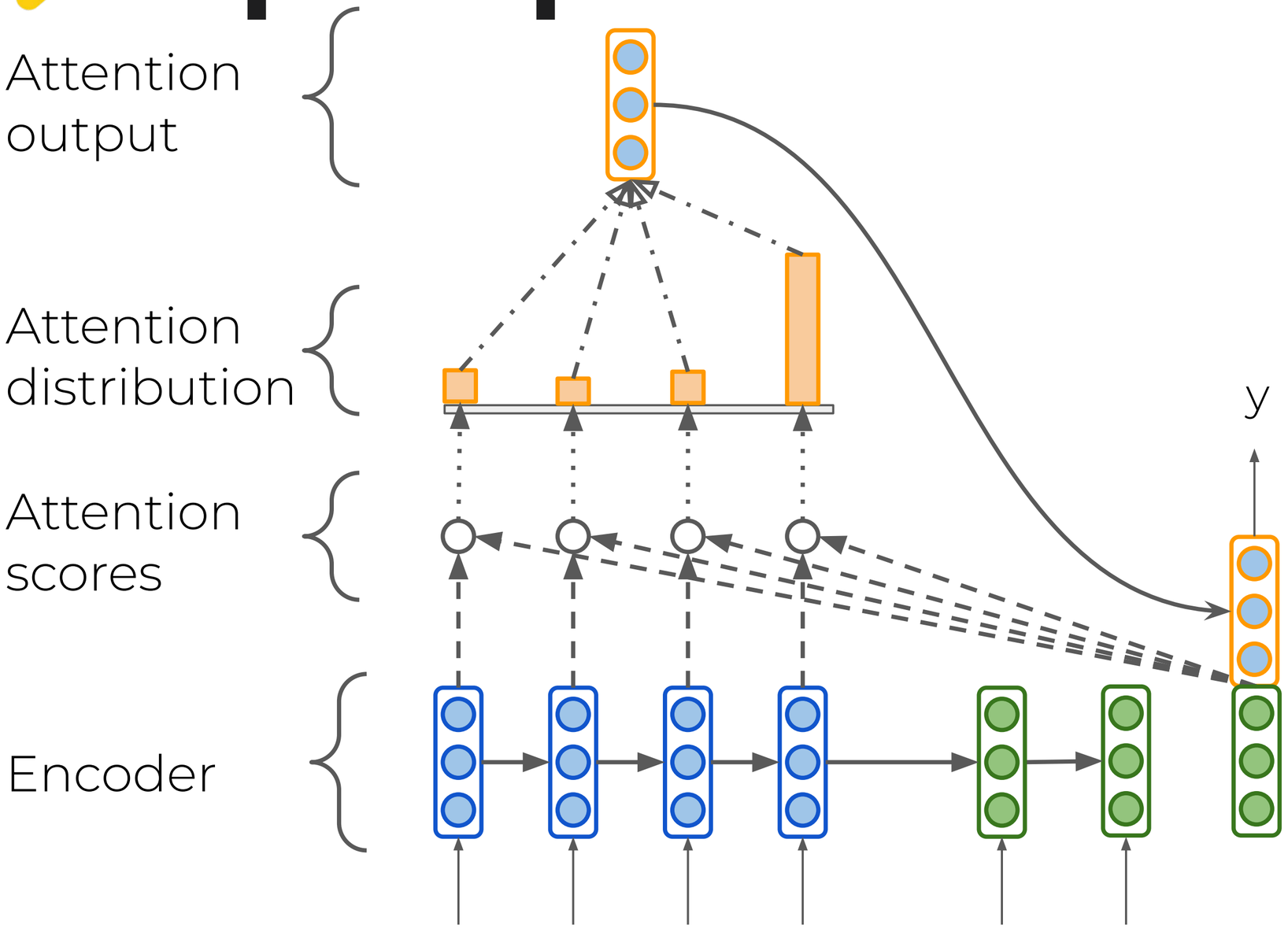
Seq2seq with attention



Seq2seq with attention



Seq2seq with attention



Attention in equations

Denote encoder hidden states $\mathbf{h}_1, \ldots, \mathbf{h}_N \in \mathbb{R}^k$

and decoder hidden state at time step t $\mathbf{s}_t \in \mathbb{R}^k$

The attention scores $\mathbf{e}^t$ can be computed as dot product

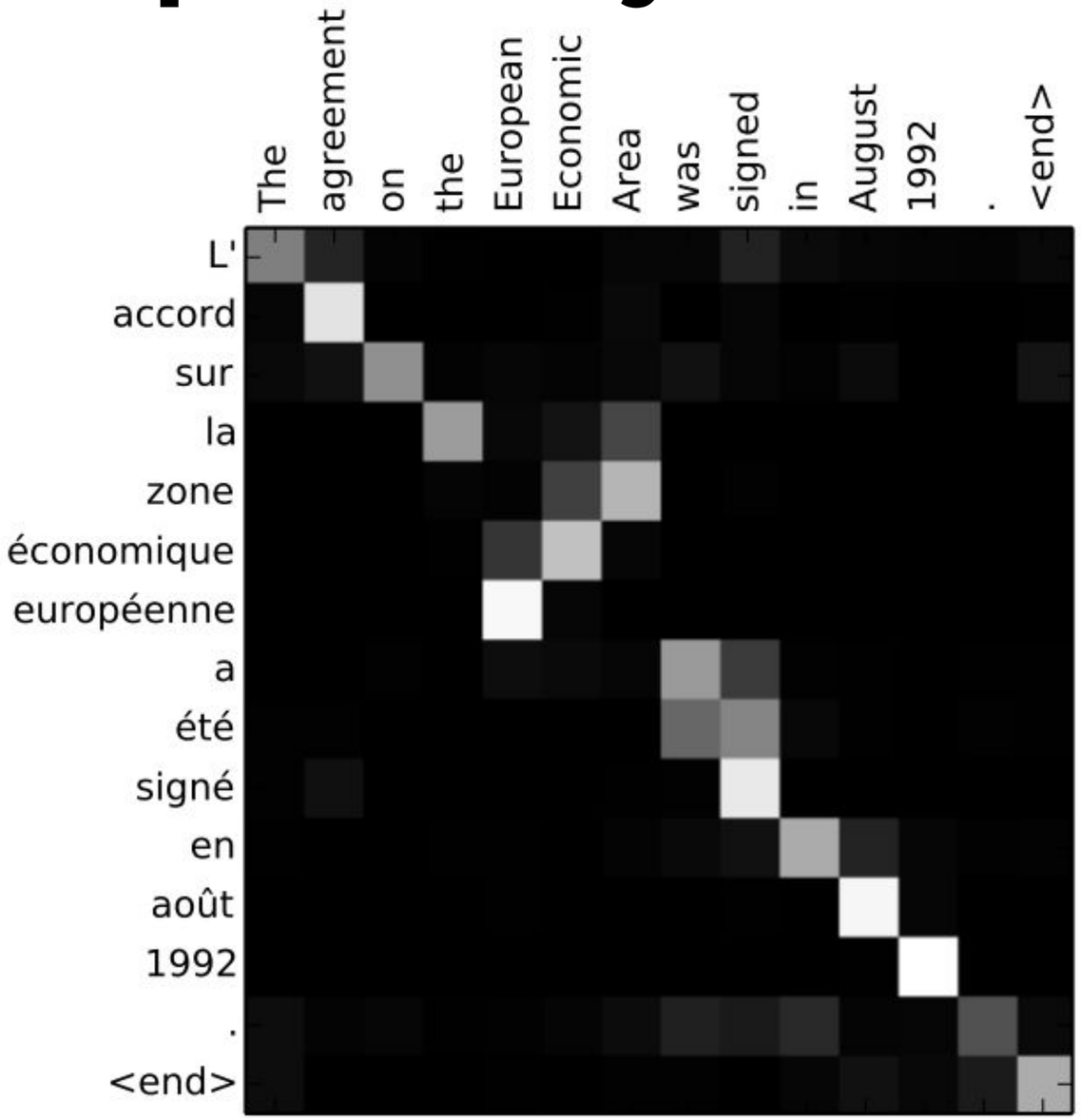
$$\mathbf{e}^t = [\mathbf{s}^T\mathbf{h}_1, \ldots, \mathbf{s}^T\mathbf{h}_N]$$

Then the attention vector is a linear combination of encoder states

$$\mathbf{a}_t = \sum_{i=1}^{N} \alpha_i^t \mathbf{h}_i \quad \text{where} \quad \mathbf{a}_t \in \mathbb{R}^k \qquad \boldsymbol{\alpha}_t = \text{softmax}(\mathbf{e}_t)$$

Attention provides interpretability

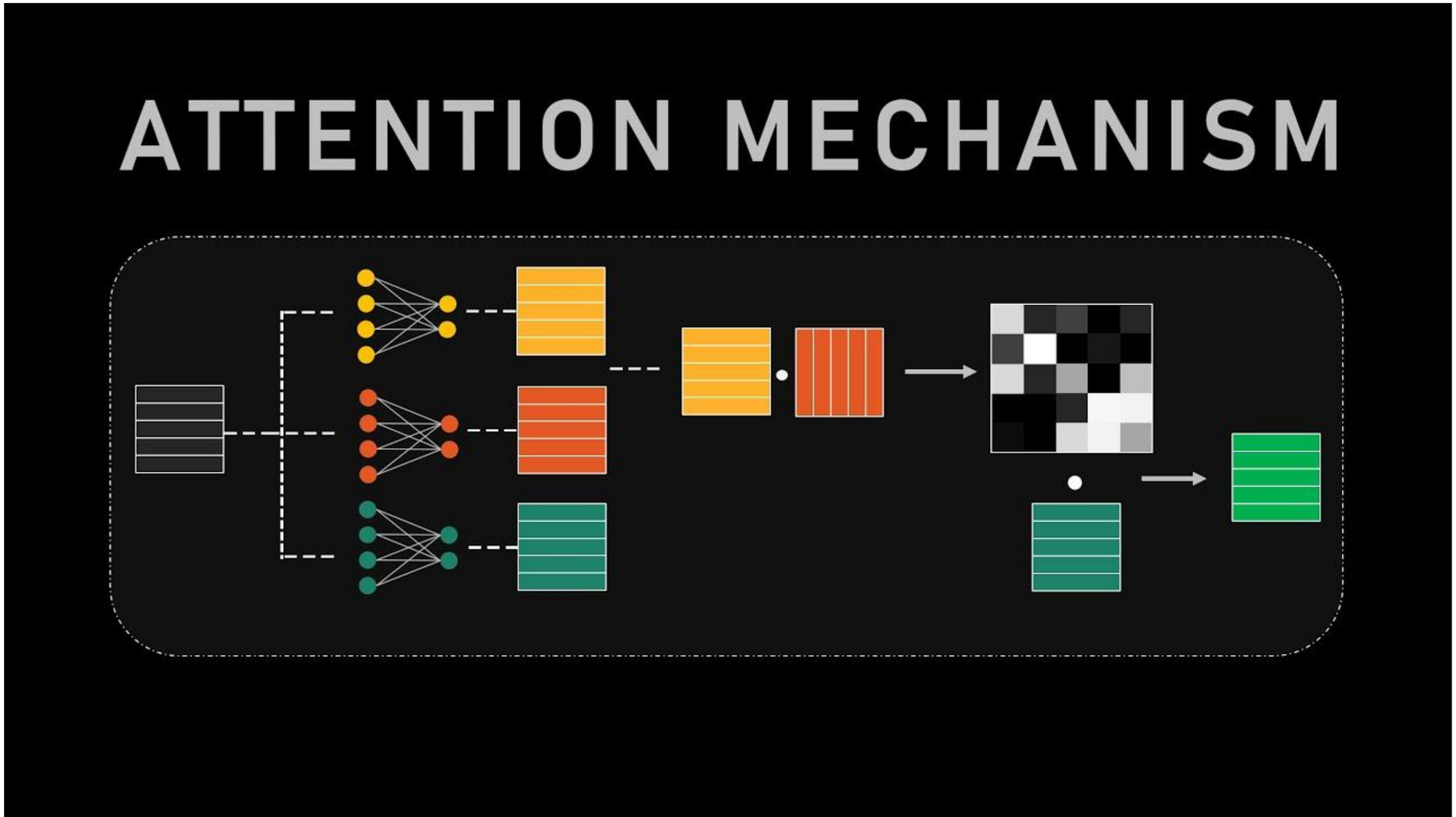
- We may see what the decoder was focusing on
- We get word alignment for free!



Bahdanau et al. "Neural Machine Translation by Jointly Learning to Align and Translate", 2014

Attention explained

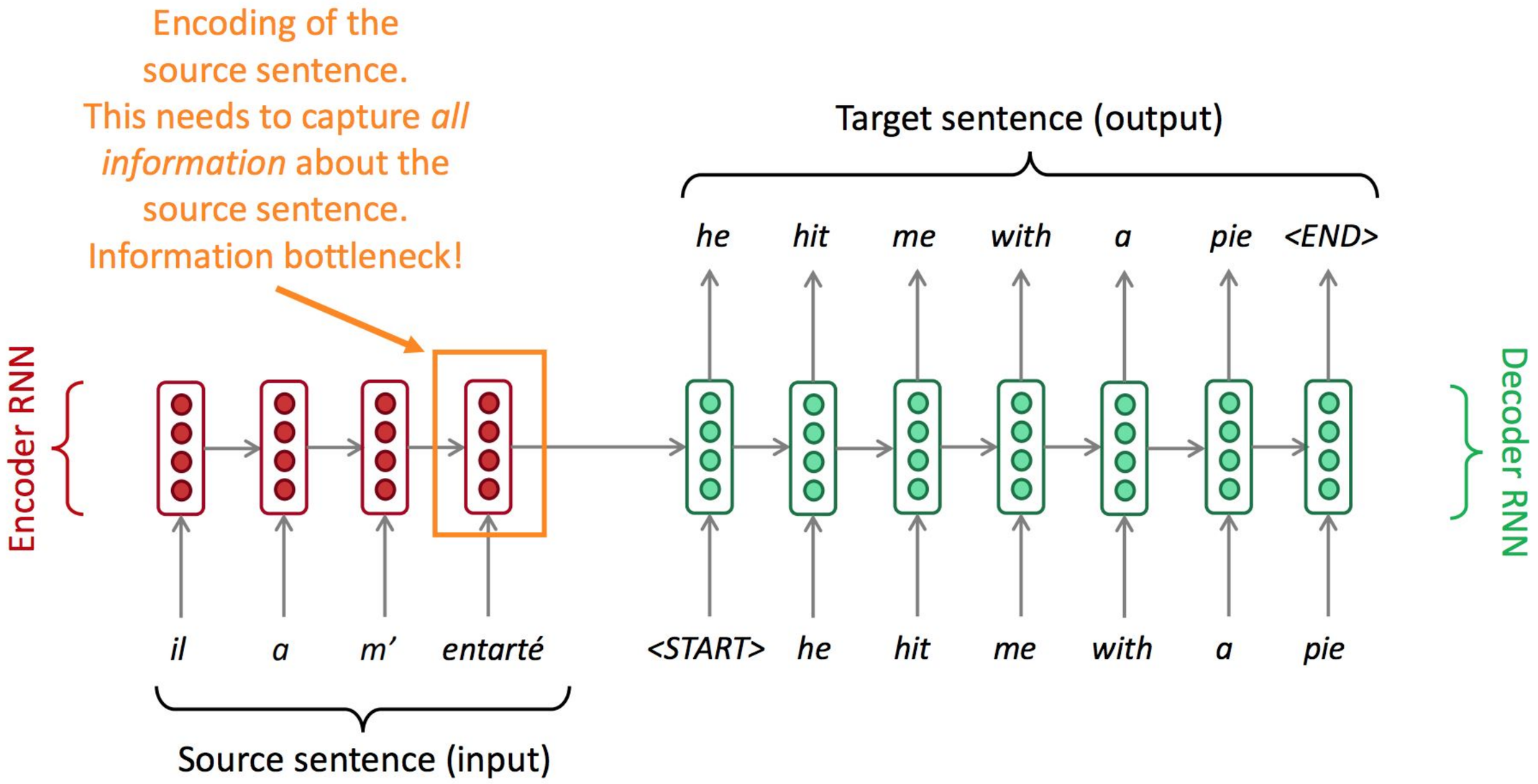
https://www.youtube.com/watch?v=KMHkbXzHn7s



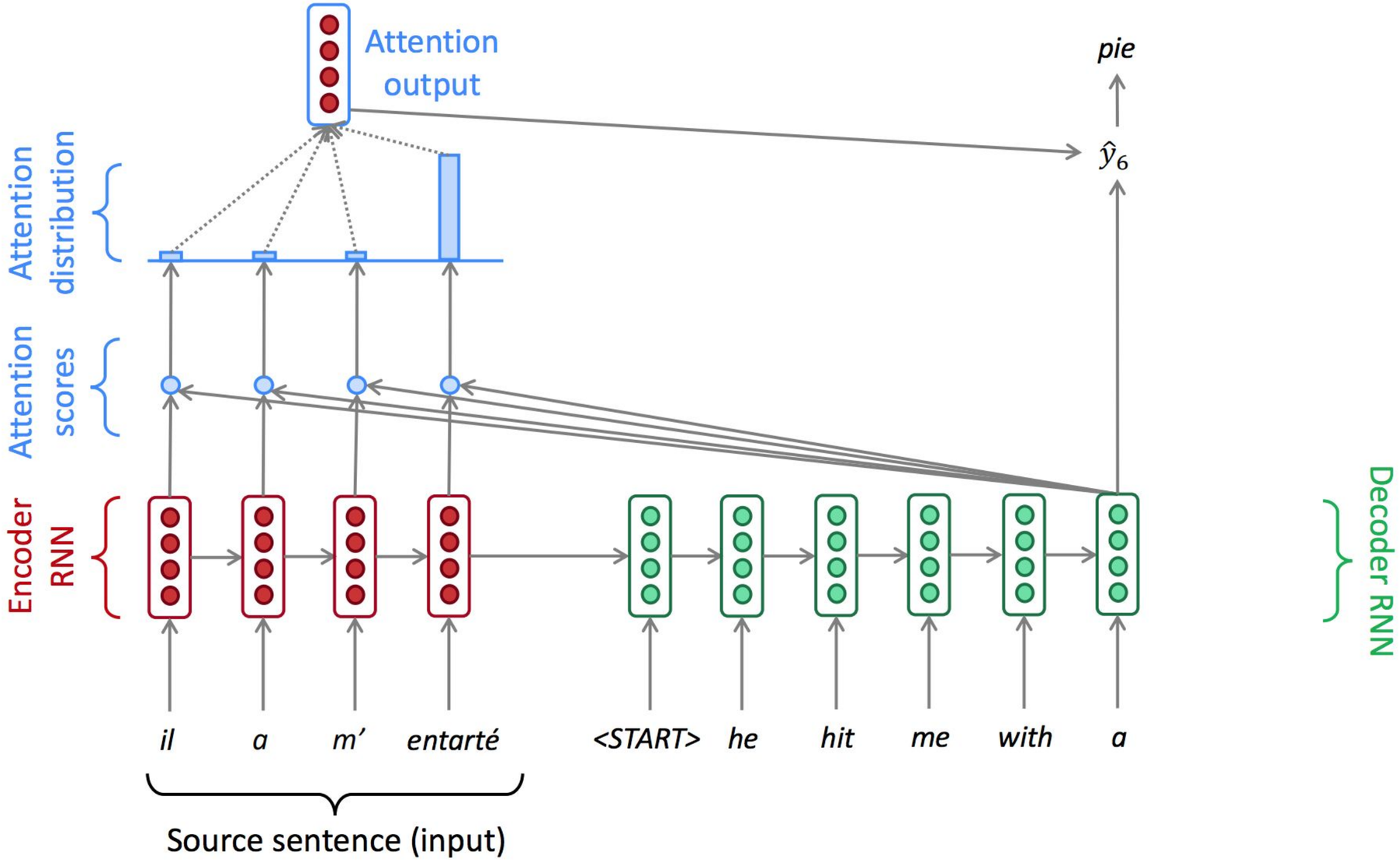
Attention variants

- Basic dot-product (the one discussed before): $\boldsymbol{e}_i = \boldsymbol{s}^T \boldsymbol{h}_i \in \mathbb{R}$
- Multiplicative attention: $\boldsymbol{e}_i = \boldsymbol{s}^T \boldsymbol{W} \boldsymbol{h}_i \in \mathbb{R}$
  - $\boldsymbol{W} \in \mathbb{R}^{d_2 \times d_1}$ - weight matrix
- Additive attention: $\boldsymbol{e}_i = \boldsymbol{v}^T \tanh(\boldsymbol{W}_1 \boldsymbol{h}_i + \boldsymbol{W}_2 \boldsymbol{s}) \in \mathbb{R}$
  - $\boldsymbol{W}_1 \in \mathbb{R}^{d_3 \times d_1}, \boldsymbol{W}_2 \in \mathbb{R}^{d_3 \times d_2}$ - weight matrices
  - $\boldsymbol{v} \in \mathbb{R}^{d_3}$ - weight vector

recap: Attention in seq2seq

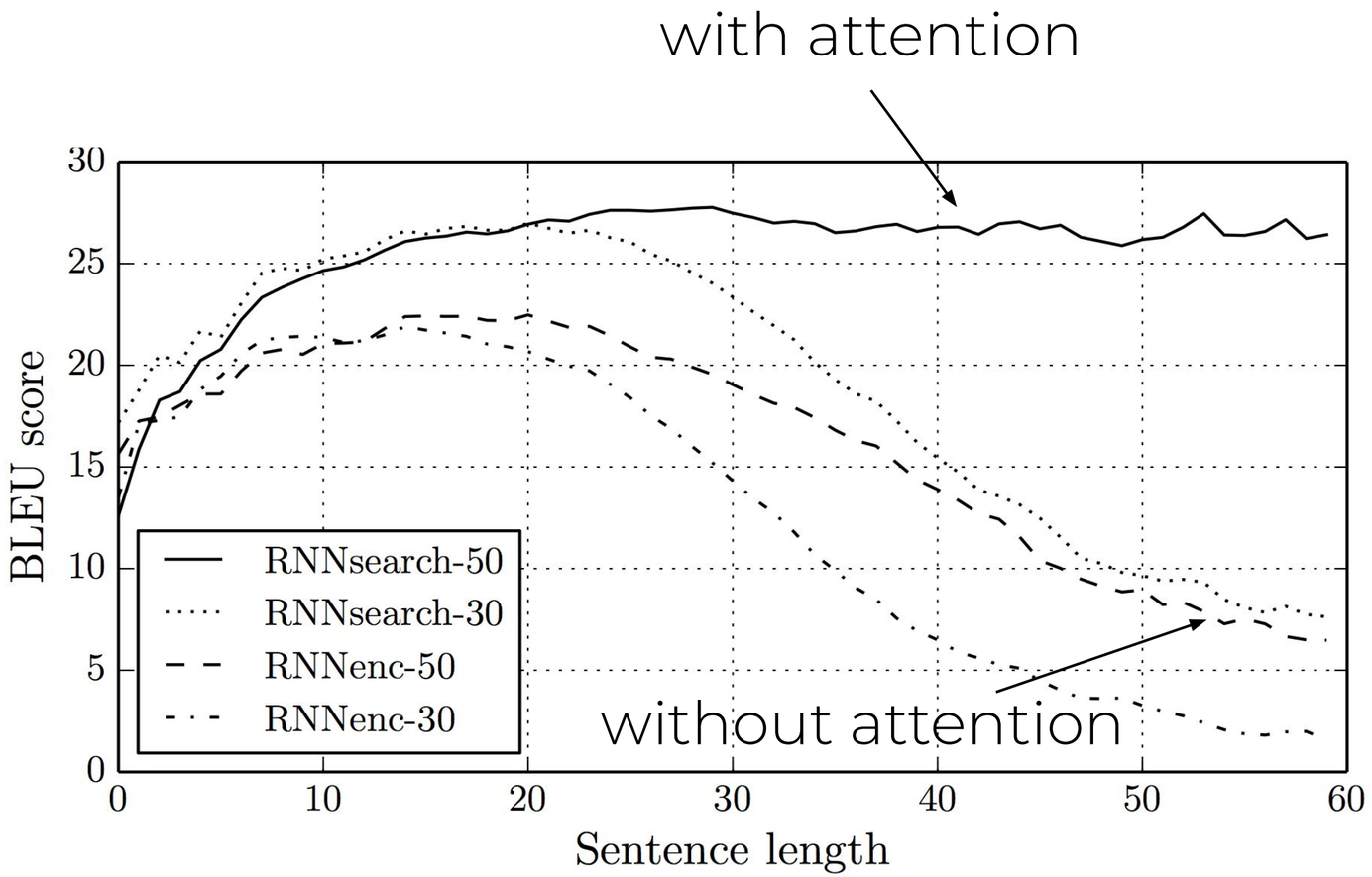


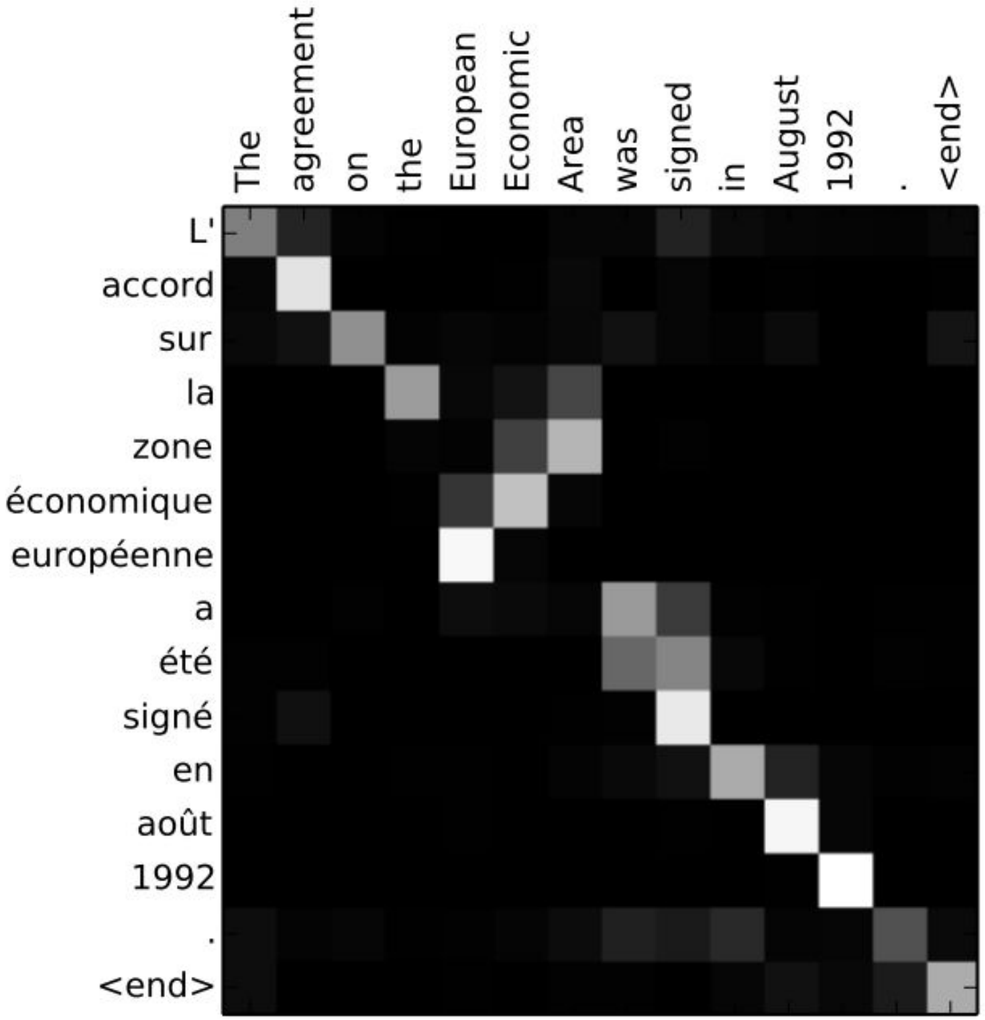
recap: Attention in seq2seq



Attention advantages

- “Free” word alignment
- Better results on long sequences





Self-Attention

girafe ai

04

Self-Attention at a High Level

”The animal didn't cross the street because it was too tired”

- What does “it” in this sentence refer to?

Self-Attention at a High Level

”The animal didn't cross the street because it was too tired”

- What does “it” in this sentence refer to?
- We want self-attention to associate “it” with “animal”

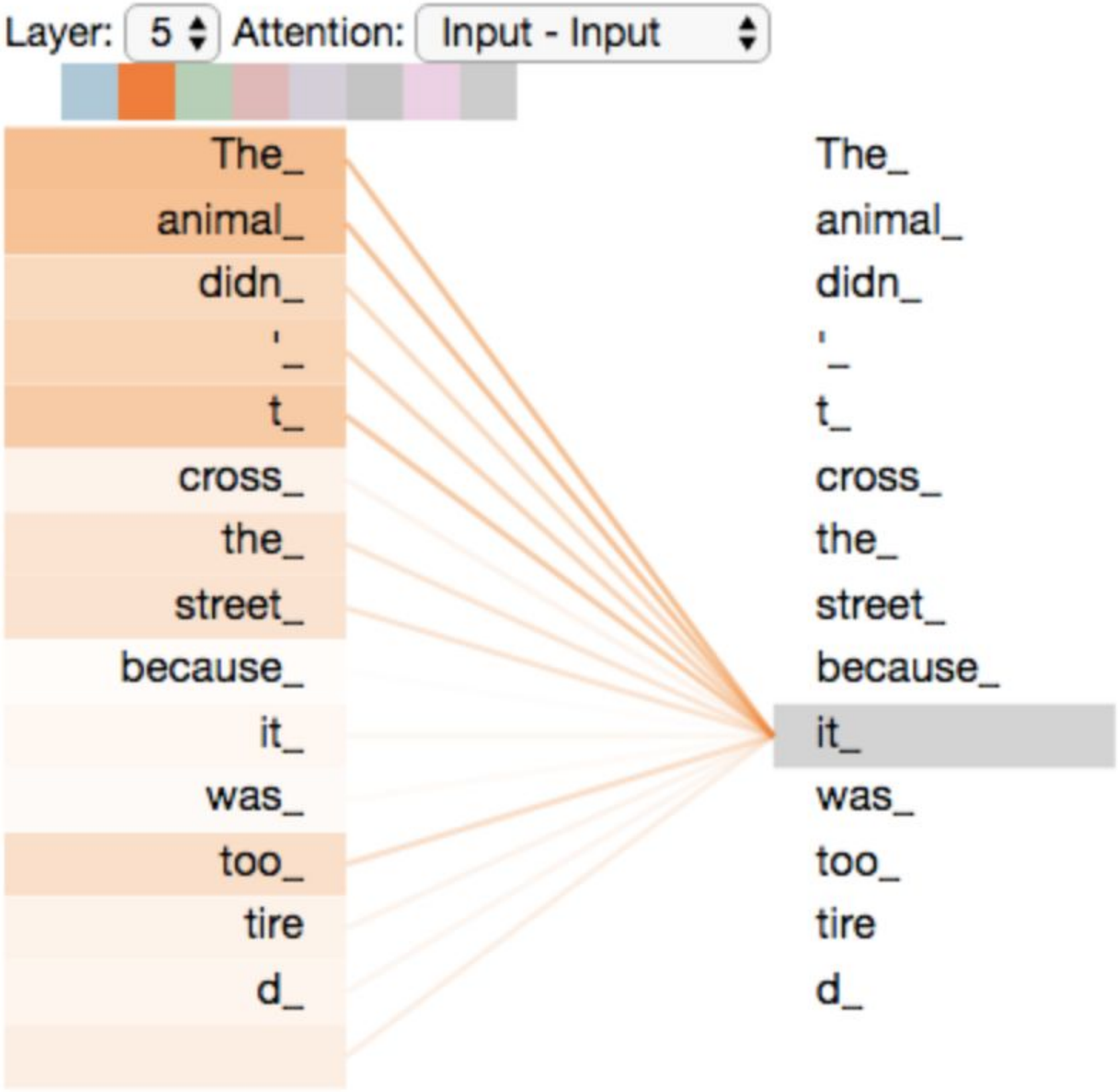
Self-Attention at a High Level

”The animal didn't cross the street because it was too tired”

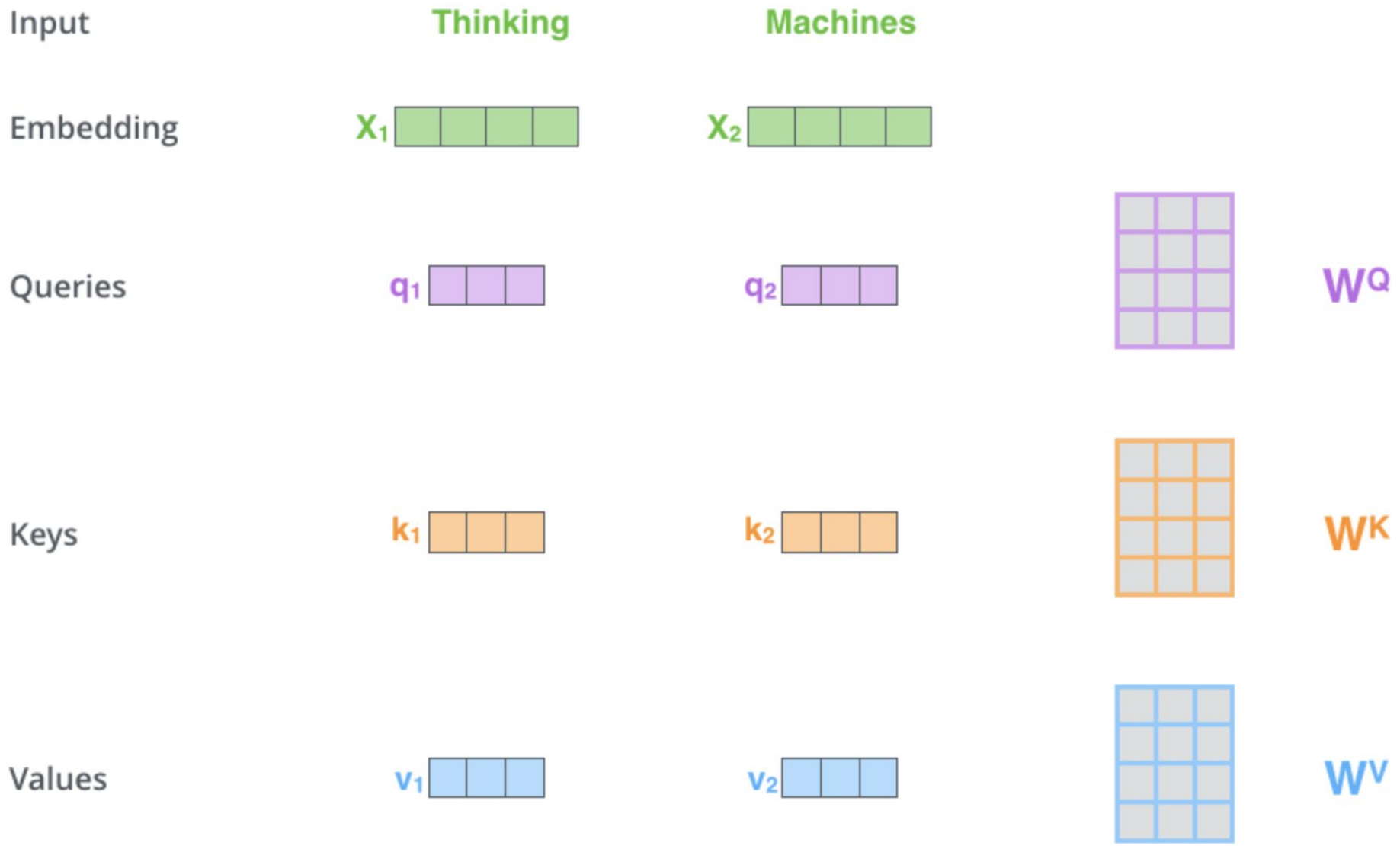
- What does “it” in this sentence refer to?
- We want self-attention to associate “it” with “animal”

- Self-attention is the method the Transformer uses to bake the “understanding” of other relevant words into the one we’re currently processing

Self-Attention at a High Level



Self-Attention: detailed explanation

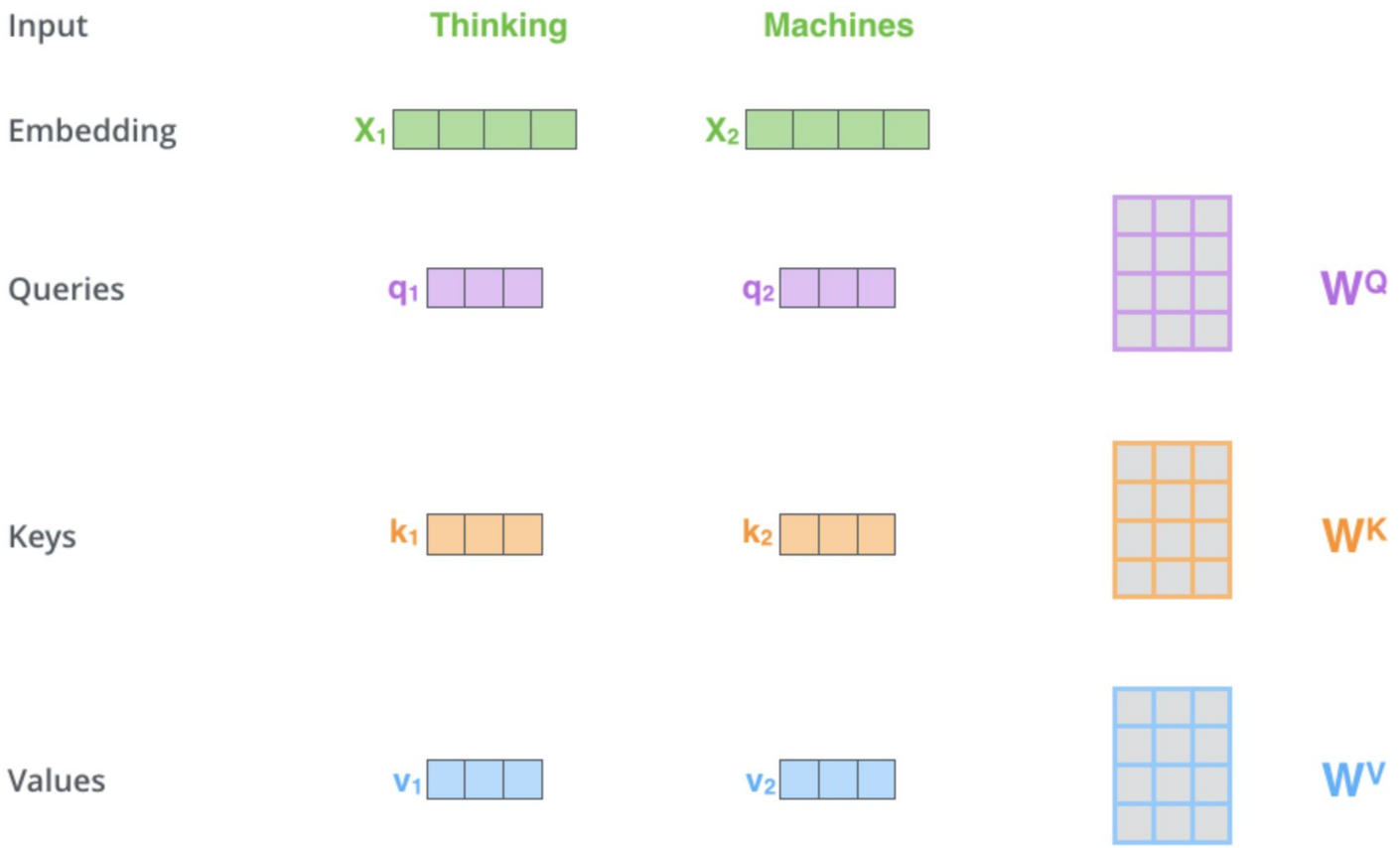


Self-Attention: detailed explanation

STEP 1:

create 3 vectors (Query, Key, Value)

from each of the encoder's input vectors



Self-Attention: detailed explanation

What are the “query”, “key”, and “value” vectors?

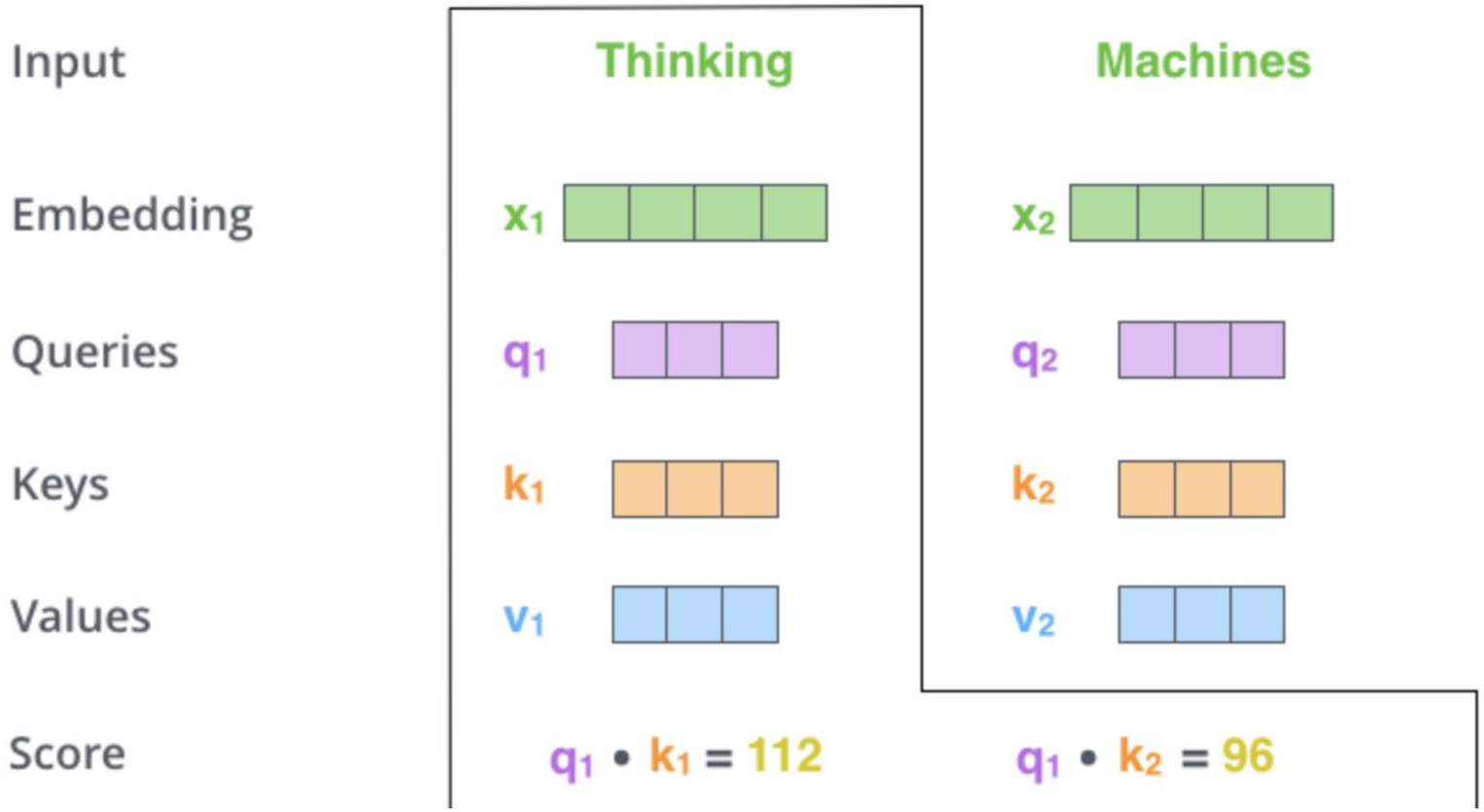
They’re abstractions that are useful for calculating and thinking about attention.

Self-Attention: detailed explanation

STEP 2:

calculate a score

(score each word of the input sentence against the current word)



Self-Attention: detailed explanation

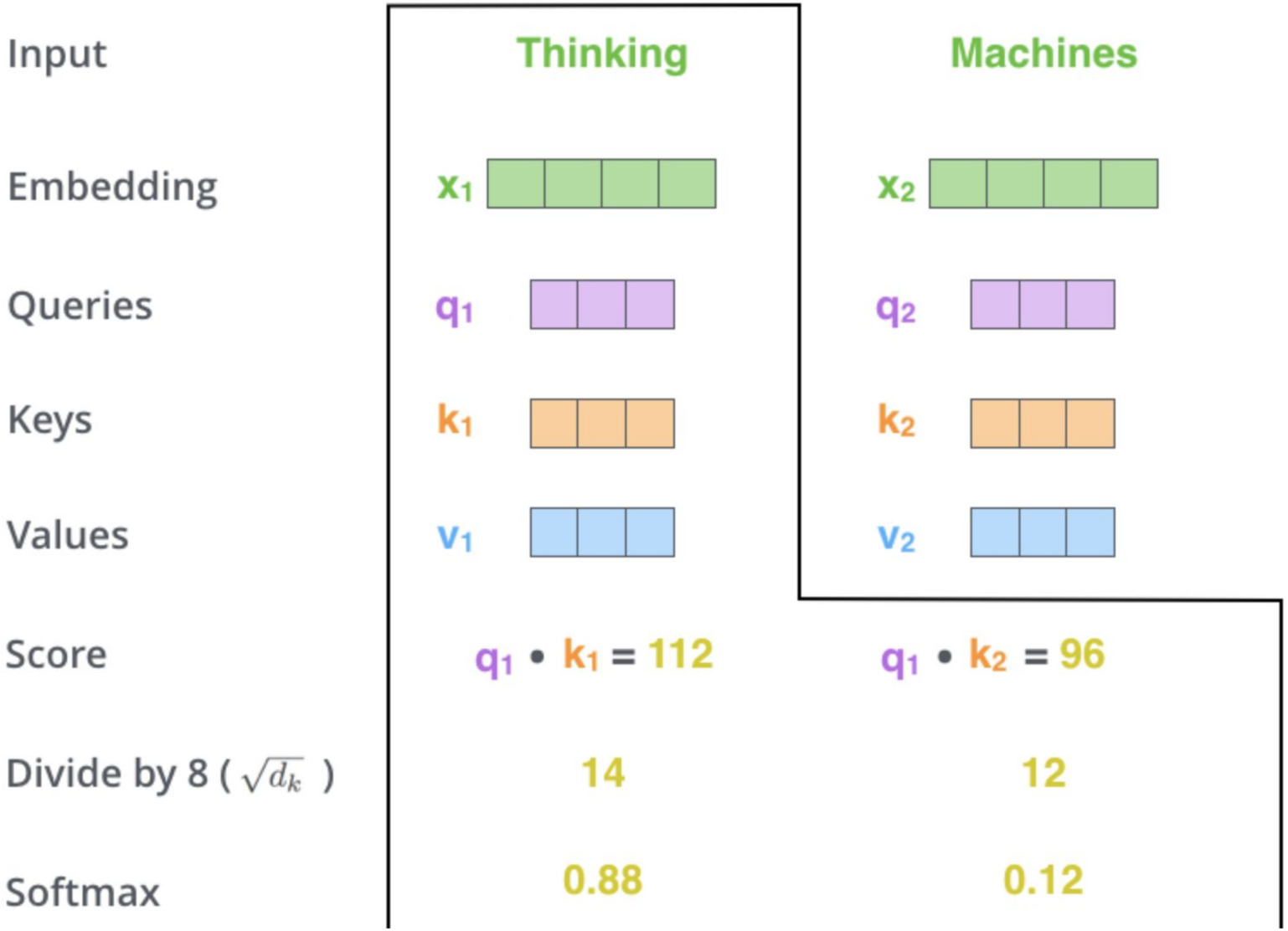
STEP 3:

divide the scores by 8

(the square root of the dimension of the key vectors)

STEP 4:

softmax



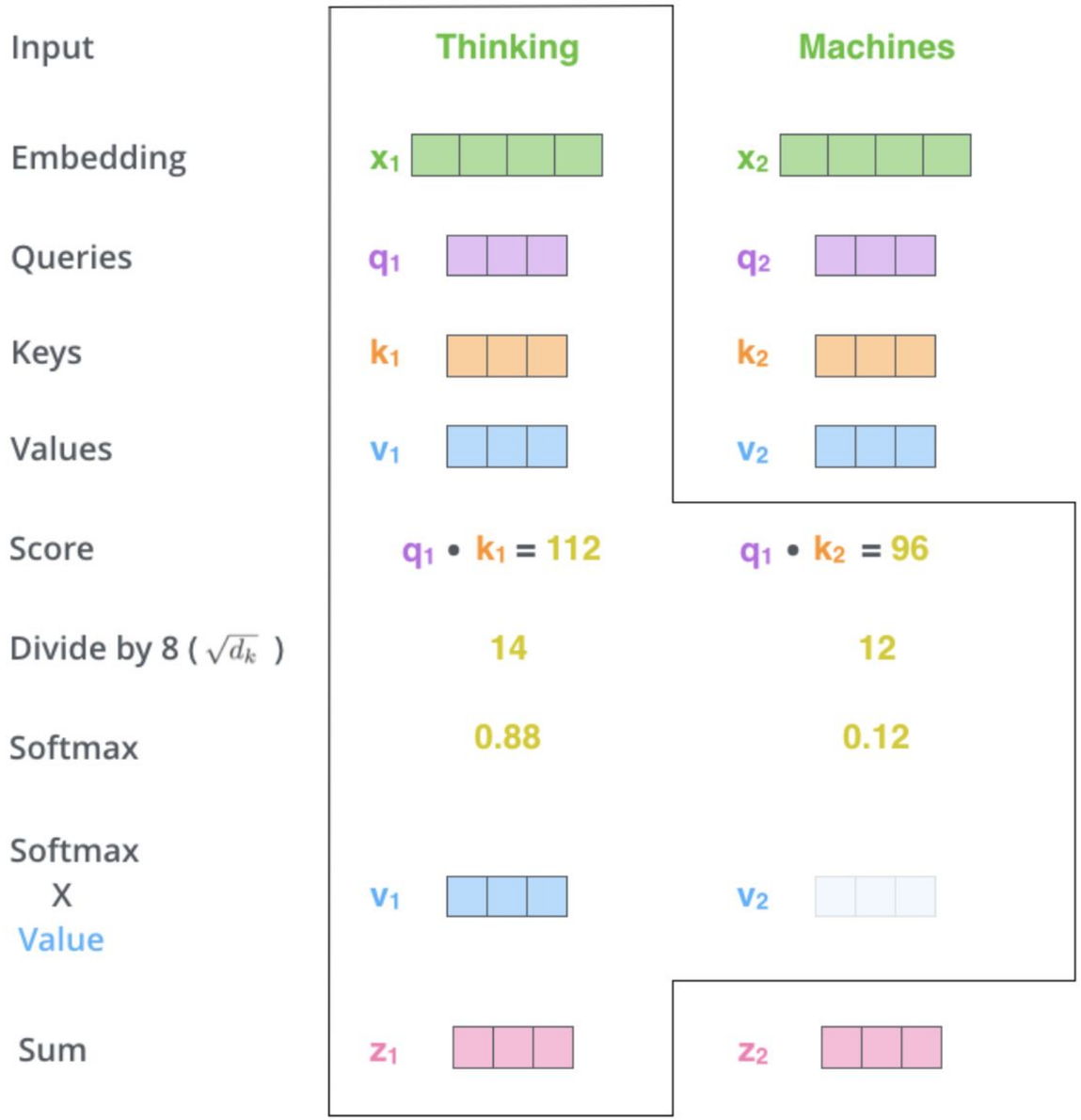
Self-Attention: detailed explanation

STEP 5:

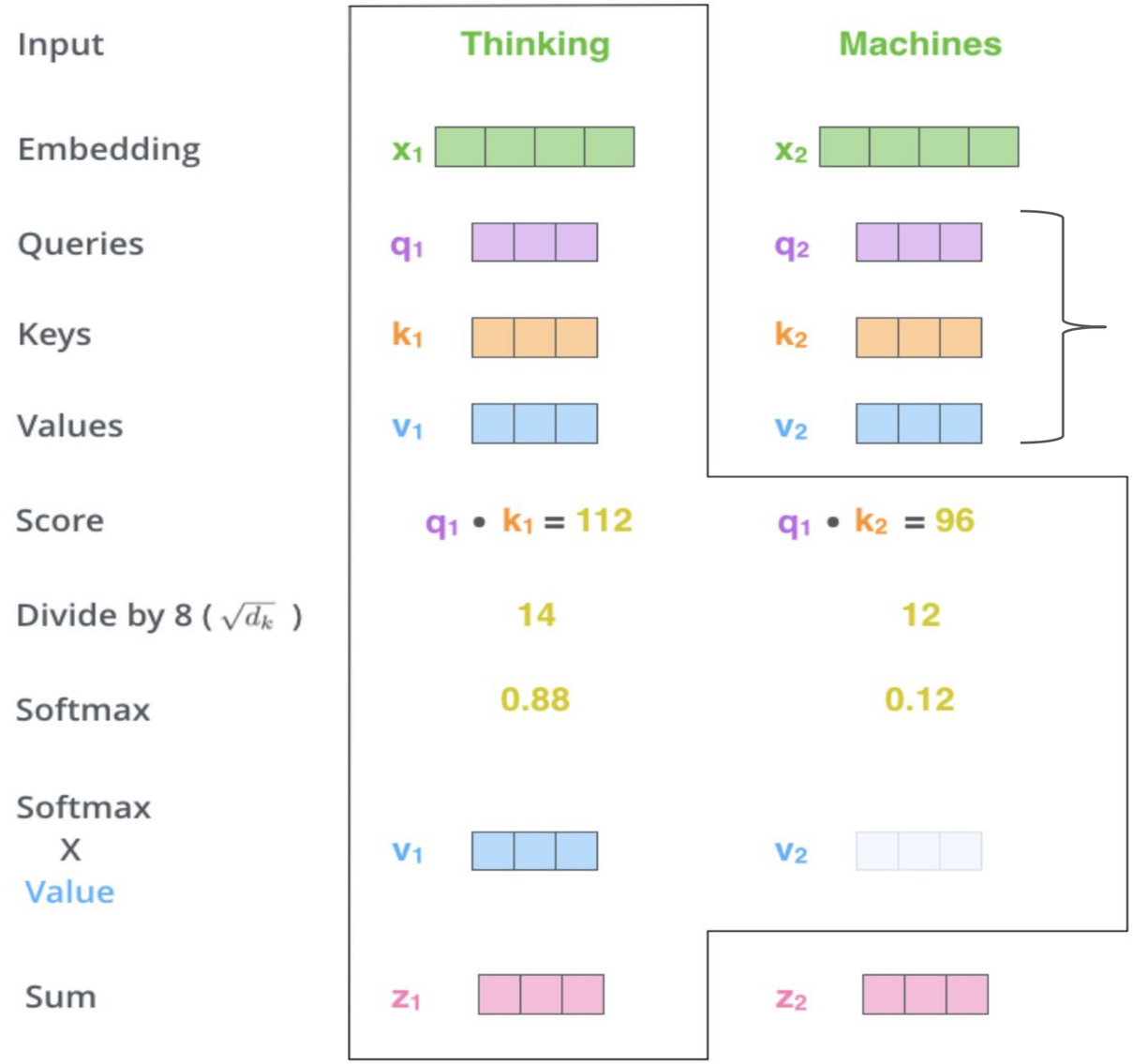
multiply each value vector by the softmax score

STEP 6:

sum up the weighted value vectors



Self-Attention



STEP 1: create Query, Key, Value

STEP 2: calculate scores
STEP 3: divide by $\sqrt{d_k}$
STEP 4: softmax

STEP 5: multiply each value vector by the softmax score

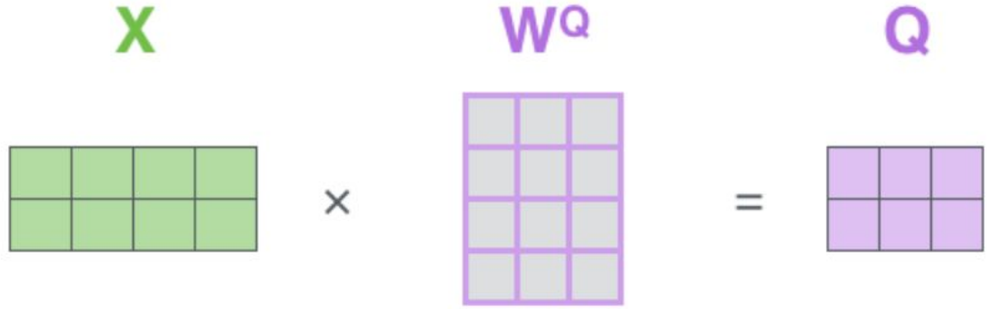
STEP 6: sum up the weighted value vectors

Self-Attention: Matrix Calculation

Pack embeddings into matrix X

Multiply X by weight matrices we've trained (Wk, Wq, Wv)

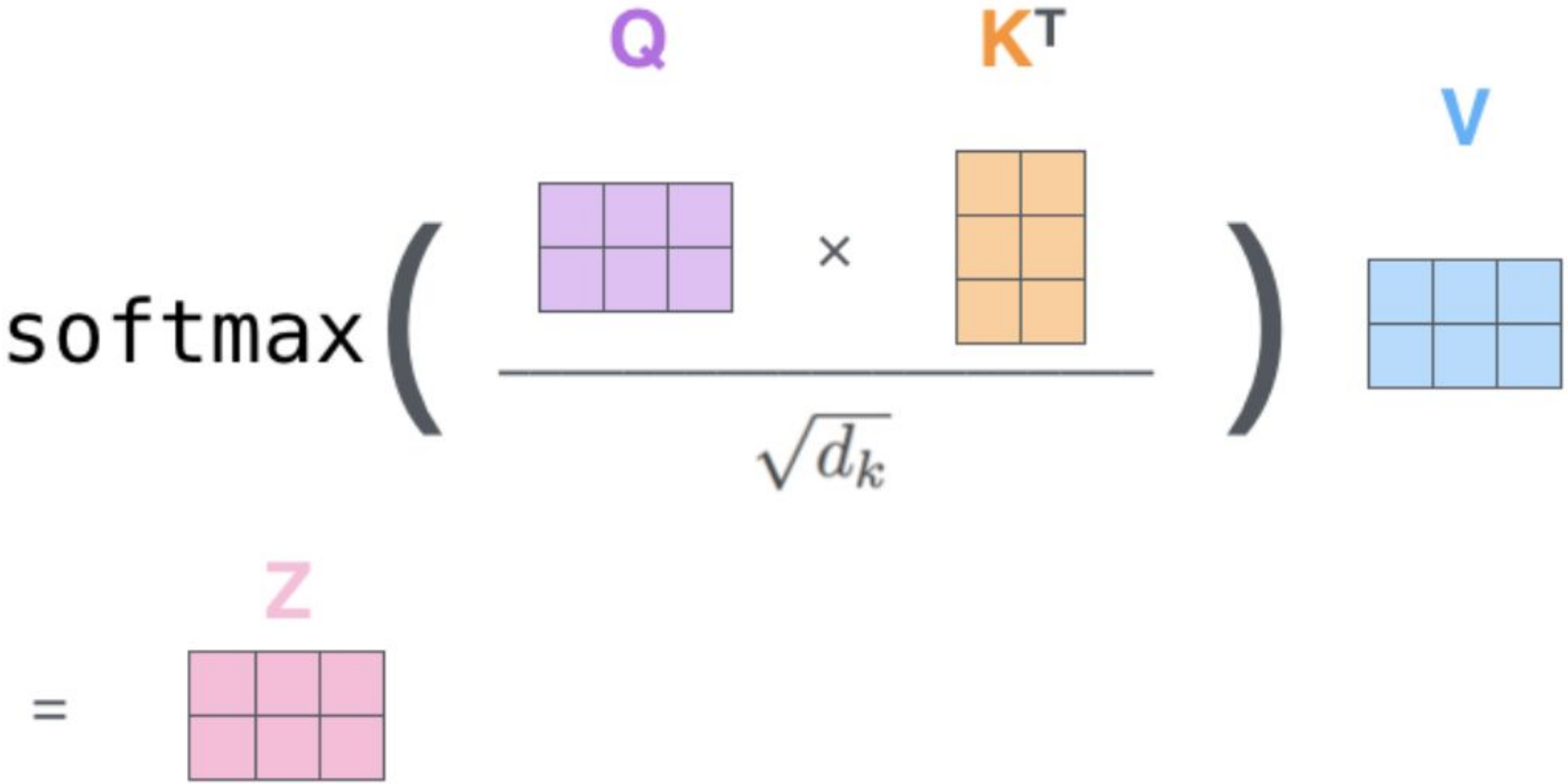
Alternative naming: [cross-attention]



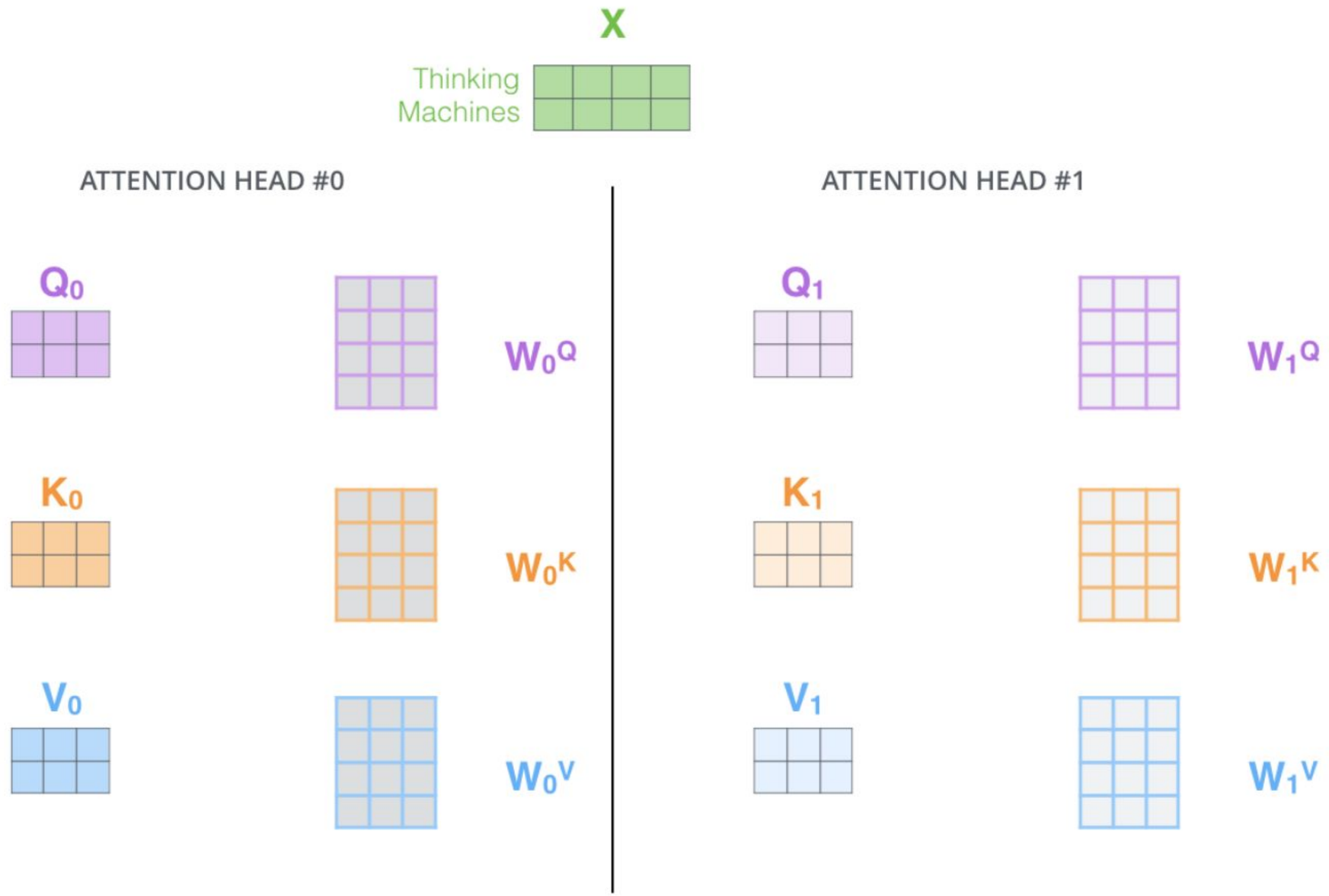




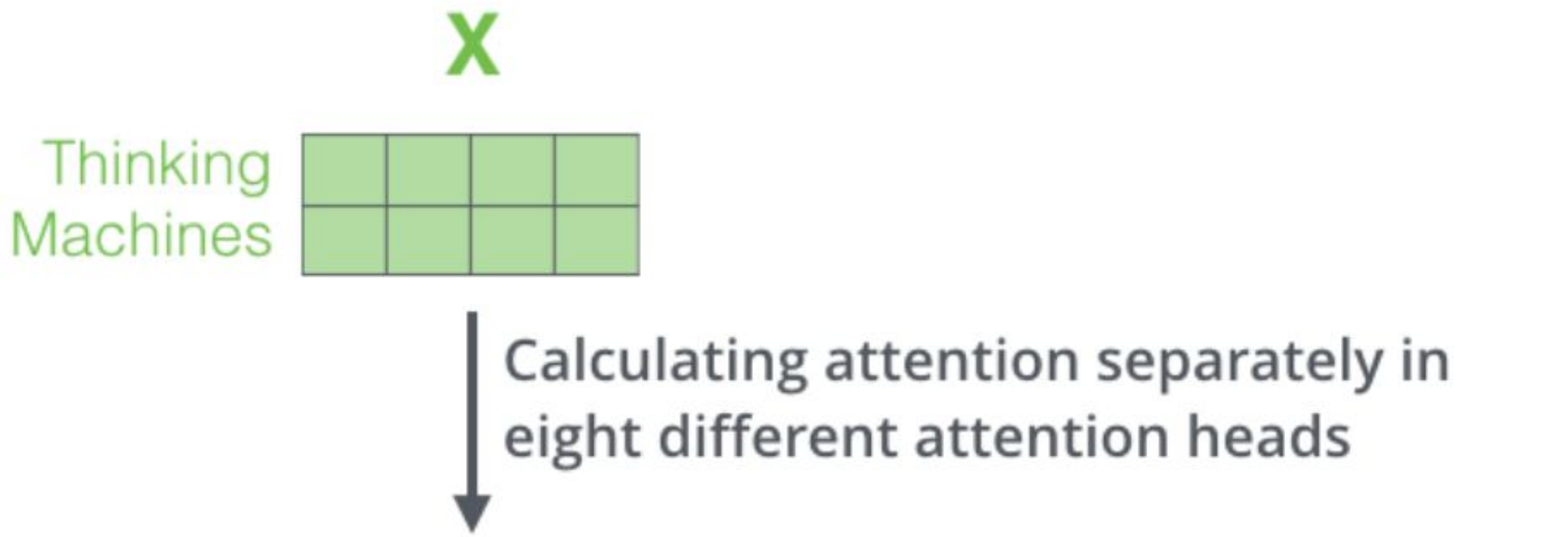
Self-Attention: Matrix Calculation



Multi-Head Attention



Multi-Head Attention



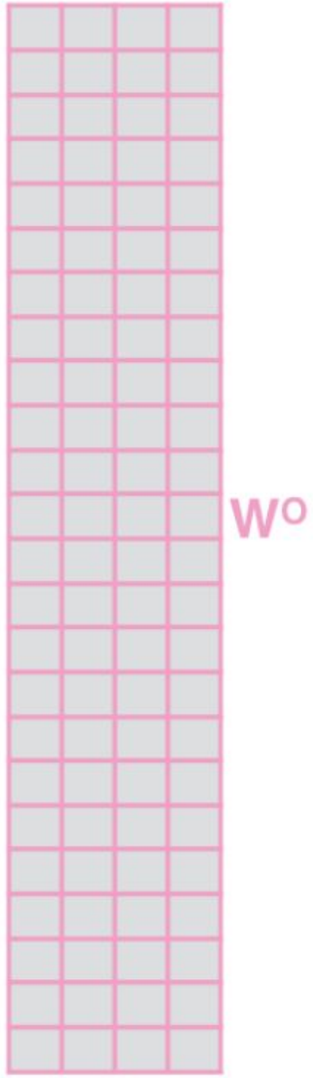
Multi-Head Attention

1) Concatenate all the attention heads



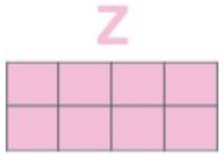
2) Multiply with a weight matrix W^O that was trained jointly with the model

X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

=



1) This is our input sentence*

2) We embed each word*

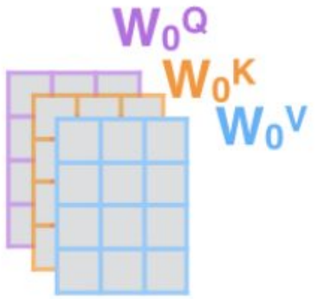
3) Split into 8 heads. We multiply X or R with weight matrices

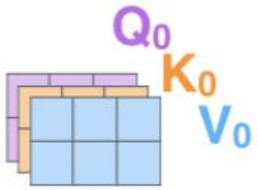
4) Calculate attention using the resulting Q/K/V matrices

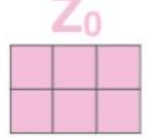
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

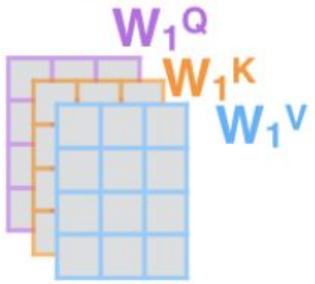
Thinking
Machines

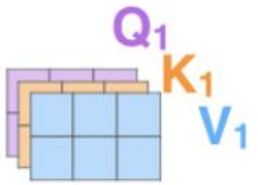


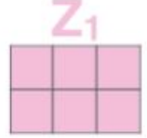




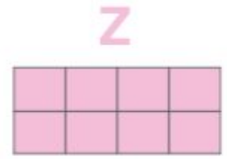








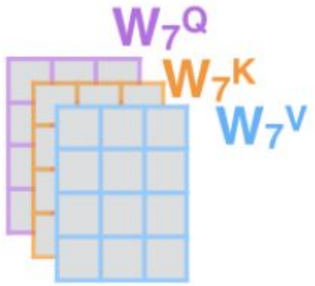




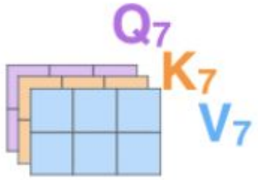
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



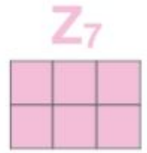
...



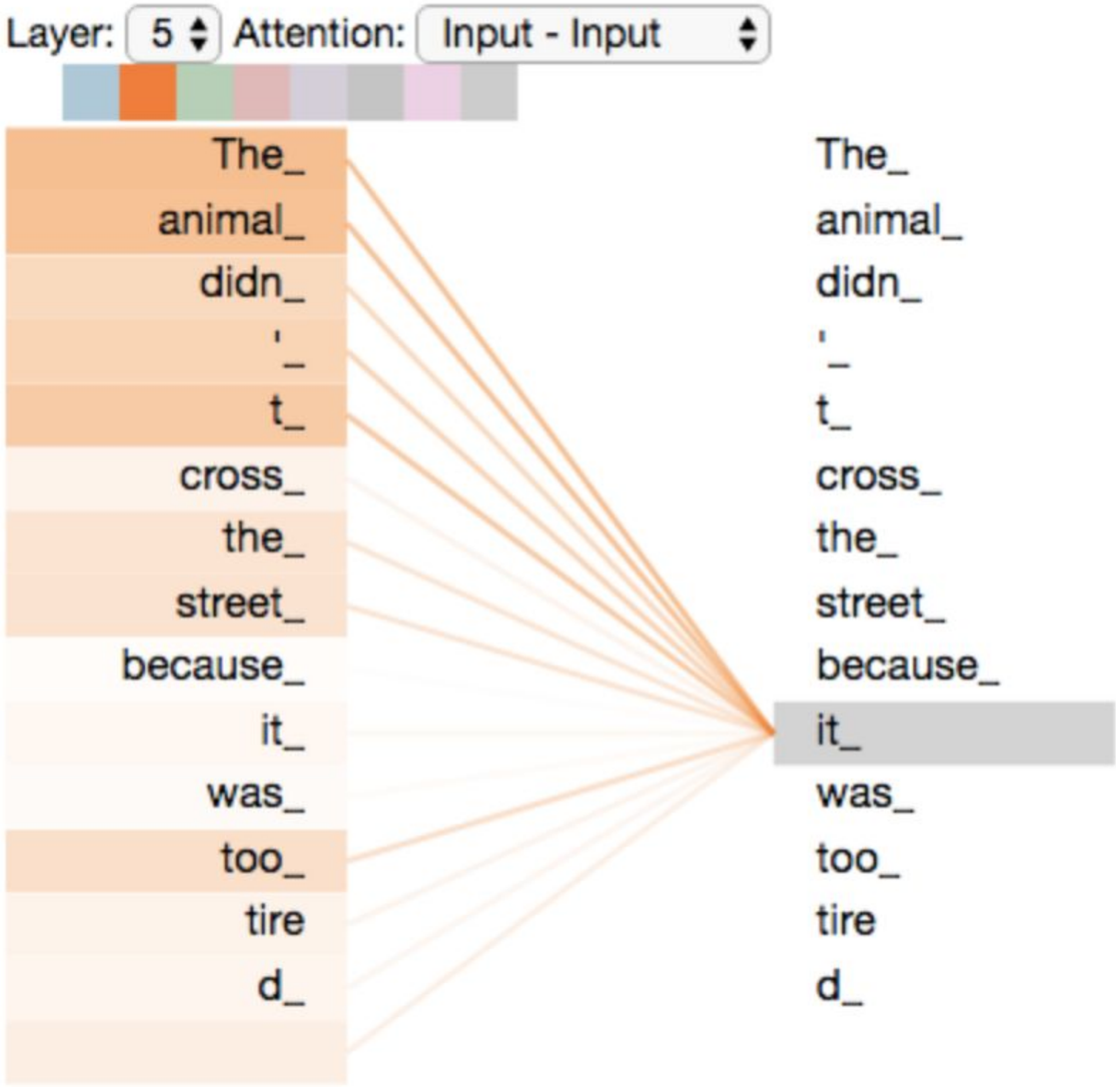
...

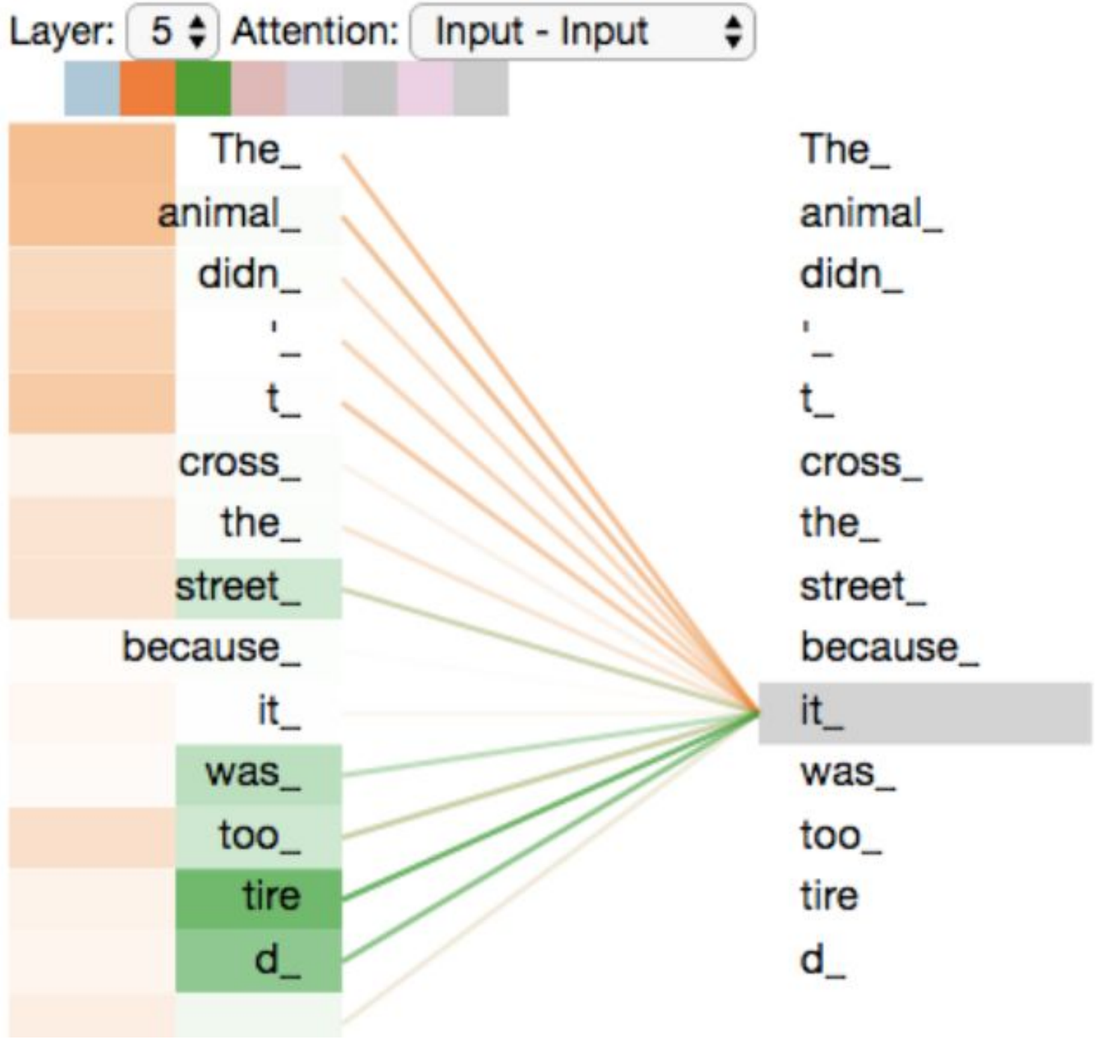


...

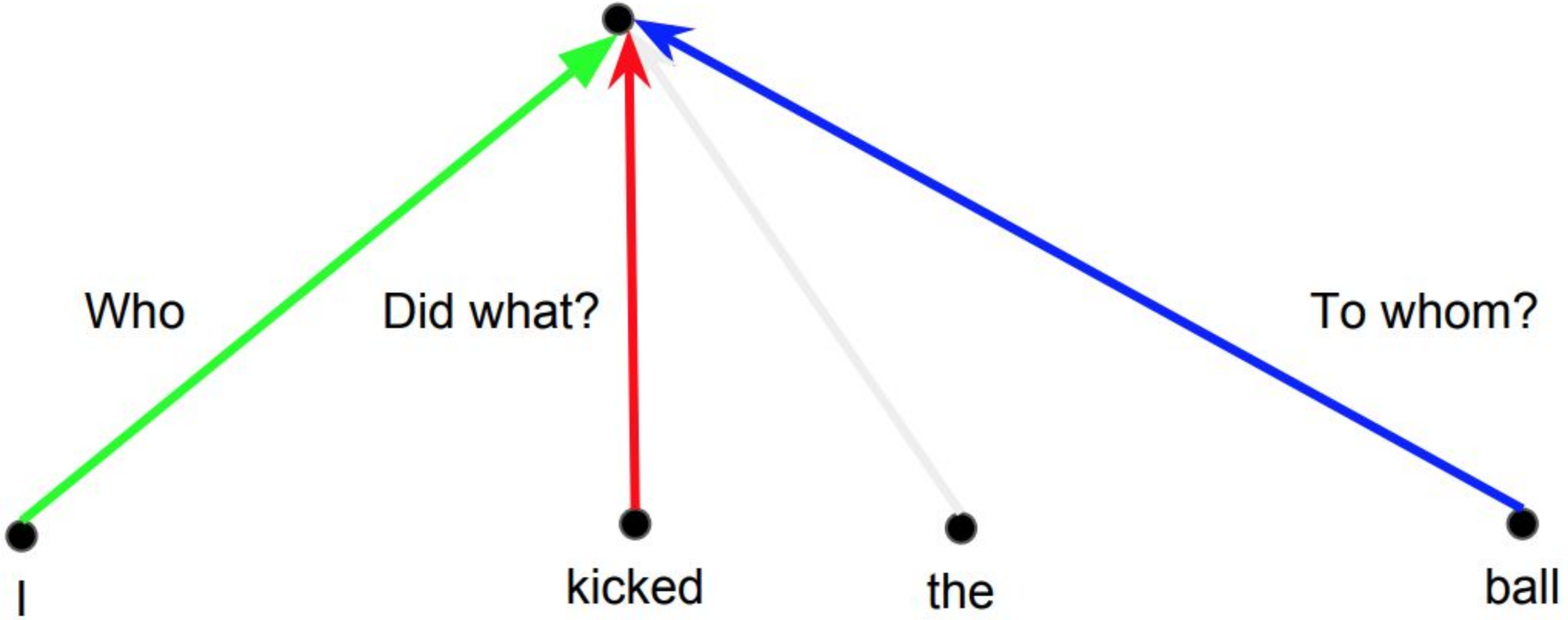


Multi-Head Attention

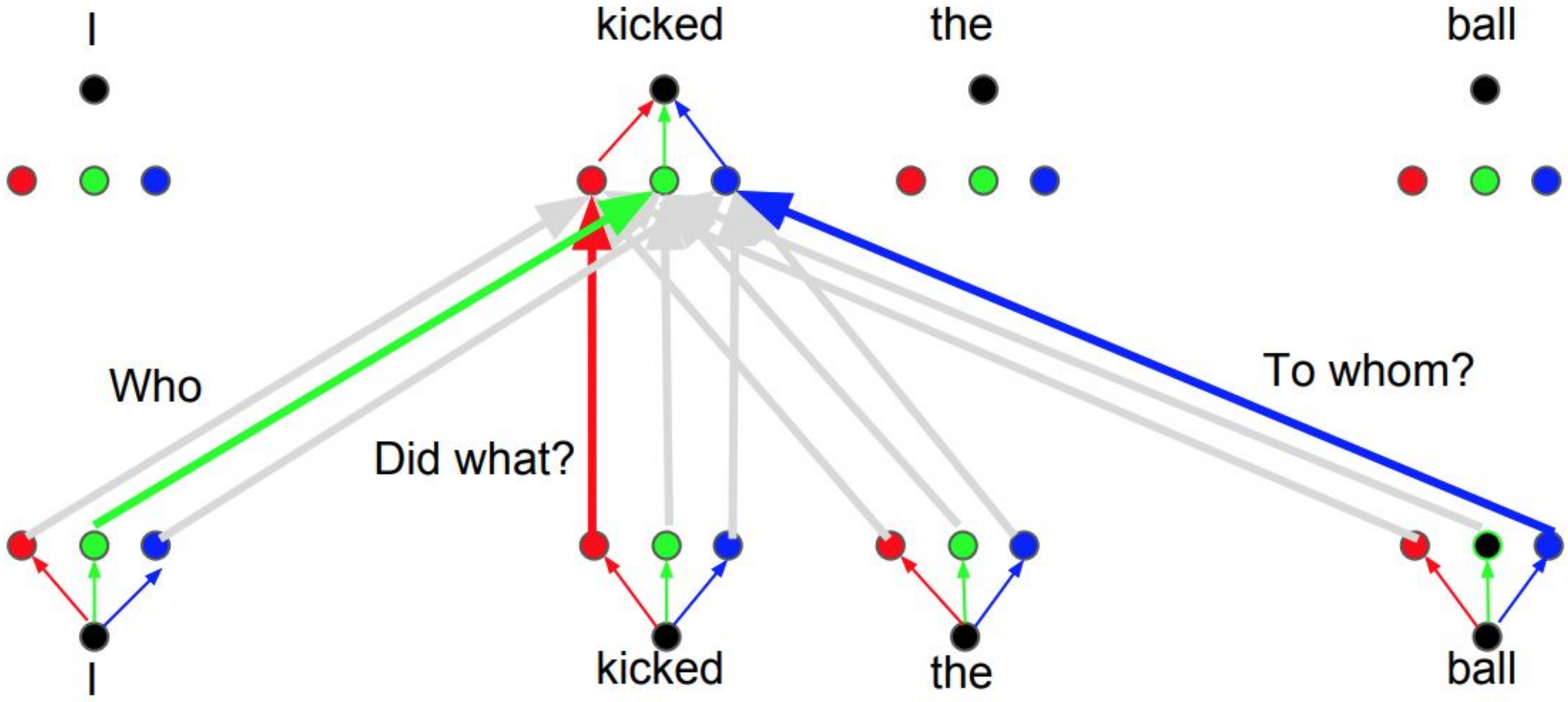




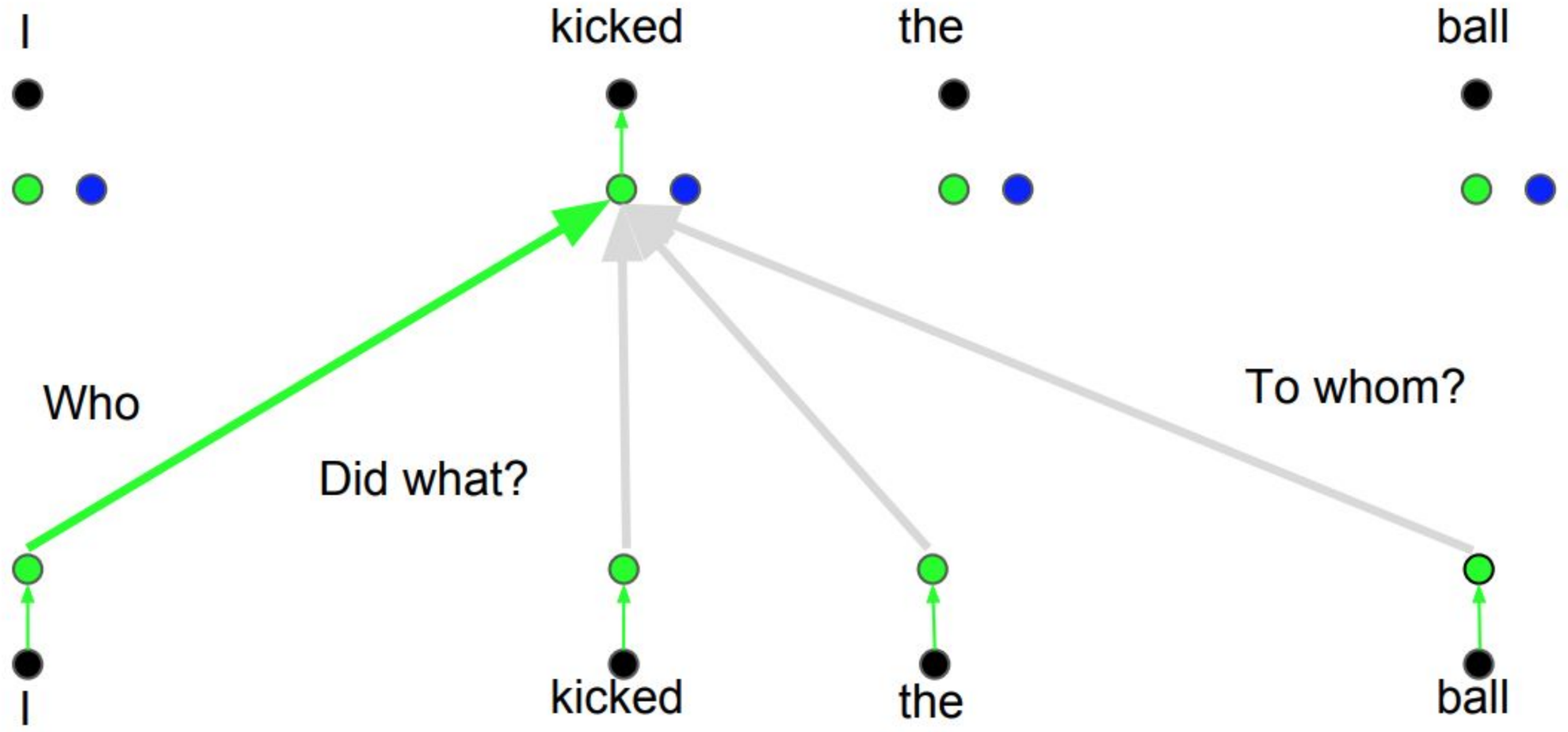
Why Multi-Head Attention?



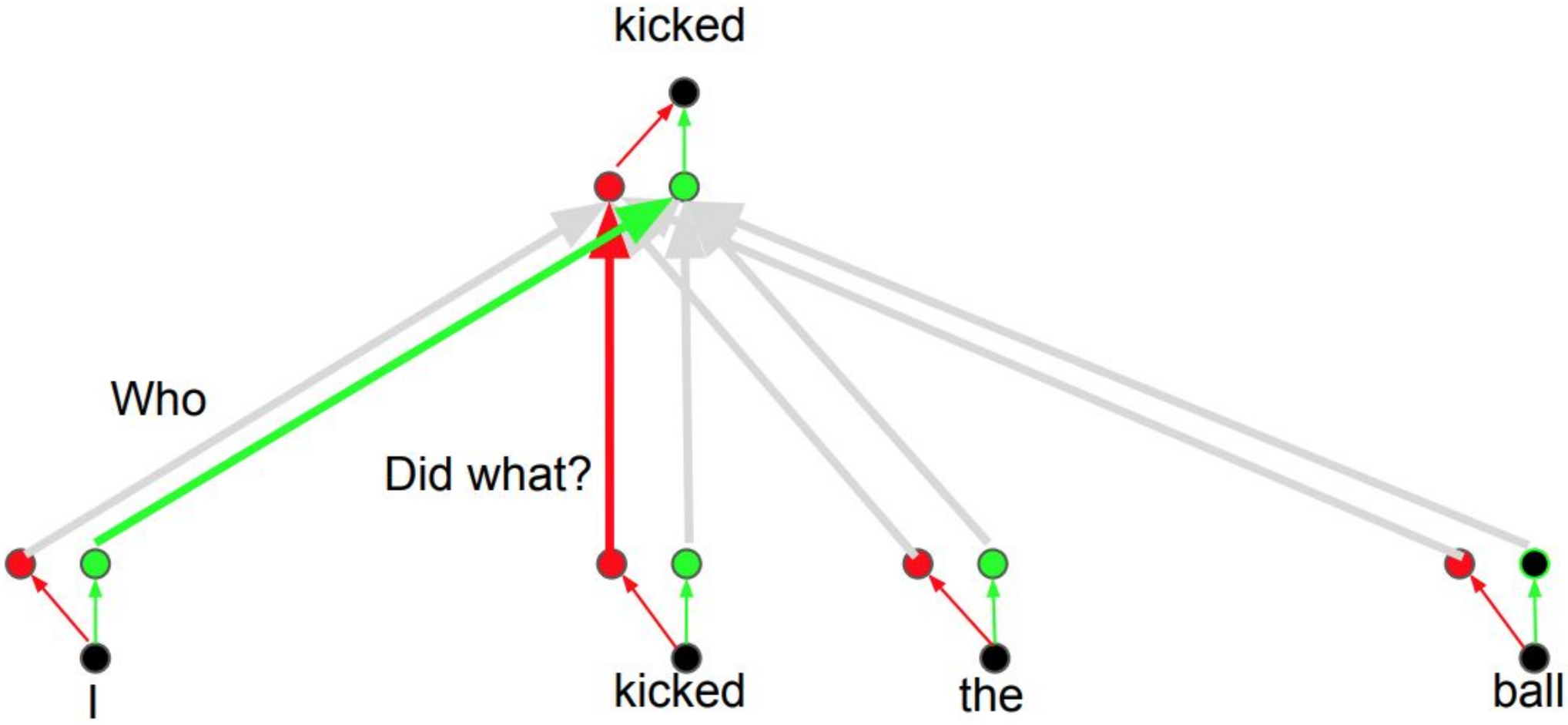
Why Multi-Head Attention?



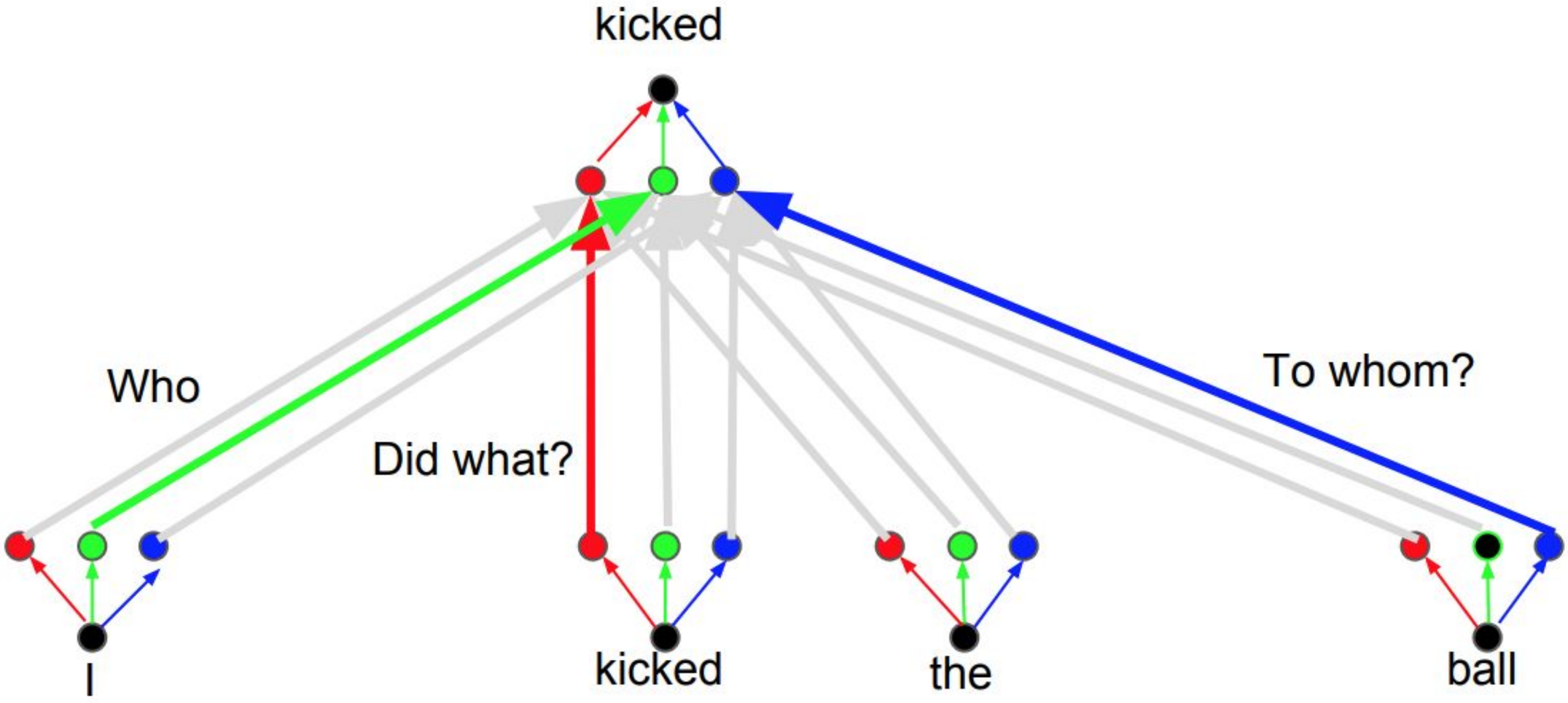
Attention head: Who



Attention head: Did what?

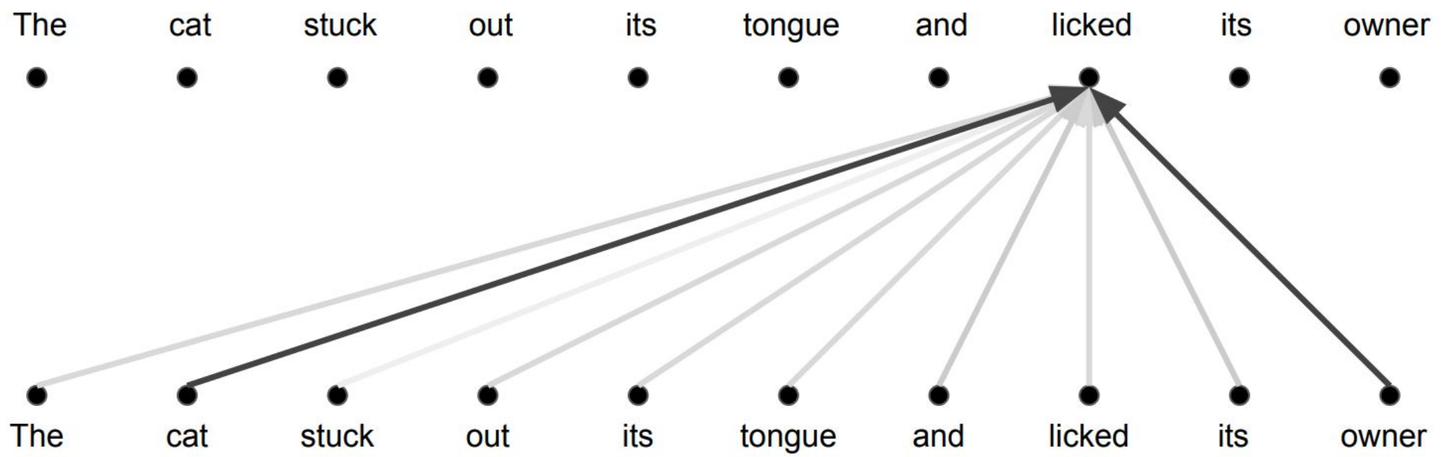


Attention head: To Whom?



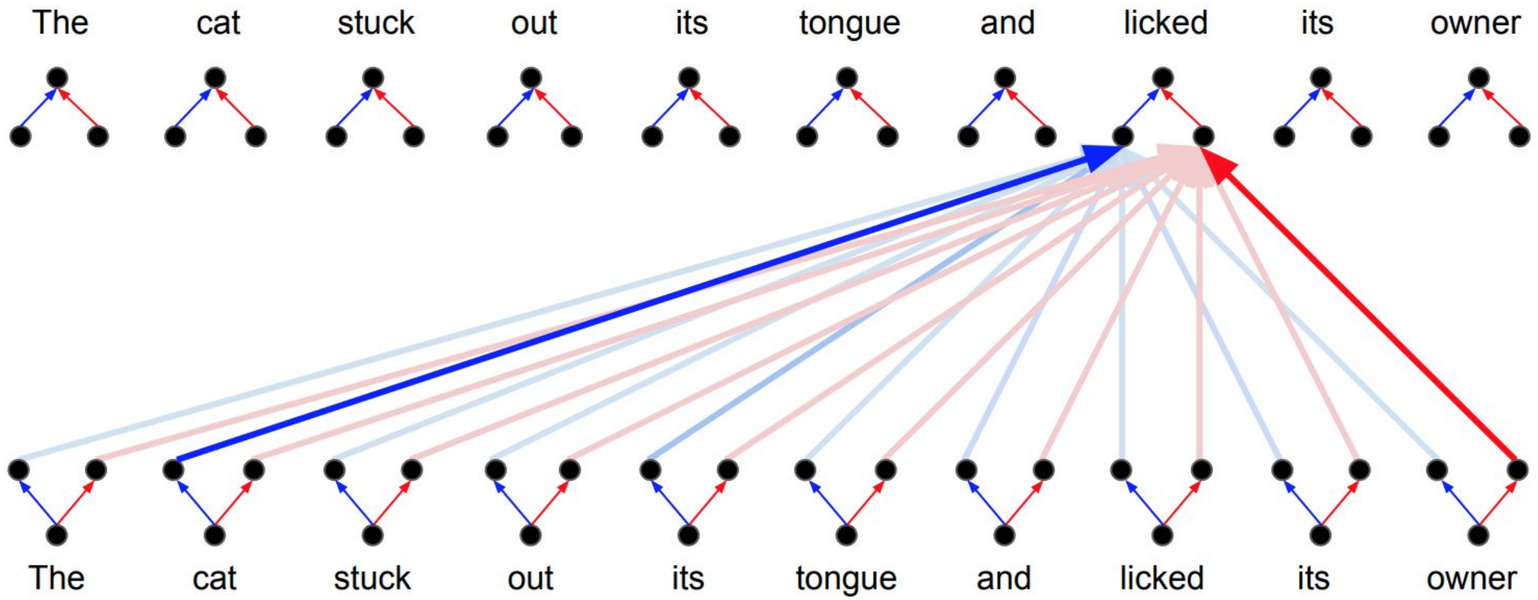
Attention vs. Multi-Head Attention

Attention: a weighted average



Multi-Head Attention:

parallel attention layers with different linear transformations on input and output.



Performance: WMT 2014 BLEU

	EN-DE	EN-FR
GNMT (orig)	24.6	39.9
ConvSeq2Seq	25.2	40.5
Transformer*	**28.4**	**41.8**

*Transformer models trained >3x faster than the others.

Research Challenges

- Constant ‘path length’ between any two positions.
- Unbounded memory.
- Trivial to parallelize (per layer).
- Models Self-Similarity.
- Relative attention provides expressive timing, equivariance, and extends naturally to graphs.

The Transformer

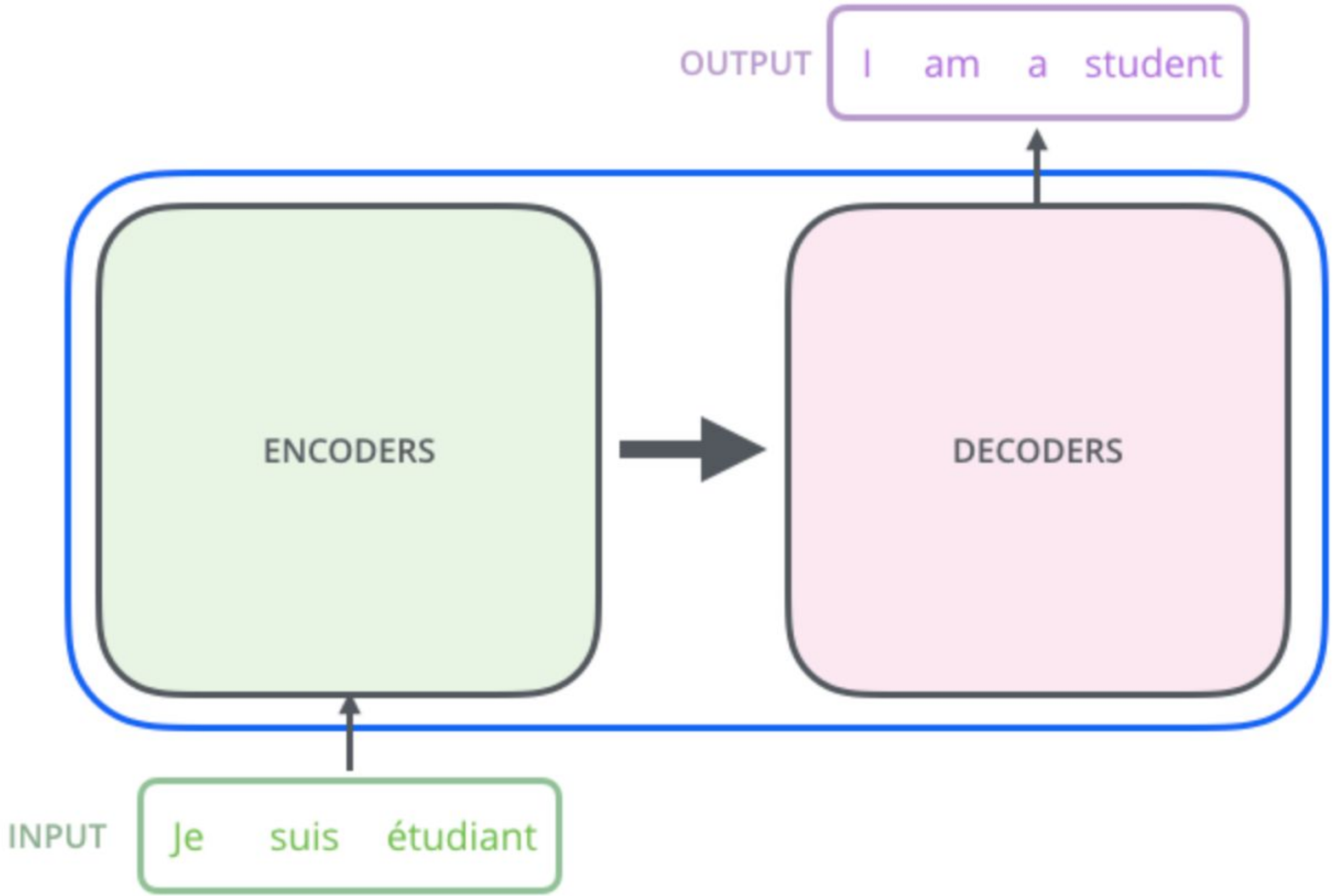
girafe ai

05

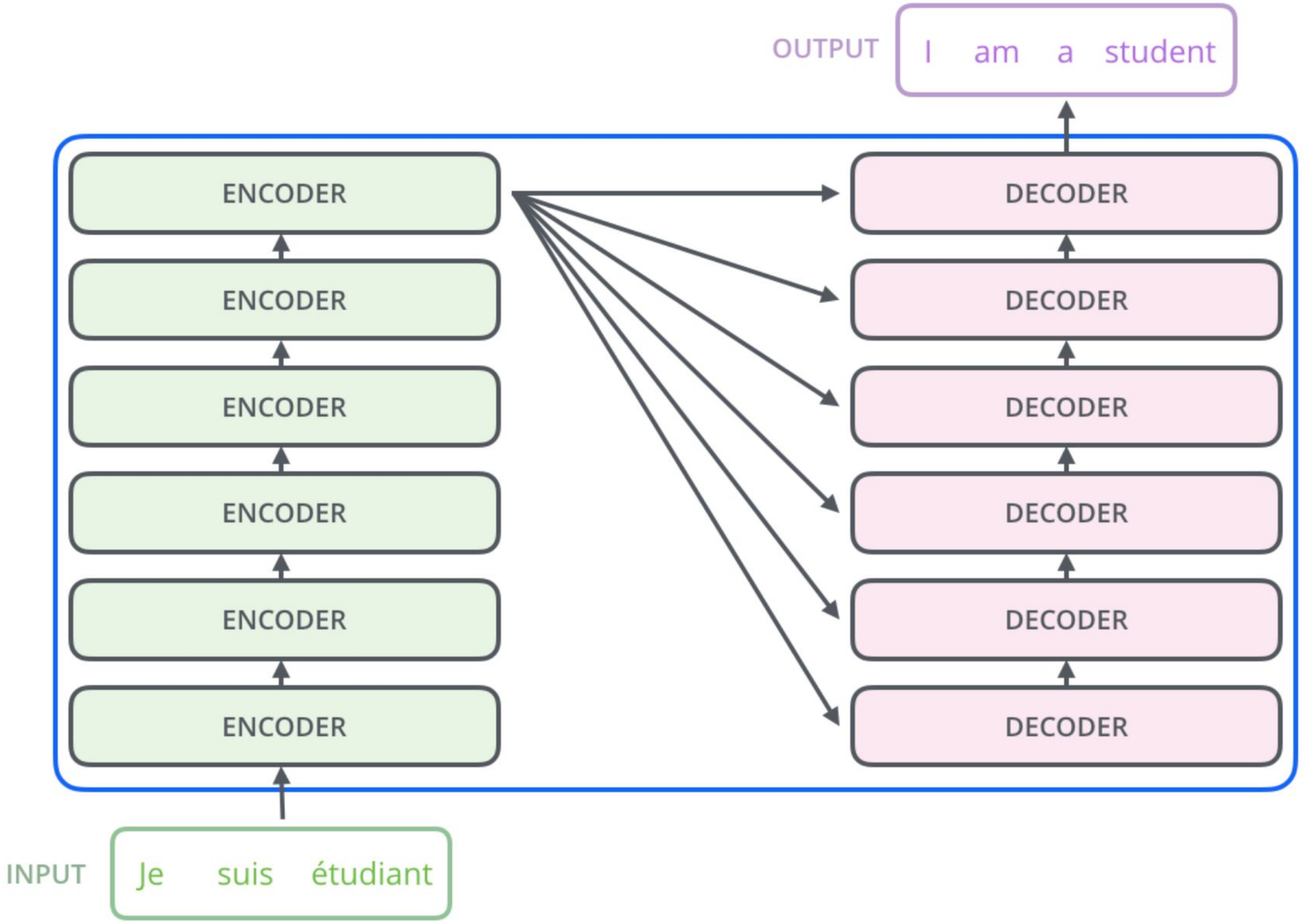
The Transformer



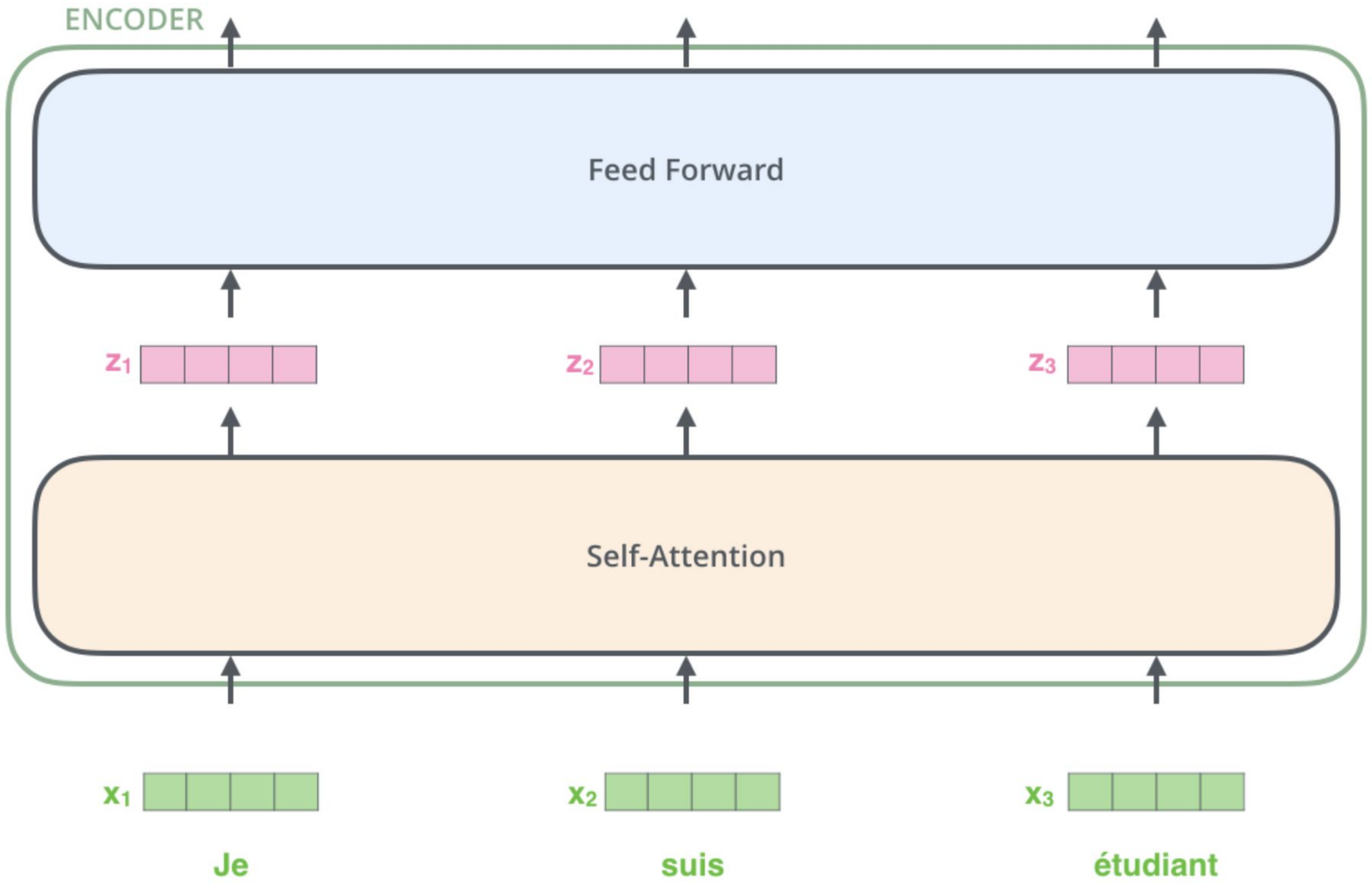
The Transformer



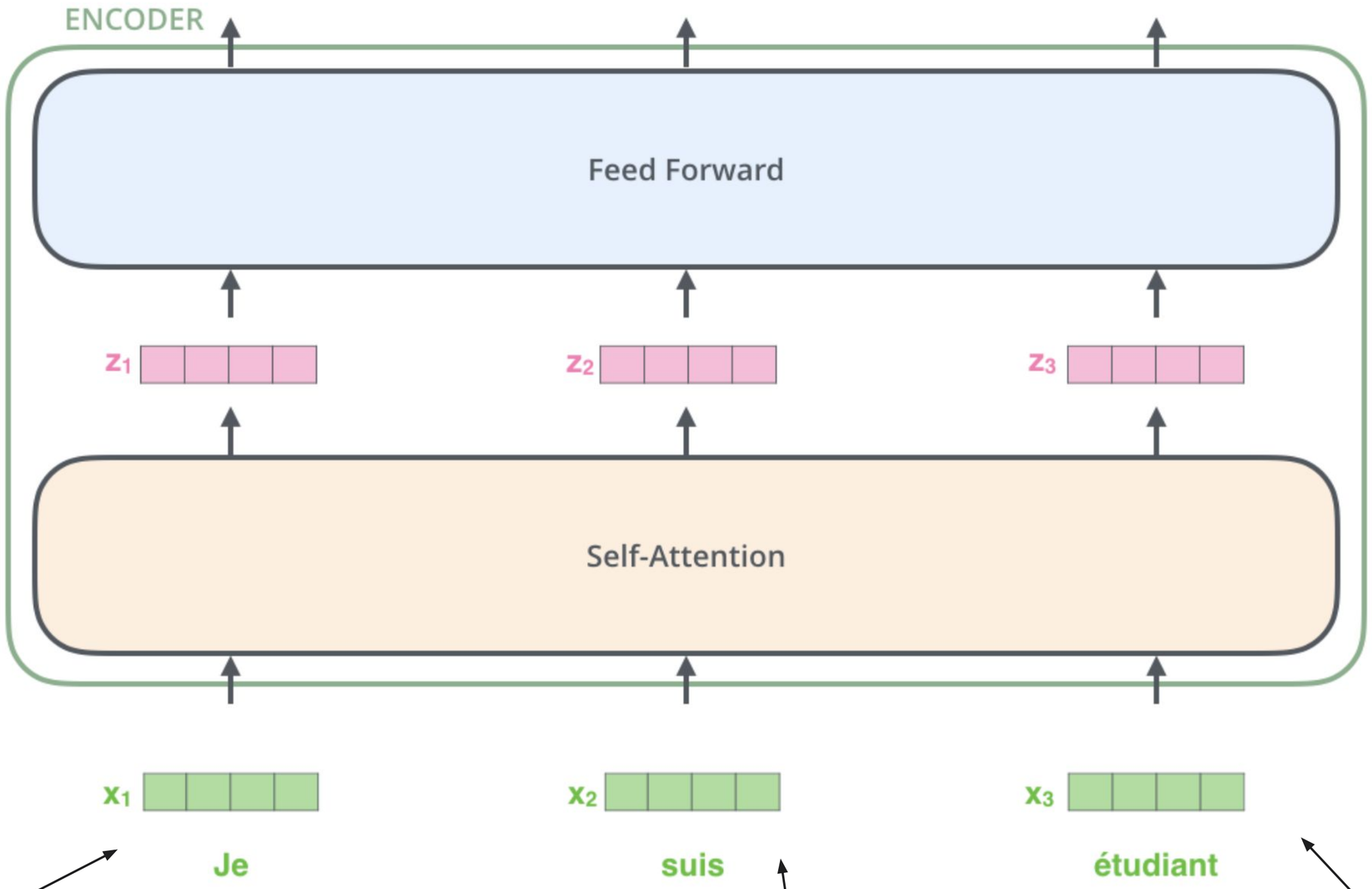
The Transformer



The Encoder Side



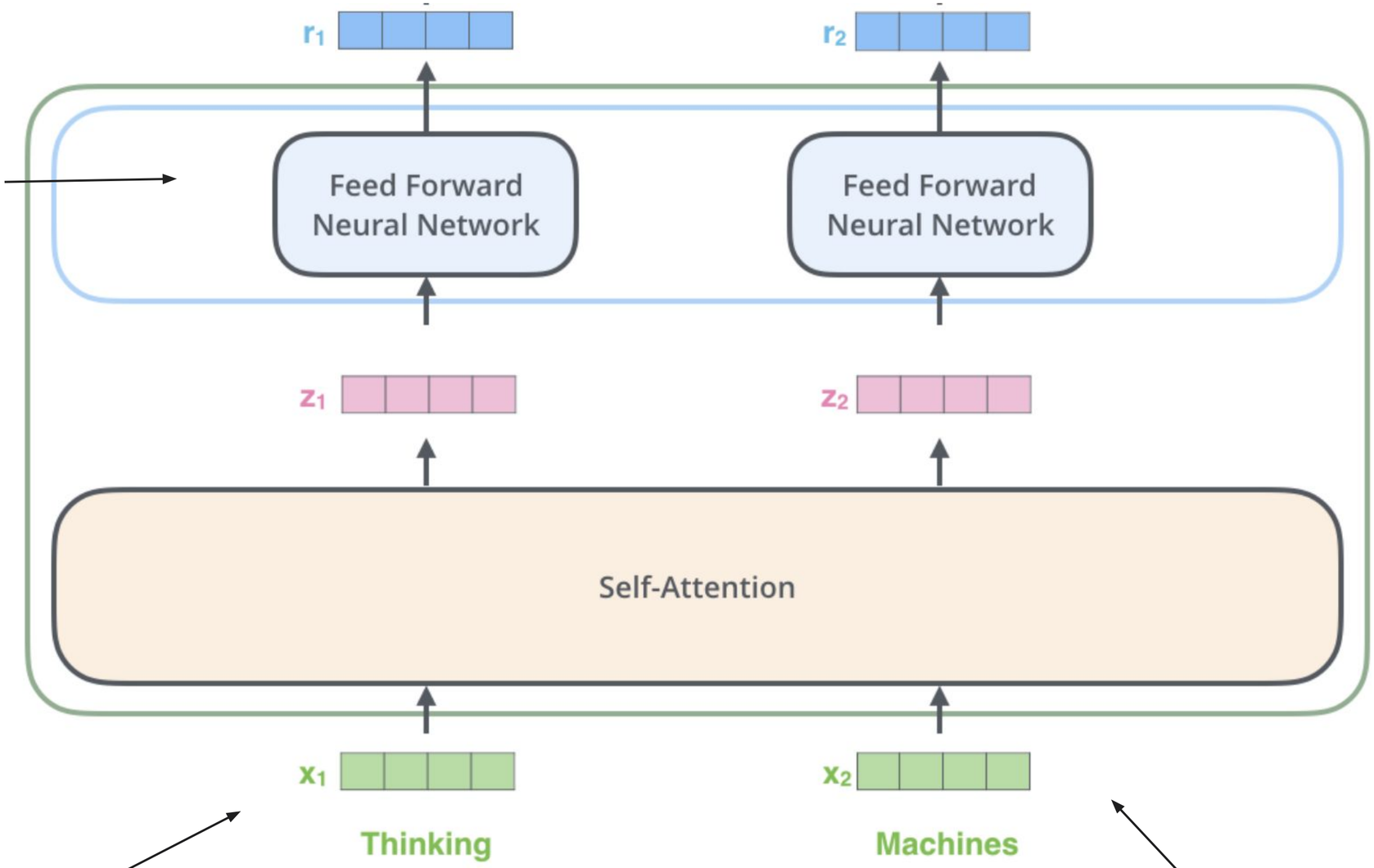
The Encoder Side



the word in each position flows through its own path in the encoder

The Encoder Side

Can be parallelized



the word in each position flows through its own path in the encoder

The Transformer: quick overview

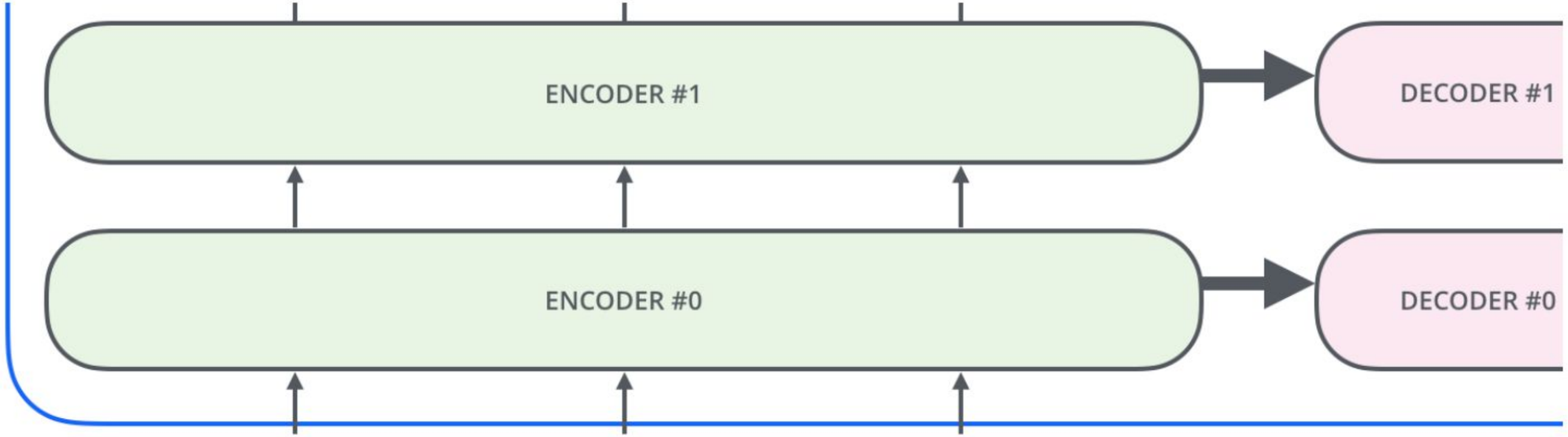
- Proposed in the paper “Attention is All You Need” (Ashish Vaswani et al.)
- No recurrent or convolutional neural networks -> just attention
- Beats seq2seq in machine translation task
  - 28.4 BLEU on the WMT 2014 English-to-German translation task
- Much faster
- Uses self-attention concept

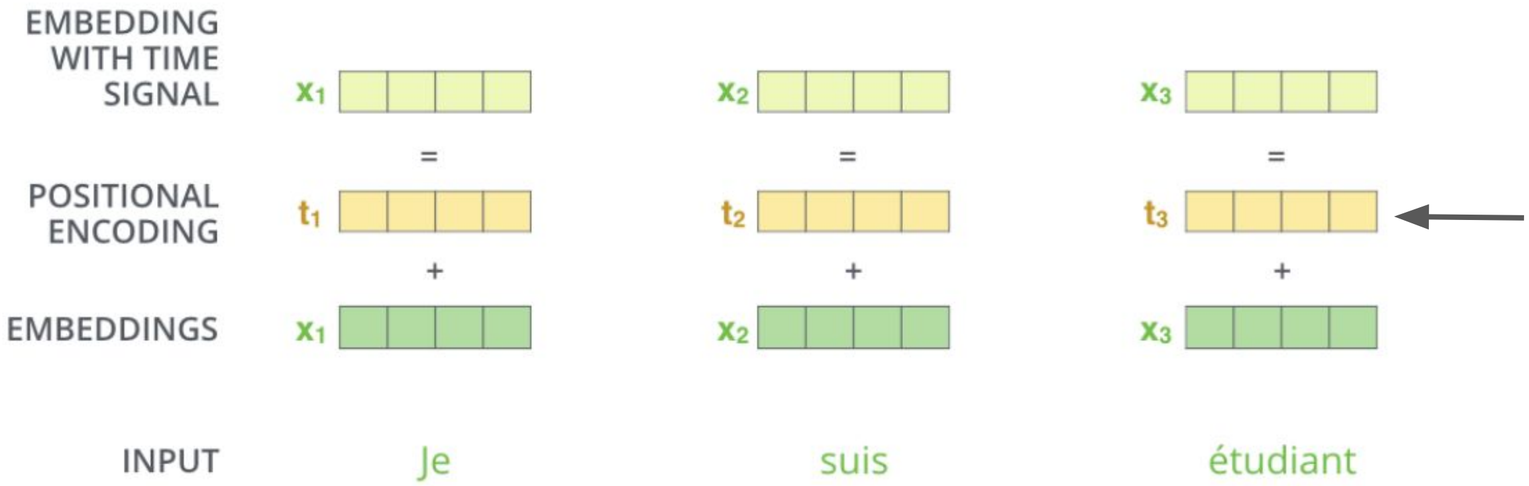
Positional Encoding

girafe ai

06

Positional Encoding





It provides meaningful distances between the embedding vectors once they're projected into Q/K/V vectors and during dot-product attention

Positional Encoding

$$PE_{(pos,\ 2i)} = sin(pos\ /\ 10000^{2i\ /\ d_{\text{model}}})$$

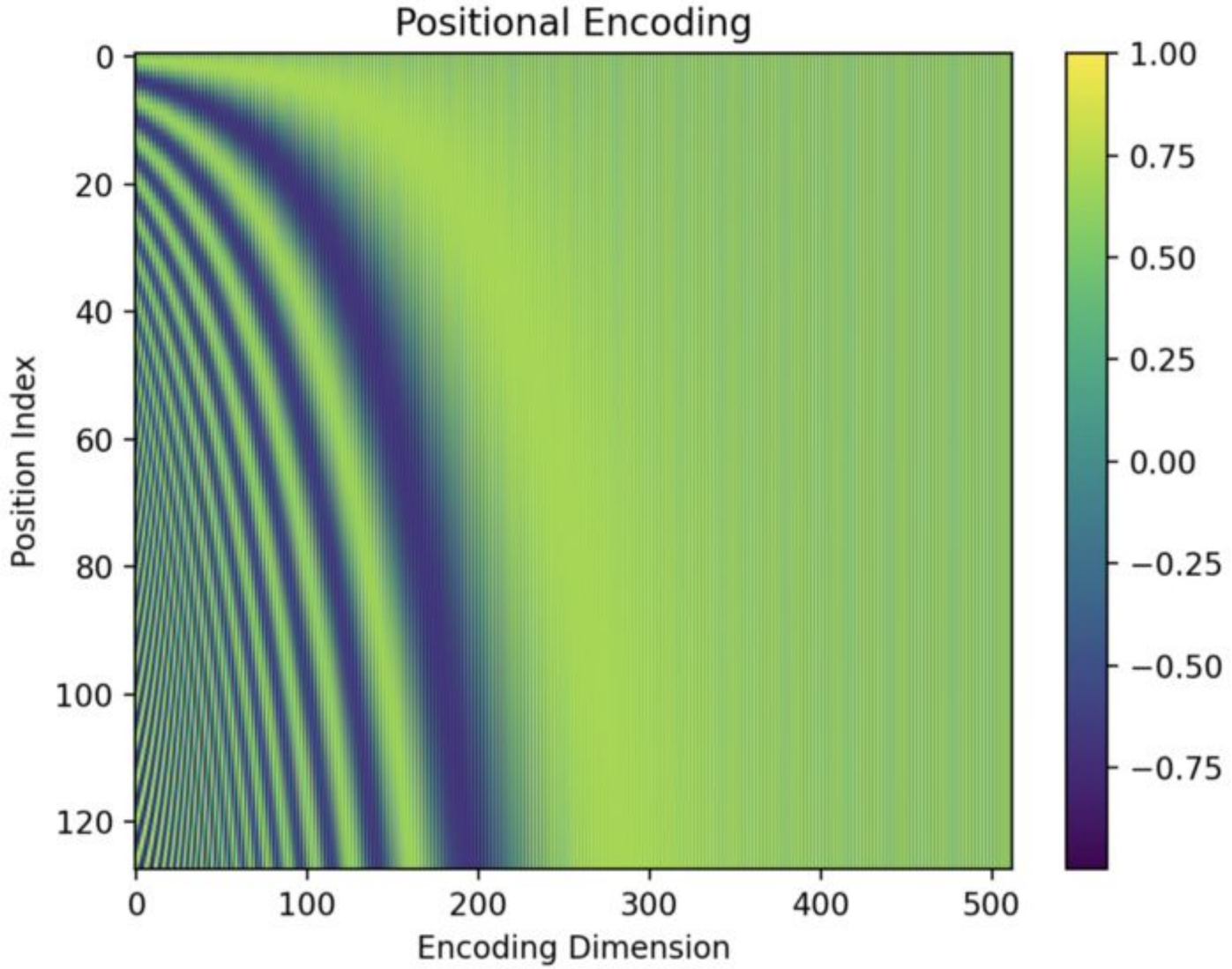
$$PE_{(pos,\ 2i+1)} = cos(pos\ /\ 10000^{2i\ /\ d_{\text{model}}})$$

- *pos* is the position
- *i* is the dimension.

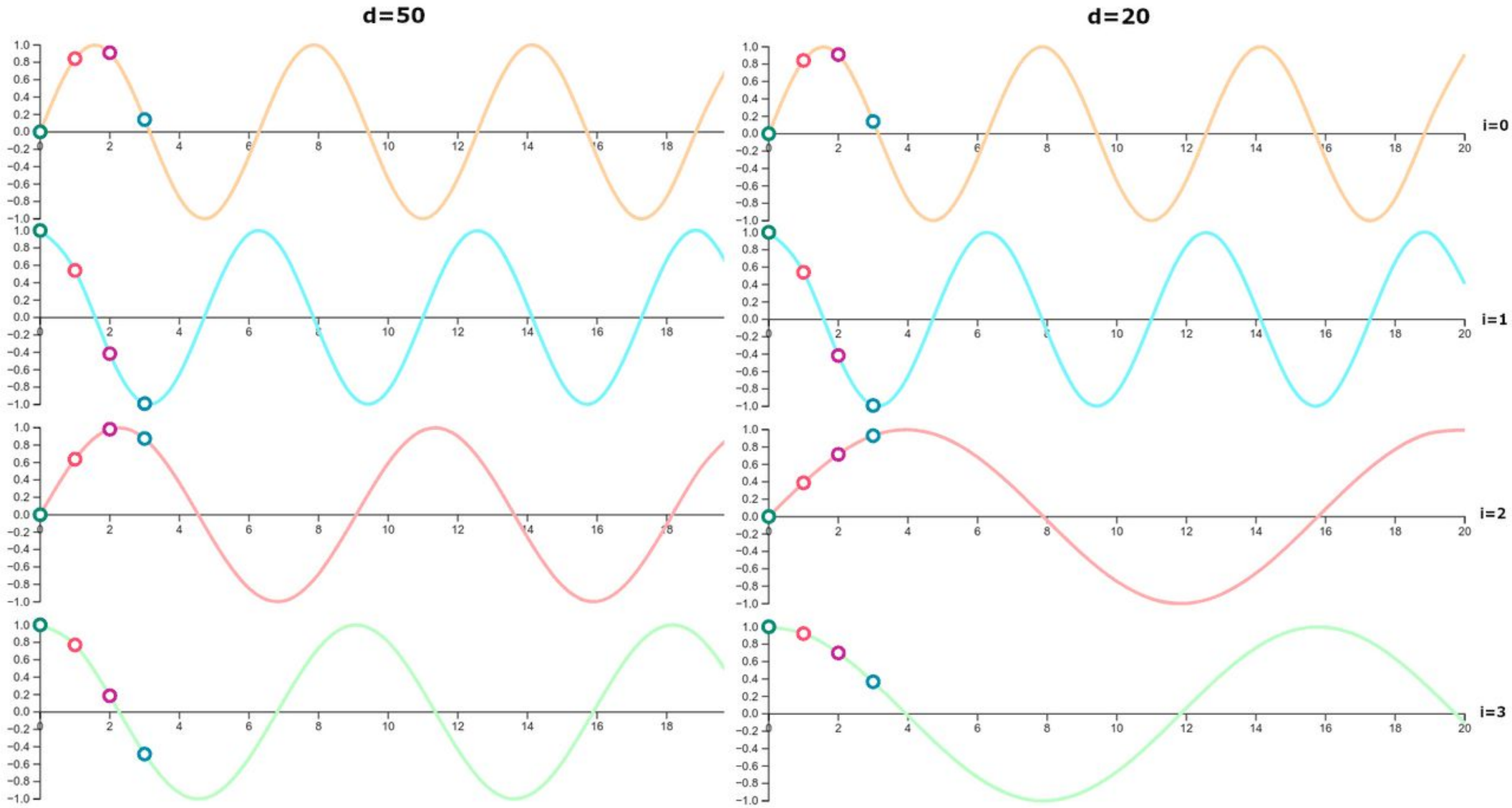
Each dimension of the positional encoding corresponds to a sinusoid.

The wavelengths form a geometric progression from 2pi to 2pi * 10 000

Positional Encoding



Positional Encoding



Positional Encoding

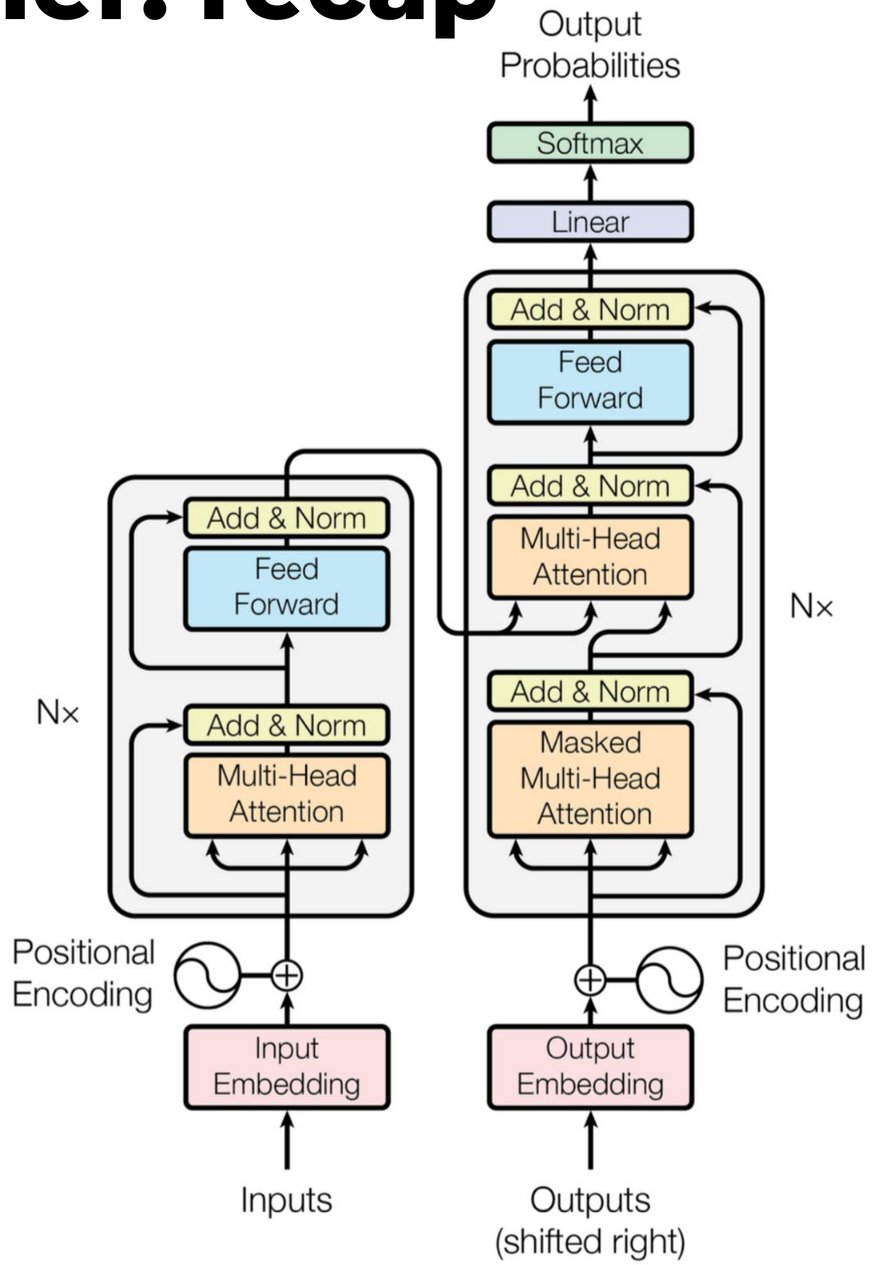


Layer Normalization

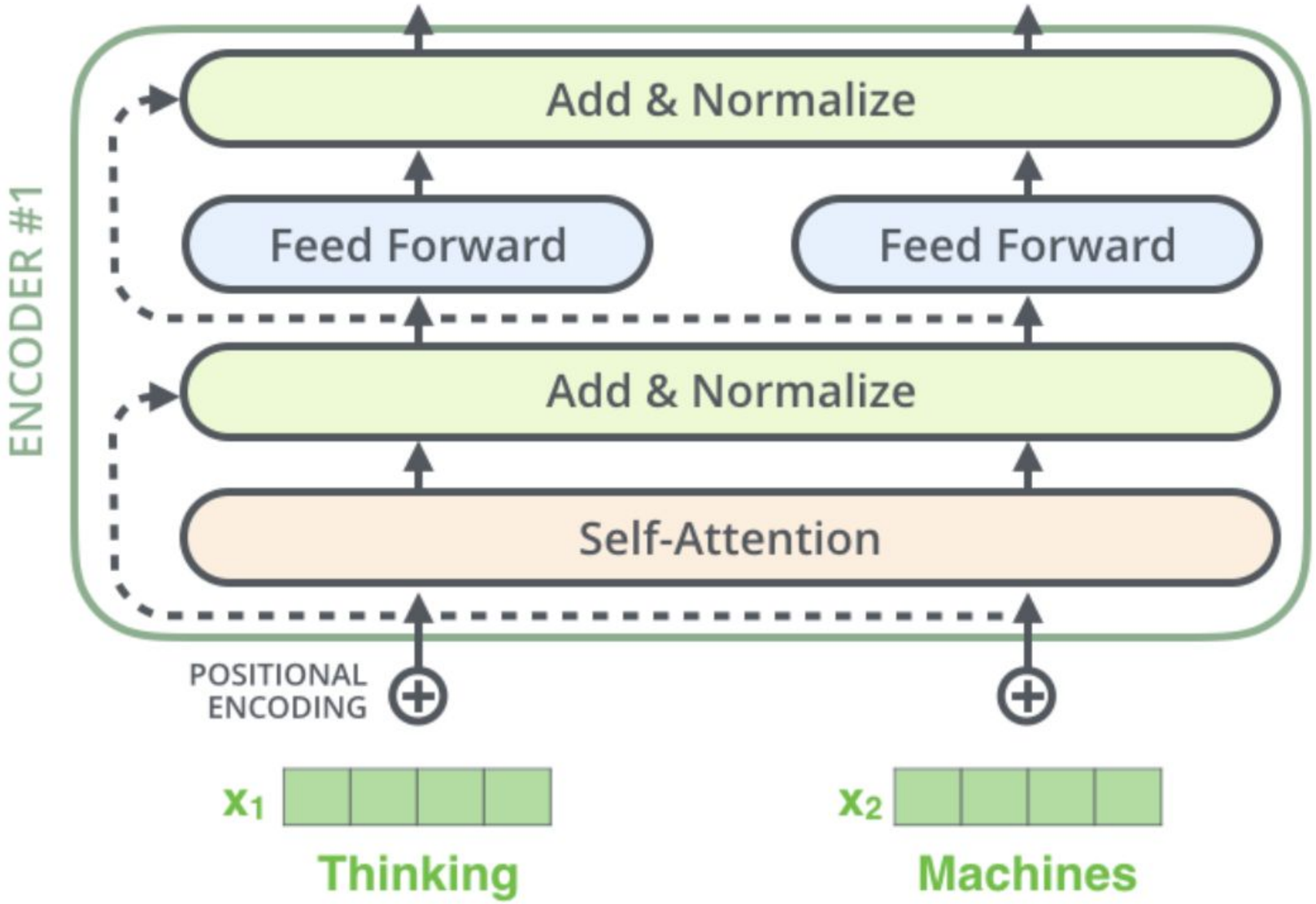
girafe ai

07

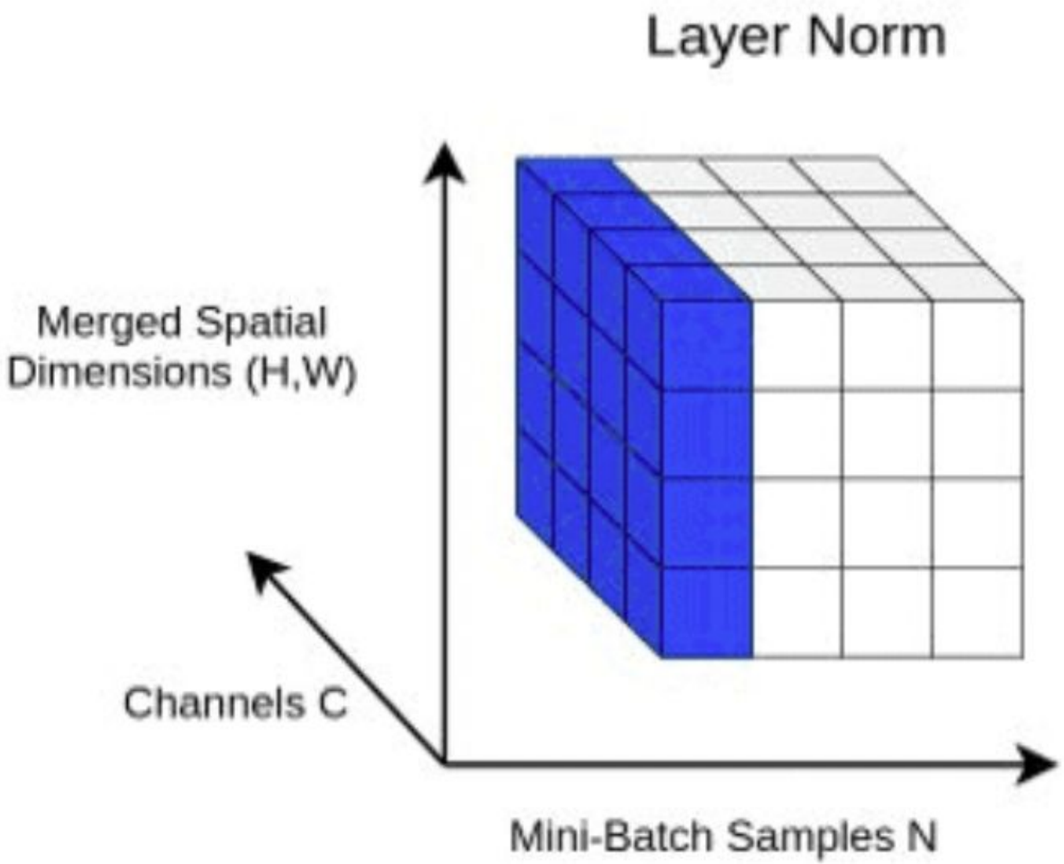
The Transformer: recap

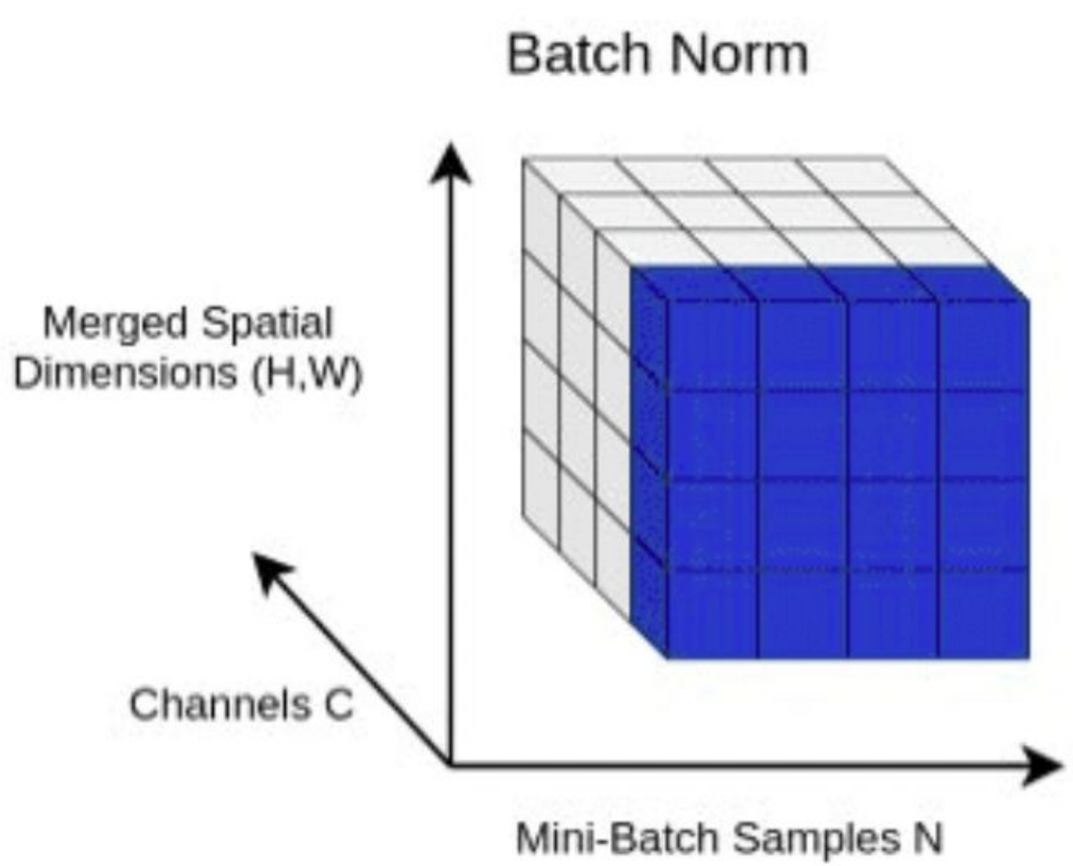


Layer Normalization

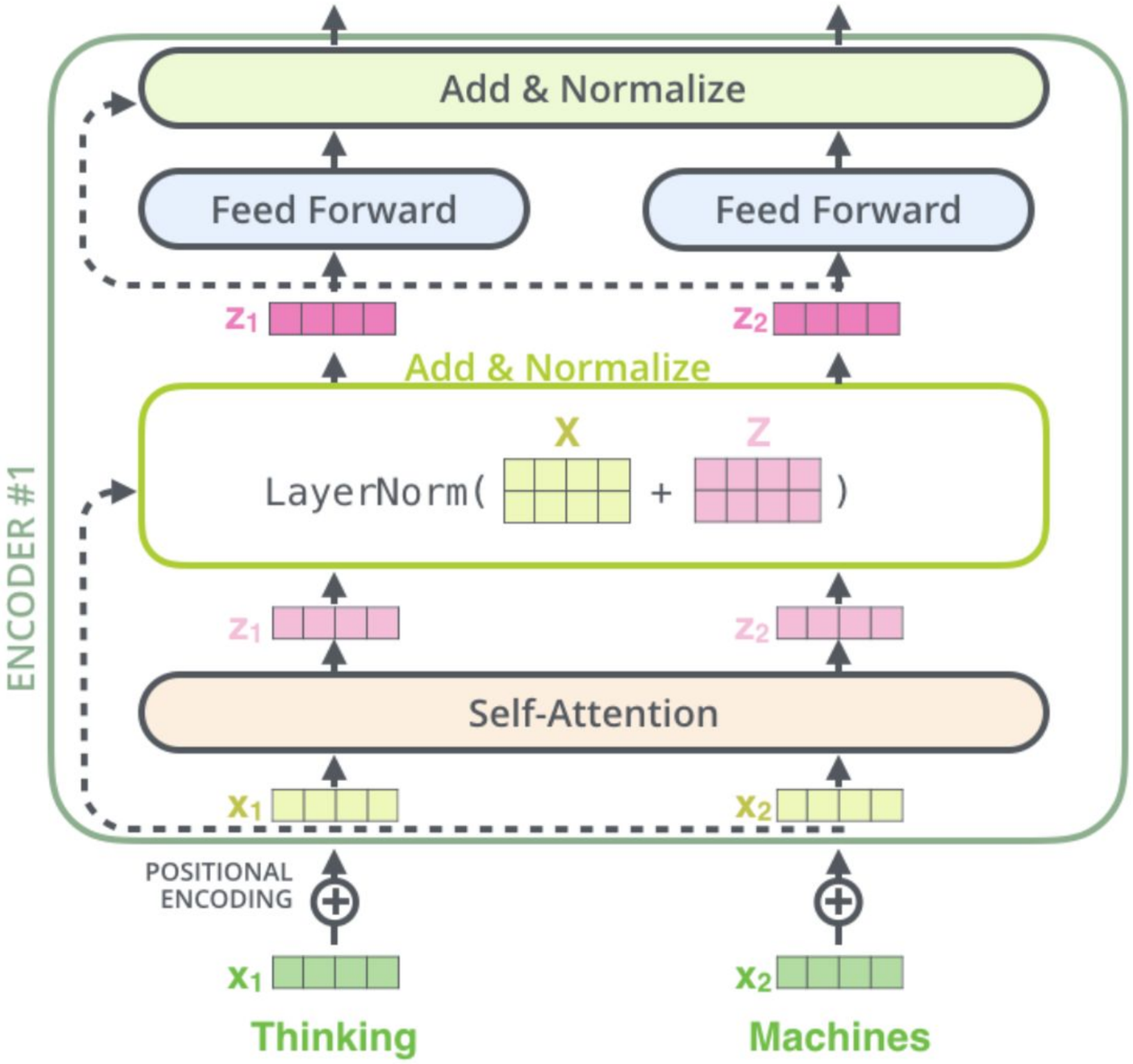


Layer Norm vs. Batch Norm

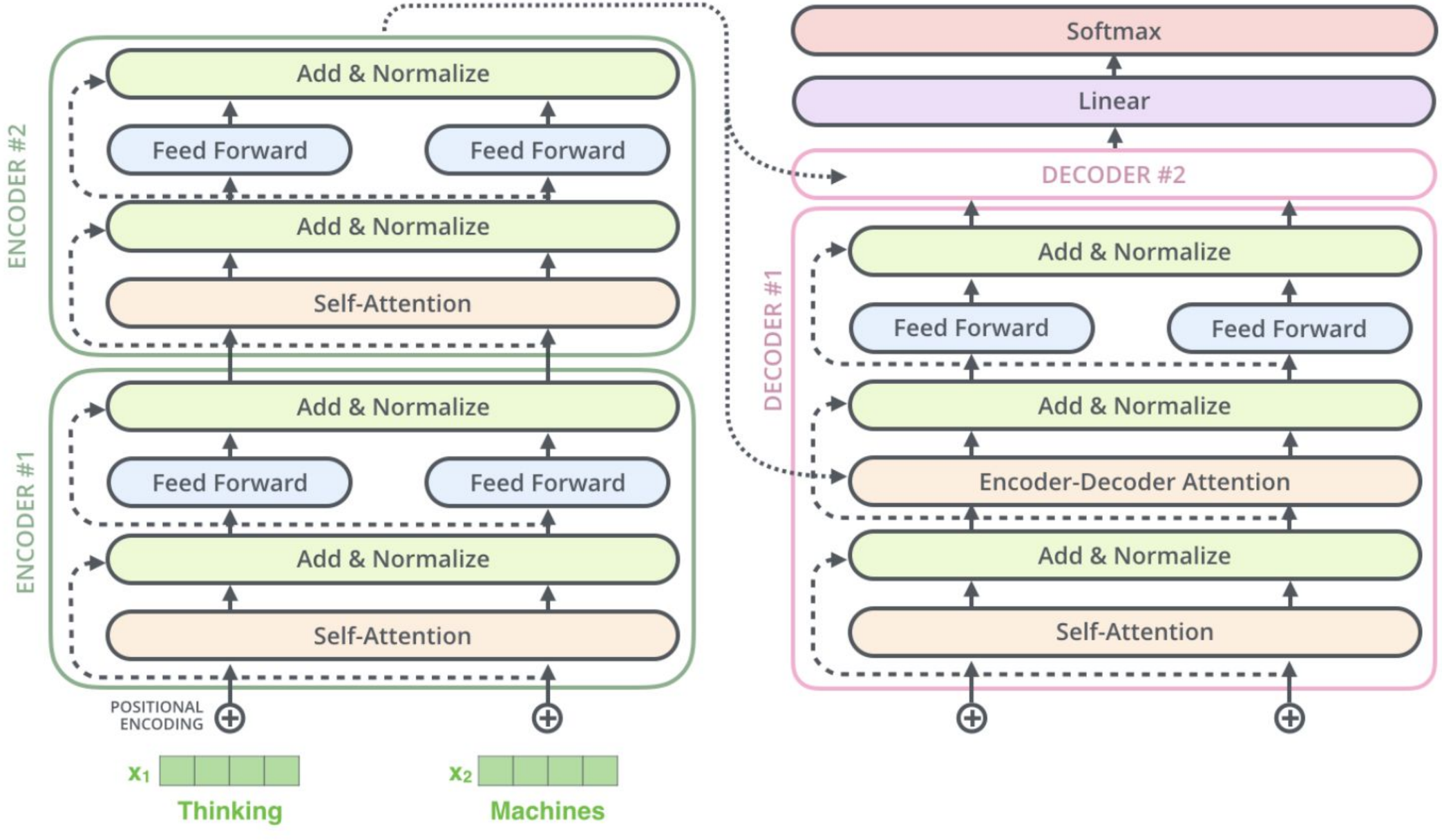




Layer Normalization



Layer Normalization

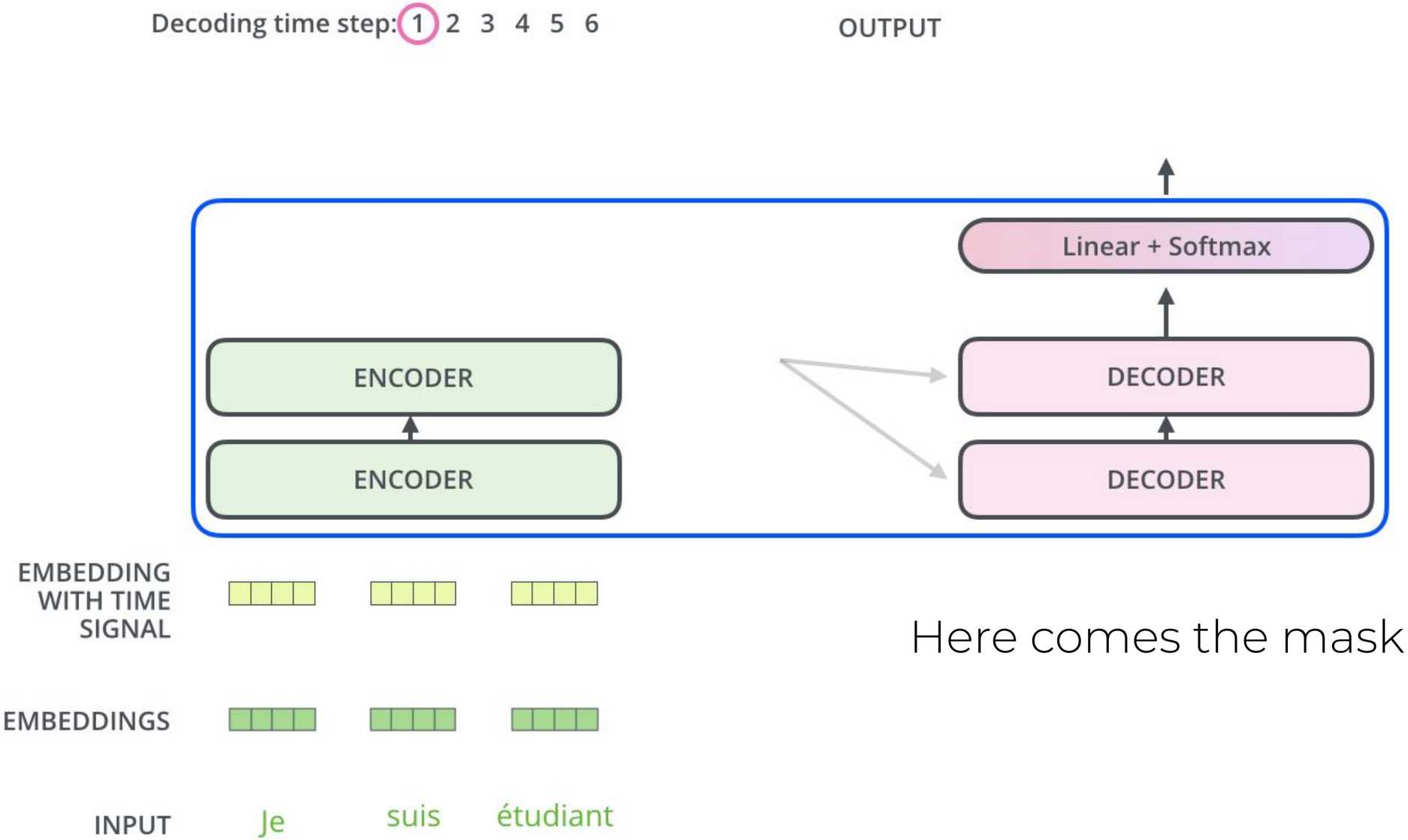


The Decoder

girafe ai

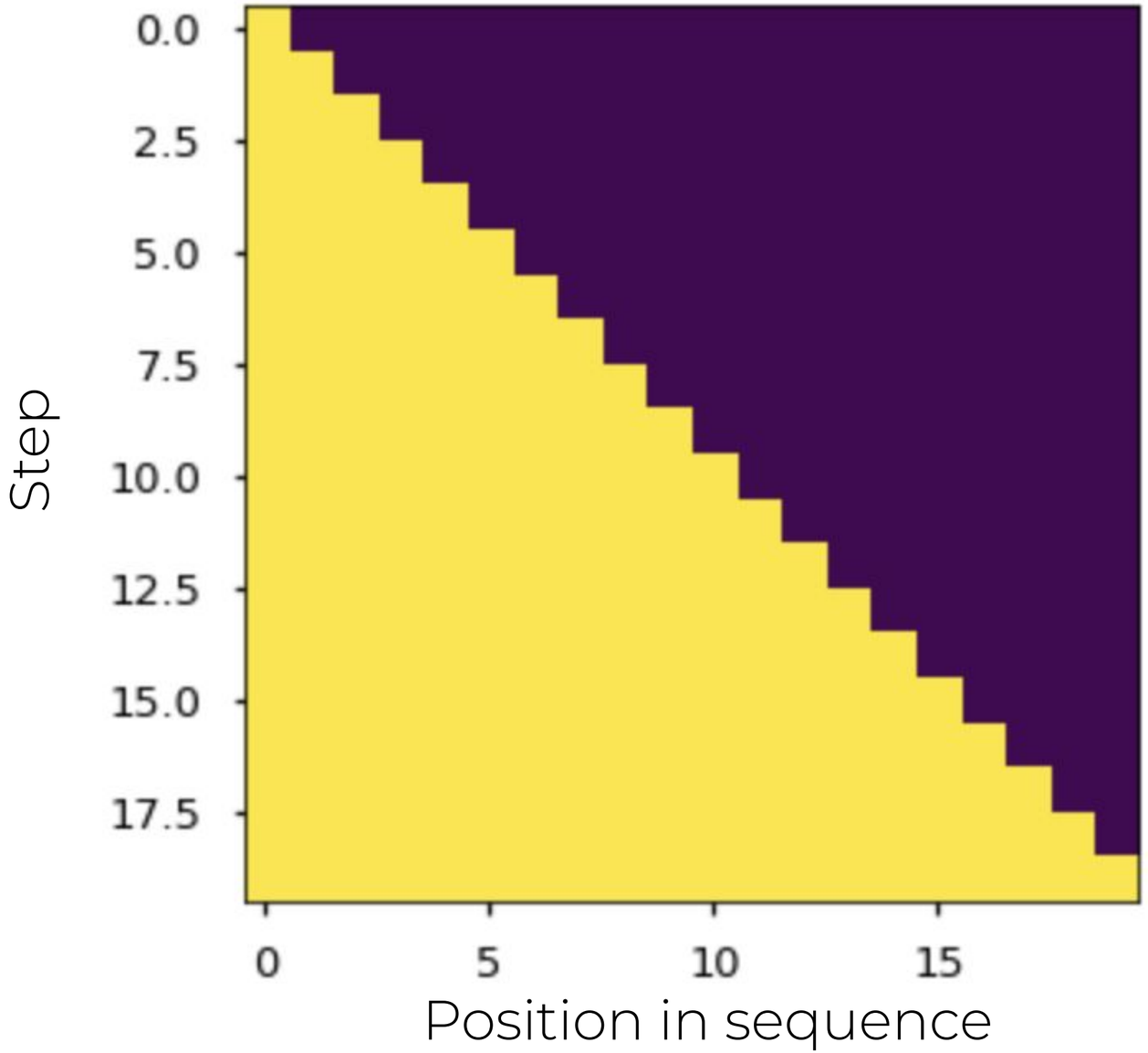
08

The Decoder Side

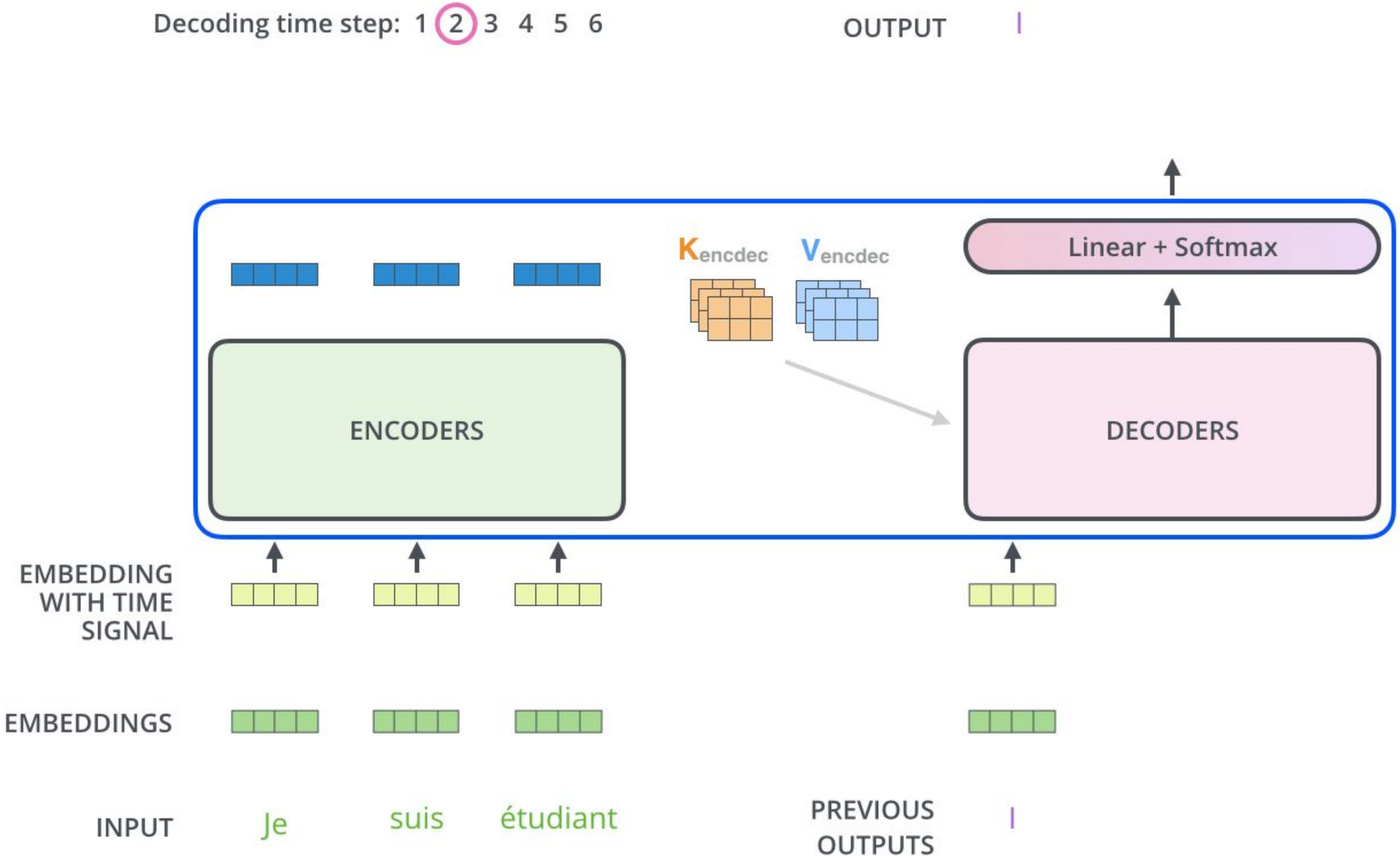


Here comes the mask

The masked decoder input



The Decoder Side



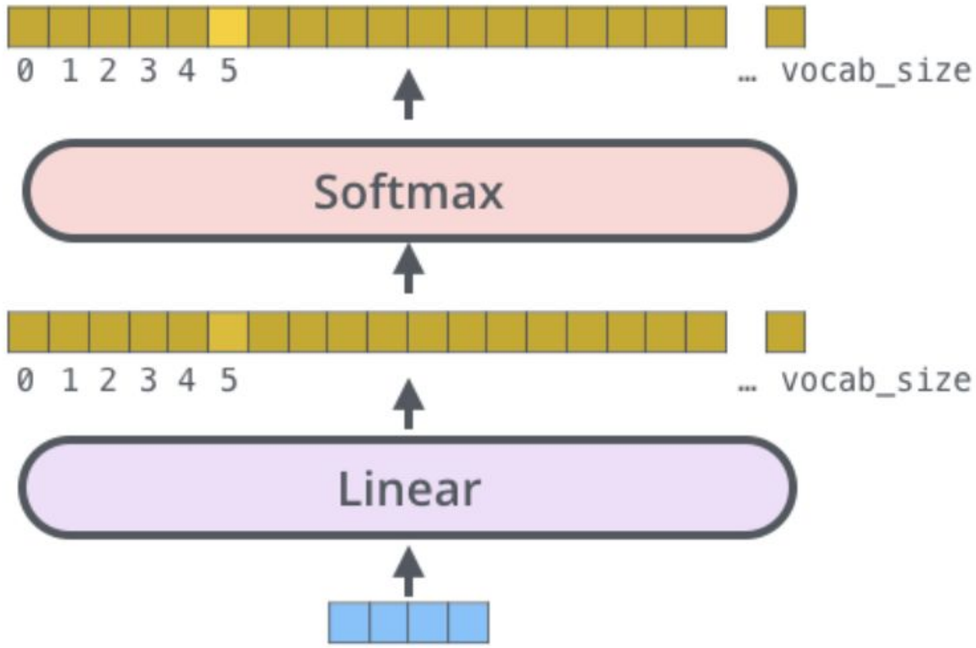
Final Linear and Softmax Layer

Which word in our vocabulary is associated with this index?

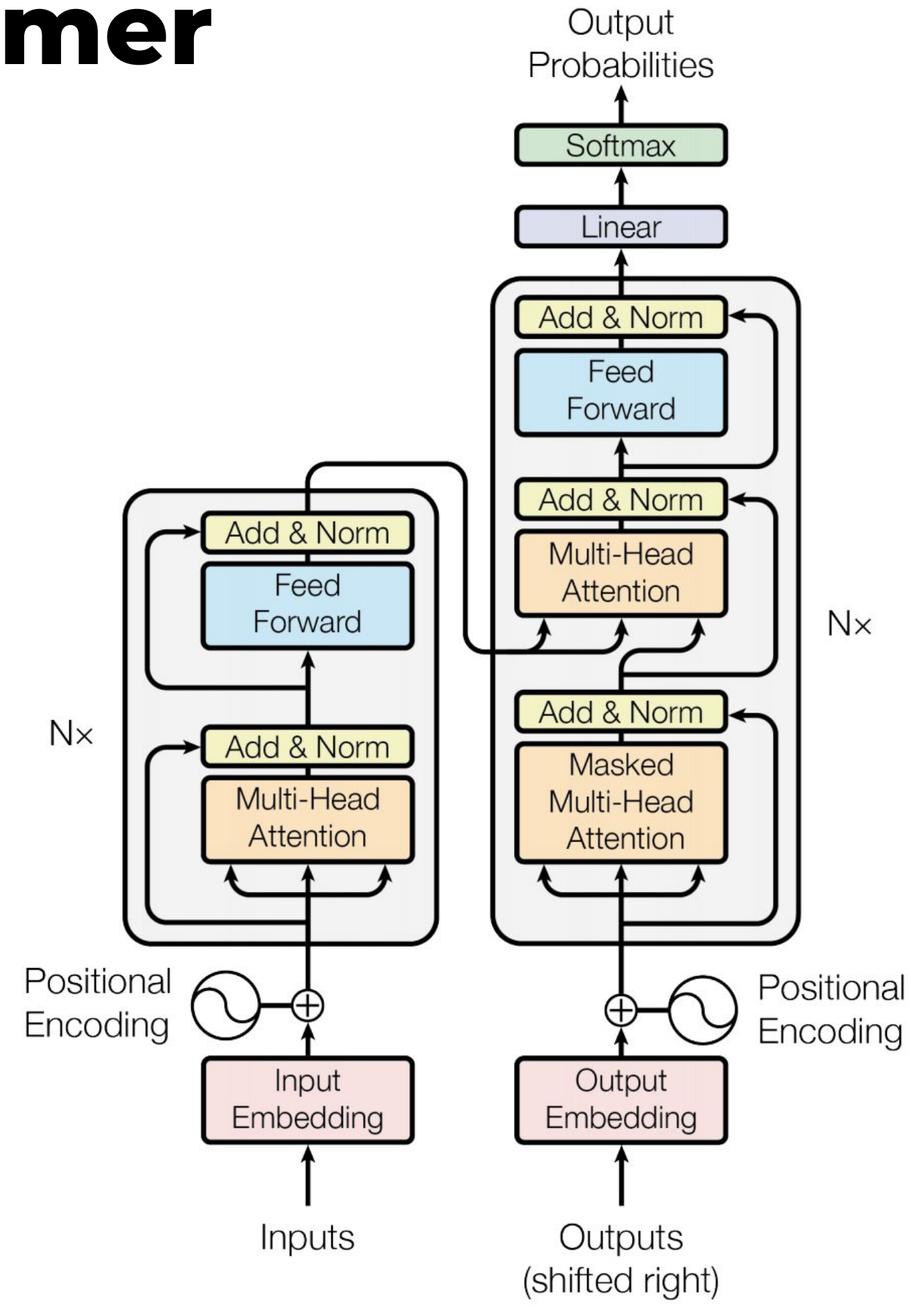
am

Get the index of the cell with the highest value (**argmax**)

5



The Transformer



Outro and Q&A

- Transformer is a very powerful architecture based on self-attention mechanism
- Physical analogues can help you

- Further readings are available in the repo

BERT

It’s encoder-only model based on self-attention

https://github.com/v-goncharenko/madmo-adv/blob/madmo-adv-21-10/08_attention_nmt/madmo_08_attention_transformer_bert.pdf

Writing your own LLM

- https://github.com/karpathy/nanoGPT
  - https://github.com/karpathy/build-nanogpt
  - https://www.youtube.com/watch?v=l8pRSuU81PU
- Deep Seek from scratch
  - https://www.youtube.com/watch?v=QWNxQIq0hMo&list=PLPTV0NXA_ZSiOpKKlHCyOq9lnp-dLvlms&index=3
-

Materials

- https://aman.ai/primers/ai/transformers/
- https://jalammar.github.io/illustrated-transformer/
- Consistent mathematical description
  - https://arxiv.org/pdf/2207.09238
-
- https://arena-chapter1-transformer-interp.streamlit.app/
-

Thanks for attention!

Questions?



girafe
ai